

**EVALUATING NLP AND DEEP LEARNING APPROACHES FOR
FORECASTING OF INTRADAY PHILIPPINE STOCK MARKET
REACTIONS USING HETEROGENEOUS DATA SOURCES**

Presented to the
Faculty of the Data Science Program
Department of Mathematics
School of Sciences, Engineering, and Technology
University of Asia and the Pacific

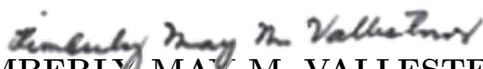
In Partial Fulfillment of the
Requirements for the Degree of
Bachelor of Science in Data Science

By
Marcus Joshua T. Tornea

July 2026

APPROVAL SHEET

The capstone entitled EVALUATING NLP AND DEEP LEARNING APPROACHES FOR FORECASTING OF INTRADAY PHILIPPINE STOCK MARKET REACTIONS USING HETEROGENEOUS DATA SOURCES, prepared and submitted by Marcus Joshua T. Tornea in partial fulfillment of the requirements of the degree of BACHELOR OF SCIENCE IN DATA SCIENCE, is approved.


MS. KIMBERLY MAY M. VALLESTEROS

Faculty in-charge

PANEL


MR. ELMER C. PERAMO

Capstone Adviser


MS. MILLICENT B. SINGSON

Panelist


MR. CAMILO CARLO B. MENDIOLA

Panelist



UNIVERSITY OF ASIA AND THE PACIFIC
LIBRARY

Annex I.

Declaration of Authenticity by Thesis Writer

I, Marcus Joshua T. Tornea

Name of Thesis Writer

hereby certify that the soft copy of my thesis entitled:

Evaluating NLP and Deep Learning Approaches for Forecasting of Intraday Philippine
Stock Market Reactions Using Heterogeneous Data Sources

is an exact copy of the approved hardbound manuscript/work.

I make this Certification under pain of administrative and/or academic sanction/s in the
event that what I certify is untrue.

A handwritten signature in black ink, appearing to be 'MJT', written over a horizontal line.

Marcus Joshua T. Tornea
Printed Name & Signature

August 4, 2026

Date

Annex 2

AUTHORIZATION BY AUTHOR

I, Marcus Joshua T. Tornea, the author of the work/thesis entitled:

Evaluating NLP and Deep Learning Approaches for Forecasting of Intraday
Philippine Stock Market Reactions Using Heterogeneous Data Sources

hereby grant non-exclusive rights to the University of Asia and the Pacific Foundation, Inc., (UA&P) represented herein by the UA&P LIBRARY, the non-exclusive right:

1. To store, catalogue, and archive in the UA&P Library Collection, and conserve the same;
2. To make changes in the format of the work for purposes of long-term storage and electronic communication as described herein;
3. Subject to the manner and conditions described herein, to communicate the work to the public using digital or electronic means on the World Wide Web, in PDF or Adobe format, to be undertaken under the auspices and control of UA&P or its duly authorized electronic/technical resources;

A. Conditions—With this Authorization, the following conditions apply:

1. The Work/Thesis shall be uploaded to the Library database only and viewed from its website accessed via the University Library by *bona fide* students and employees of UA&P and other researchers, with the corresponding fee fixed and enforced by the Library.
2. The abstract and the preliminary pages shall be made accessible over the World Wide Web through the university library website after electronic acceptance of *Terms of Use* (TOU) by all users.
3. The Full Text of the Work/Thesis may, at the sole discretion of UA&P be made accessible to UA&P patrons, other researchers, and other users over the World Wide Web exclusively accessible only via pre-approved subscription online and with electronic acceptance of *Terms of Use* (TOU) by all users.
4. UA&P shall install the appropriate software that will electronically block:
 - (a) Object linking and embedding function
 - (b) Storage into any user's storage devices
 - (c) Printing

and ensure that copyright restrictions are on view to the public.

B. Author's Undertaking and Warranty—As author of the work, I undertake and represent:

1. To submit to the UA&P Library a soft copy of the above work in addition to its hard copy, as approved by the Dean, ensuring that the copy submitted follows the guidelines for submission and is identical, in all respects, to the original approved work.
2. That the Thesis/Feasibility Study is original work and does not infringe on any intellectual property rights of others;
3. That I own full copyright over the work with full publishing rights;
4. That I am solely responsible for the contents of the work, including all supplementary materials;
5. That I hold the UA&P Library and UA&P Foundation free and harmless from any and all claims arising from any violation of intellectual property rights.
6. That I have the moral right under the Law to execute this Authorization in favor of UA&P;

Pasig City, Philippines, August 4, 2026.



Marcus Joshua T. Tornea
Signature over Printed Name
Author

Lot 6, Blk. 6, Moonlight Loop, Ridgemont Executive

Village, Brgy. San Isidro, Taytay Rizal, 1920
Address

Annex 3**TERMS OF USE**
For users of UA&P Online Resources

Use of the UA&P Library resources does not give users ownership of any intellectual property rights in its Services, or the content accessed. Users may not use content from our Services unless permission is obtained from its owner or the use is otherwise permitted by law under the principle of Fair Use in the Intellectual Property Code, and in all cases, the user shall exercise proper attribution of authorship of resources.

The UA&P Library, its officers, agents, and employees shall not be responsible for lost profits, financial losses or indirect, special, consequential, exemplary, or punitive damages that may arise in the course of the use of its Services or the content accessed, and are held by the user to be free and harmless from any claim, suit or action arising from or related to the use of the Services or violation of these terms, including any liability or expense arising from claims, losses, damages, suits, judgments, litigation costs and attorneys' fees.

☐ I accept.

DEDICATION

To my parents and my family, for their support, love, and guidance.

To my partner, Liah, who was there when this work began as a simple idea over a coffee table, and never stopped believing in me until the very end.

ABSTRACT

Predicting the stock market is a challenging task, given by constantly shifting market volatility regimes, market efficiency, and overall low signal. Even a small capacity at predicting the stock market is already a significant advantage; thus, using fast, automated predictive methods has been researched thoroughly for the possibility of gaining a competitive edge in the market. Uniquely engineered deep learning and NLP (natural language processing) methods have become common pathways of novel research, exploiting the flexibility of deep learning architectures to support unique data configurations, which opens questions about the general advantage of certain kinds of deep learning solutions from others. In this study, certain design factors in building an intraday (per-minute) stock return direction classifier using deep learning and NLP methods, such as *overall pipeline complexity* (transformed-based vs. MLP-based pipelines) and *prediction horizon*, as well as text modality inclusions such as the *inclusion of news data* and *inclusion of social media data* are analyzed by quantifying their main and interaction effects on three model evaluation metrics—raw machine learning performance, stock confidence ranking behavior, and temporal stability of prediction confidence—through a factorial experiment. Prediction horizons have the single strongest effect, showcasing that a switch from 10-minute to 30-minute returns as the prediction target diminishes prediction capacity. Overall pipeline complexity and the inclusions of news data and social media data do not exhibit dominating main effects, but have significant interacting mechanisms, showcasing that combinations of design configurations matter more than individually varying settings.

ACKNOWLEDGMENT

I would like to express my deepest gratitude to my parents, for providing me the motivation, guidance, and tahe required financial support for proprietary data collection and access to cloud computing services. I would also like to thank my institution for providing an avenue for collecting the required financial intraday data across a vast range of financial markets both locally and internationally. I also thank my panel, professor, and thesis adviser for their patience and guidance in identifying the strengths and weaknesses of my overall work. Finally, I would like to express my most heartfelt gratitude to my partner for her emotional and moral support, and for her company, both in times of greatest need and in celebration of success.

Contents

Approval Sheet	ii
Annex 1: Declaration of Authenticity by Thesis Writer	iii
Annex 2: Authorization by Author	iv
Annex 3: Terms of Use	vi
Dedication	vii
Abstract	viii
Acknowledgement	ix
List of Tables	xv
List of Figures	xvi
List of Abbreviations and Acronyms	xvii
Chapter 1: Introduction	1
Background of the Study	1
Statement of the Problem	5
Objectives of the Study	8
Significance of the Study	10
Scope and Limitations	12
Chapter 2: Review of Related Works	15
Predictability in Efficient Markets	15
The Efficient Market Hypothesis	15
Mechanics behind Financial Markets	16

The Fate of Prediction Algorithms	21
Review of Statistical, Machine Learning, and Deep Learning Methods	27
Autoregressive Integrated Moving Average (ARIMA)	29
Exponential Smoothing Models	30
Support Vector Machines	31
Logistic Regression	32
Random Forest	33
Extreme Gradient Boosting (XGBoost)	33
Feed-forward Neural Networks	34
Long Short-Term Memory (LSTM) Networks	35
Transformers	38
Methodologically Informative Studies	43
Data Structure and Preprocessing	44
Modelling	65
Evaluation	77
Summary of Gaps in the Literature	80
Chapter 3: Methodology	82
Research Design	82
Data Generation	92
Data Treatment	92
Chapter 4: Data Understanding	94
Financial Time-series Datasets	94
News Data	109
Social Media Data	111
Chapter 5: Data Preprocessing	115

Chapter 6: Modelling	119
Notation	119
The Differentiable Top-K Operator	121
Forward Pass: Monte Carlo Smoothing	121
Backward Pass: Perturbed Optimizer Gradient Estimator	122
Time2Vec	122
Input Embeddings	123
FinEmbedding (Stock Token Embedding)	123
NewsEmbedding	123
SocialEmbedding	124
Self-Attention Building Blocks	124
SelfAttentionBlock	124
FeedForward	125
SelfAttnTransformerLayer and Stacking	125
StockTransformer (base model)	125
Dynamic Auxiliary-Item Selection (DynamicSelection)	126
Selective Cross-Attention (CrossAttentionBlock)	127
StockNewsTransformer	129
StockSocialTransformer	129
StockNewsSocialTransformer	129
Baseline Non-Transformer Model	130
HiddenLayer	130
MultiLayerPerceptron	130
Datasets and Temporal Windowing	130
Base Stock Windows (StockTransformerDataset)	130
Causal Auxiliary Evidence Windows (News / Social Media)	131
Flat Baseline Dataset (StockMLPDataset)	131

Batch Collation	132
Operationalization of Datasets and Models	132
Class-Imbalance-Weighted Objective	133
Optimization Procedure	133
Optimizer and Gradient Accumulation	133
Learning Rate Schedule	134
Perturbed Top-K Temperature Annealing (SigmaAnnealer)	134
Early Stopping	134
Checkpointing	135
Validation	135
Model Training Operationalization	135
Chapter 7: Evaluation	136
Notation	136
The Matthews Correlation Coefficient Curve	136
Threshold Calibration	137
Static, Per-stock Calibration	137
Expanding-window (Test-time-adaptive) Calibration	137
Testing and Metric Computation	137
Model Drift Computation	138
Statistical Analysis of Model Performance	139
Classical Machine-Learning Baselines	139
Trading Simulation	140
Chapter 8: Results and Discussion	141
MCC Model Performance	143
Model Drift	148
Trading Simulation Model Performance	152

Chapter 9: Conclusions	159
Chapter 10: Recommendations	162
Recommendations for Future Research	162
Recommendations for Practitioners	165

List of Tables

1	Objectives Matrix	9
2	chrF++ Translation Quality Scores (to English)	88
3	All Financial Datasets Constituting All Technical Time-series Data	94
4	Relevant Raw Variable Columns for All Time-series Datasets	98
5	Relevant Raw Variable Columns for Social Media Data	113
6	Model Performance and Drift Descriptives	141
7	Regression Results (DV: MCC)	143
8	Mean MCC of Best Deep Learning Model vs. Machine Learning Baselines	146
9	Regression Results: MCC by Model Type (Reference: Stock MLP News 10)	147
10	Regression Results (DV: Drift Score)	148
11	Mean Combined Drift Score by Model Type: Best Deep Learning Model vs.	
	Machine Learning Baselines	150
12	Regression Results: Combined Drift Score by Model Type (Reference: Stock	
	Social MLP 30)	151
13	Regression Results: Trading Simulation Cumulative Profit	154
14	Regression Results: Trading Simulation Cumulative Profit, Best DL Model	
	vs. ML Baselines	157

List of Figures

1	PSEi Price and Volume	101
2	PSFI and Constituent Stocks' Price and Volume	102
3	PSHO and Constituent Stocks' Price and Volume	103
4	PSIN and Constituent Stocks' Price and Volume	104
5	PSMO Price and Volume	105
6	PSPR and Constituent Stocks' Price and Volume	105
7	PSSE and Constituent Stocks' Price and Volume	106
8	Government Bonds' Prices	107
9	COMEX Continuous Copper Futures' Prices	108
10	Brent Crude Oil Continuous Futures' Prices	108
11	PHP to USD Exchange Rate	109
12	PHP to Gold Exchange Rate	110
13	Top Non-English News Headline Languages	110
14	News Sentiment Distribution	112
15	Social Media Sentiment Distribution	113
16	Top Relevant Features from mRMR for 10-minute Prediction Horizon Models	117
17	Top Relevant Features from mRMR for 30-minute Prediction Horizon Models	118
18	Cumulative Trading Profit over Time per Experimental Setting	153
19	Cumulative Trading Profit over Time for The Best DL Model Compared	
	against ML Baselines	156

LIST OF ABBREVIATIONS AND ACRONYMS

NLP	Natural Language Processing
LLM	Large Language Model
ML	Machine Learning
DL	Deep Learning
GRU	Gated Recurrent Unit
GDP	Gross Domestic Product
LSTM	Long Short Term Memory
PSE	Philippine Stock Exchange
AI	Artificial Intelligence
MCC	Matthew's Correlation Coefficient
PHP	Philippine Peso
USD	United States Dollar
LSEG	London Stock Exchange Group
BERT	Bidirectional Encoded Representations from Transformers
SHAP	SHapley Additive exPlanations
LIME	Local Interpretable Model-agnostic Explanations
EMH	Efficient Market Hypothesis
CLOB	Central Limit Order Book
OHLCV	Open High Low Close Volume
ARIMA	Autoregressive Integrated Moving Average

Chapter 1: Introduction

Background of the Study

Predicting the stock market is an inherently difficult task (Yang et al., 2022). Stock prices fluctuate based on a combination of social, economic, and geopolitical factors, as well as other influences that themselves vary with uncertainty across time. The ability to predict stock prices is a practical advantage in the finance sector; therefore, studying stock market patterns, despite the difficulty, is a worthwhile pursuit. This has led to numerous techniques, both heuristic and computational in nature, that aim to predict stock prices. Historically, the focus has shifted from traditional statistical methods to machine learning (ML) techniques, which offer greater flexibility for the complexity of the task (Darwish et al., 2025). Furthermore, developments into deep learning (DL) have allowed for even greater flexibility, with the ability to process unstructured textual data through the use of complex natural language processing (NLP) approaches such as large language models (LLMs) and structured textual data with models such as gated recurrent units (GRU) (Puh & Babac, 2023), which outperform traditional approaches such as sentiment analysis.

A comprehensive list of factors that contribute to stock market prices would be conceptually vast, including local and global events of many kinds. Relevant numerical data include rates of gross domestic product (GDP) growth, inflation rates, employment levels, and company performance metrics, while other non-numeric factors include the occurrence of earthquakes, tropical cyclones, pandemics, and other external shocks that cause any scale of economic disruption. Other relevant events include elections, government policy changes, and trade activity (Thompson, 2024). In spite of their interdomain scope, a common denominator among these factors is the availability of information in news sources, which are intended to collate all local and global events into a centralized space. It is therefore likely that the news is a central hub of influence, and given that it directly evokes investor sentiment, affects decision-making (Puh & Babac, 2023), and—to some degree—describes the landscapes of the relevant domains, it is reasonable to prioritize the

news in a comprehensive attempt to predict stock market reactions. This, coupled with the fact that NLP and deep learning are becoming more developed and accessible technologies (Feng et al., 2025) that can capture much more complex structures than traditional solutions, aligns with the growing interest in financial forecasting through the use of NLP and deep learning methods trained on news sources. For instance, methods such as Long Short-Term Memory (LSTM) models have been used for company-level forecasting from newspaper data spanning about a decade (Akita et al., 2016). Similar reasoning can be used to justify the inclusion of public social media sentiment, as it is an all-encompassing source, as demonstrated in a proposed hybrid method combining traditional machine learning and deep learning on historical stock price data and social media sentiment (Mehtab & Sen, 2019).

A similar trend can be observed in the national landscape. Proposed methods for forecasting the Philippine Stock Exchange (PSE) at various levels—from individual company-level stock prices to the PSE index (PSEi)—have been published, including a neural network-based algorithm trained on stock prices (Barrios & Gerardo, 2024), an LSTM model trained on a mixture of data including news sentiment (Sombrito et al., 2023), and a time-series approach with prediction horizons spanning a year (Lacanilao & Signo, 2021). Thus, the models presented in the literature are varied, each with a methodological differentiating factor—signaling an active national research interest in the topic of modelling the PSE, albeit no industry-standard or leading candidate solutions exist yet. This growing literature benefits a vast array of stakeholders—including policy makers and investors at different levels of investing capabilities. Policy makers can indirectly benefit from stock prediction models as they rely on future overall market trends to create well-informed and data-driven policies, while institutional investors such as banks and funds can also indirectly benefit from the supplementary information it can provide for portfolio optimization and risk forecasting.

The main stakeholder, however, who directly benefits from these solutions, is

medium-frequency investors and institutional investors, namely both professional and non-professional individuals who invest their own money for their own benefit, or for others. In particular, this study focuses on medium-frequency investment, whose participants may rely on country-level indices like the PSEi—a nationwide metric reflecting the general performance of top national companies—to decide on whether to focus their investment activities on the local market. They may also prefer a more selective approach relying on sectoral indices—industry-level metrics—to decide on an optimal industry on which to focus their budgets. At an even more basic level, they may also rely on company-level stock prices to find an individual company to invest in. This hierarchical approach makes the stock market an accessible playing field for investors at any level.

As the literature on stock market prediction grows, so do the mainstream tools available at the investors' disposal, as well as the quality of those tools, allowing them better access to enhancement opportunities for their investment decisions. These tools, however, have yet to establish the capacity to deeply focus on an emerging market such as the Philippines. Industry solutions are based on general and worldwide data, which, albeit useful for international stock market investment, may not be suited for investors in the PSE. Stock prediction features that may be found within Philippine-based banking and finance apps might already feature some fine-tuning on Philippine-based information sources, but a state-of-the-art/industry-standard solution for the PSE remains elusive. Thus, retail investors in the PSE currently do not have a widely accepted and accessible solution optimized for the local stock market. Additionally, existing solutions in Philippine-based studies have yet to tackle the aspect of different prediction horizons that may be useful for different kinds of retail investors. For instance, day traders or medium-frequency traders may rely on intraday-level granularity in predictions, such as by-minute or hourly forecasts, and most retail investors rely on lower-frequency predictions spanning weeks, months, or years. The flexibility of optimization for different prediction horizons required to address this varying need has not been a subject in the literature,

further limiting available solutions for retail investors in the PSE. Lastly, the nature of the stock market being unpredictable and highly time-dependent makes it difficult to avoid model drift, whereby predictions become increasingly inaccurate the farther in the future they are made. Despite the research activity on the topic, most studies only aggregate accuracy, disregarding the variation of errors over time, while only a few make it a point to evaluate model drift, which may mislead retail investors into relying on a solution that is reliable on paper as the study was made, but may fail in practice in the future.

A further discussion of the ideal attributes of such solutions covers more than the aforementioned. The current discussion herein has implied that fine-tuning of such solutions to the national landscape is ideal, given that stock market behavior is sensitive to geographical location, creating a need for location-specific relevance. Secondly, optimized solutions for different prediction horizons are ideal for varied investor use cases, addressing a need for flexibility. It also has been so far implied that when paired with DL and NLP architectures with high enough representational capacity, multimodal data integration—approaches that incorporate heterogeneous data types (news sentiment, social media sentiment, numerical data, etc.)—addresses the need for robustness, created by the fact that regime shifts—widespread changes such as new policies, interest rate hikes, large-scale economic disruptions (i.e. the COVID-19 pandemic)—moderate the underlying mechanisms of stock price movement. Such underlying interactions are what contribute to model drift, further highlighting the need for robustness; thus, the more a solution exhibits stable prediction accuracies over time, the more ideal it is. For all these reasons, access to stock prediction tools with high representational capacity, flexible prediction horizons, and stable forecast accuracies, based on multimodal data integration with a special focus on the Philippine landscape, is ideal among Filipino investors and stockholders.

At the national level, however, the quality of stock prediction tools has its limitations. On top of the aforementioned drawbacks on national-level fine-tuning, flexibility over prediction horizons, and robustness against model drift, most methods in the literature

that rely on deep learning approaches are black box approaches, where predictions are made but the underlying mechanisms remain unstudied. Moreover, while multimodal approaches that incorporate heterogeneous data types (news sentiment, numerical data, etc.) are commonly explored in the literature, mainstream solutions often create separate analytical streams corresponding to different data modalities, instead of leveraging full integration of all data sources into a single predictive model. Global forecasting solutions are also less frequently explored, with a significant proportion of studies focusing on solutions that predict a single index or stock price. This essentially establishes the gap that the current study aims to address.

Statement of the Problem

Given the nature of and uncertainty behind investing, investors in the PSE are interested in optimizing their portfolios, ensuring that their decisions are highly tailored to the best likelihood of gaining profits. Given that the PSE is a collection of hundreds of publicly listed companies that trade their shares, retail investors have a wealth of choices for investment, ranging from financials, industrials, real estate companies, holding firms, and services, but under limited budgets. This essentially creates their decision problem: how can retail investors in the PSE make better investment decisions regarding budget allocations among available stocks, and when to buy or sell stocks for maximum returns? Many different domains of study yield varied attempts to approach this highly critical issue, but tech-relevant solutions such as AI-, ML-, DL-, and NLP-based stock price modelling show significant promise in contributing beneficially.

In creating any model, the natural approach is to figure out how to best model human behaviors of research and analysis of the news, disclosures, social media sentiment, and other sources, leading to a trading decision. Current tools are limited, in unique respects: time-series-only models struggle to account for unbounded time-dependency lengths often associated with real-life decision-making, basic financial sentiment analysis fails to differentiate causal effects as opposed to merely correlational effects, and so on. It is thus

logical to explore the performance of multimodal DL/NLP methods such as multimodal fusion networks which may address concerns related to the fixed time-dependency lengths of time-series-only approaches, and also thus theoretically provide a partial solution to model drift. In enhancing the model, on the other hand, other kinds of challenges—such as tradeoffs—may appear; integrating heterogeneous data may increase dimensionality to the point of overfitting. The same problem appears when moving up the representational capacity ladder: more complex models, such as transformers, would be more prone to overfitting compared to LSTMs. Furthermore, fine-tuning on new data might cause catastrophic forgetting—where models forget previously-learned information—which might counterintuitively render model performance stagnant or even worse. Moreover, while long prediction horizons are appealing, the possibility of predictions many steps in time away becoming pure noise renders them no more useful—and may even be less useful—than models trained on shorter prediction horizons. It is thus logical to take a balanced approach—one that prioritizes model performance over model enhancements and complexity—even when it may be tempting to push for the latter. With all this, the central research problem essentially reduces to finding the best configuration of model architecture and data structure modifications to create well-performing Philippine stock return prediction models.

The problem, however, is not solved by finding a state-of-the-art solution, since such solutions are tied to single-case outcomes where results are applicable only to a highly specifically engineered design. It is more principled to study which factors involved in designing a deep learning architecture can lead to better outcomes in terms of model performance and other interesting metrics, in order to discover generalities and evaluate design principles instead. Four such model design factors are interesting, namely: *overall pipeline complexity* (choices of model architecture—whether transformer-based or MLP-based, text representation method—either semantic embeddings or sentiment features, modality fusion method—either cross-attention or simple concatenation, and

global forecasting capacity—whether or not stock intercorrelations are supported by design, which are all choices between a more complex and a simpler solution respectively), whether to include news articles (*inclusion of news data*) to inform predictions, whether to include social media posts (*inclusion of social media*) to inform predictions, and how far ahead in time predictions are made (*prediction horizon*). These four factors are described in heavier detail in later chapters. In evaluating the effect of these factors, it is also a principled choice to use metrics relevant to the prediction task; stock return magnitude and/or return direction are commonly used prediction targets in the literature; therefore, one can frame the prediction task either as a *classification* task (for return directions) or a *regression* task (for signed return magnitude). Stock direction classification is the prediction task undertaken in this study; as such, metrics such as Matthew’s correlation coefficient or MCC (in the case of unbalanced classification targets, which is common for stock datasets) are of importance. The effects of the factors can also be studied in terms of the model’s performance in recommending stocks to buy or sell in trading simulations. This is unique to financial machine learning applications, where models can be evaluated by how well they rank stocks in terms of prediction confidence. Finally, the effects of the factors can also be analyzed in terms of the stability of prediction confidence amid ever-changing market regimes. This is also unique to stock forecasting, given that the dynamics of financial data change rapidly over time. All in all, three interesting model evaluation methods, namely raw machine learning performance (e.g. MCC), stock confidence ranking capacity, and stability of model prediction confidence are principled evaluation measures to compare between models to study the effects of model design factors. The first two (raw machine learning performance, and stock confidence ranking capacity) are *model performance* metrics, whereas model confidence stability can be measured by computing *model drift* metrics. The specific research problem that the current paper aims to shed light on, therefore, is determining the impact of model design factors *overall pipeline complexity, inclusion of news data, inclusion of social media data*, and

prediction horizon on model performance and model drift.

Objectives of the Study

Given the problem formulation, the study aims to evaluate the performance of a number of approaches to modelling the PSE, employing a range of models broad enough to make comparisons between solutions built on aforementioned modifications (model design factors) that are treated as experimental factors. In more specific terms, a factorial experimental design was employed to test the theory that relates the model design factors, namely overall pipeline complexity, prediction horizon, inclusion of news, and inclusion of social media posts, to model prediction performance, stock confidence ranking performance, and temporal stability of prediction confidence. A fuller specification of the experimental conditions are exposed in later chapters, but a general description of the experimental set-up would involve different deep learning models with or without multimodal fusion (depending on the input data mix), each trained with unique configurations of prediction horizons and data configurations defined by the inclusion or exclusion of each among news and social media. In the process, the study aims to answer the following questions:

1. How does each one of overall pipeline complexity, prediction horizon, inclusion of news, and inclusion of social media posts affect model performance, both directly and indirectly through joint interaction effects?
2. How does model performance evolve over time, and how does each one of overall pipeline complexity, prediction horizon, inclusion of news data, and inclusion of social media interact with this evolution, both directly and indirectly through joint interaction effects?
3. Do deep learning models offer an overall improvement over machine learning baselines?

In line with each of these questions are the research objectives in Table 1 to tackle, concretizing the tangibility of the directions to which the study is taken.

Table 1*Objectives Matrix*

Research Question	Objective
How does each one of overall pipeline complexity, prediction horizon, inclusion of news, and inclusion of social media posts affect model performance, both directly and indirectly through joint interaction effects?	Construct a factorial experimental design including every combination of enhancements and successfully evaluate main and interaction effects of factors on model performance.
How does model performance evolve over time, and how does each one of overall pipeline complexity, prediction horizon, inclusion of news data, and inclusion of social media interact with this evolution, both directly and indirectly through joint interaction effects?	Understand the limitations of the design in achieving true robustness by incorporating a longitudinal aspect into the analysis of model performances to show and measure model drift, as well as how the experimental factors interact with model drift.
Do deep learning models offer an overall improvement over machine learning baselines?	Identify whether the best NLP/deep learning approach is significantly better than statistical/machine learning methods (such as moving averages) for stock prediction, at the same prediction horizon setting.

Essentially, the central research question of how to create a well-performing model under the aforementioned practical considerations is considered in exposing each objective.

Significance of the Study

A number of different kinds of stakeholders can benefit from tackling the central research question, including medium frequency investors and financial institutions at the focal level, as well as policy-makers, data analysts/quants, and academic researchers in the field of financial AI applications.

Medium frequency investors benefit by, aside from augmenting their decision-making strategies with predictive models, having the knowledge of what kinds of stock prediction models generate the best results given the national-level context. This enables them to optimize their efforts and capitalize on stock prediction tools, especially given insights from interaction effects, providing further understanding of the mechanisms behind stock movement and which data modalities are necessary and otherwise.

Apart from retail investors, financial institutions also benefit from the knowledge of the best prediction models, given insights from interaction effects. While retail investors spend capital on stock prediction forecast tools, financial institutions adopt these efforts at a remarkably larger scale to adapt to fast-paced stock data requirements as institutional investors, thereby having much greater computational resources, time, and specialized tools. While the models built herein may be too small in scale to directly apply to institutional investor requirements, the insights gathered from main and interaction effects studies can be extended to the higher-scale model architecture and data configuration choices at their disposal, providing opportunities to enhance and optimize investment decisions for better service to client investors. Such an advantage may bring about higher profits from investment services, helping to improve the finance sector.

On a more general scale, policy-makers may benefit as well from such main and interaction effects studies, especially with the involvement of different information sources (news, social media, macroeconomic indicators), which potentially affect sectors as they propagate through the market. These observed effects can, from the policy-makers' perspective, translate into general risk assessments for entire sectors. For example, market

volatility—characterized by widespread sell-offs among stockholders, among other things—may arise from negative social media sentiment when financial pressures such as interest rate hikes, high inflation rates, and macroeconomic disturbances are present. In the current study, this may be reflected as an interaction between the inclusion of macroeconomic features and the inclusion of social media data. In such an example, policymakers can take action by creating digital communication guidelines that reduce panic and misinformation that may perpetuate this social media-driven market volatility. Extrapolating from this, information on the magnitude and direction of main and interaction effects is relevant for identifying possible measures that can help stabilize the market, which is a policy-level undertaking.

Main and interaction effects analysis can also benefit data analysts and quant teams, who are interested in researching the most relevant data types for generating financial knowledge for business value. Based on insights derived that identify the most influential data sources for stock prediction, data analysts may optimize efforts by collecting only such kinds of data. On top of that, highly relevant themes from news and social media sources derived from feature importance distributions can be used to further filter news and social media data, thereby significantly improving the quality of data collection, creating possibilities for optimized and improved analytical and machine learning predictions.

Lastly, academic researchers also benefit from the scientific knowledge of model architecture and data modality interactions. Knowledge of main and interaction effects can help establish future conceptual frameworks and strengthen mechanistic principles that further studies can explore. Opportunities to study attention distributions on news and social media data can also further push the boundaries of state-of-the-art methods for collecting highly relevant textual financial data, and current methods herein can be replicated for fine-tuning to other specific emerging markets.

Scope and Limitations

As previously discussed, the current study examines how different deep-learning model settings affect stock prediction model performance on Philippine stock market indices and prices via a factorial design involving four experimental factors, namely inclusion of news data, inclusion of social media data, overall pipeline complexity, and prediction horizon, resulting in several experimental settings, each with a unique configuration of the mentioned variables. The first two factors describe modifications to the model inputs, but certain data modalities are constant across all experimental settings. Constant data inputs include Philippine Stock Exchange indices and stock prices, global/international macroeconomic indicators such as the U.S. Dollar to Philippine Peso foreign exchange rates, global commodity prices (gold vs. PHP exchange rate, oil futures, and continuous copper futures), and Philippine government bonds of different maturity periods. The modifications associated with the first two experimental factors are only additions of certain data modalities on top of this base data configuration. That said, the following is a more specific description of each experimental factor, including its scope:

1. *Inclusion of news data* - concerns whether news data is involved among the model inputs. News data inputs include finance-, economic-, and business-related articles from local sources such as The Manila Times, Rappler, GMA News, and other sources that may be found with the help of the proprietary LSEG Workspace News Monitor 2.0.
2. *Inclusion of social media data* - concerns whether social media is involved as a data input, including only posts from X.
3. *Overall pipeline complexity* - concerns whether the deep learning model to be tested is designed based on the transformer architecture or the multi-layer perceptron.
4. *Prediction horizon* - concerns settings for the length of time away the model predicts returns from the latest data input timestamp. The exact ranges for the prediction horizon are described more fully in the Methodology.

That said, the models built primarily involve transformers and multi-layer perceptrons for technical and exogenous financial data. On experimental settings including textual data, frozen NLP encoders are used to encode data into embeddings or sentiment vectors, which, depending on the model architecture, are processed further with cross-attention transformer modules. NLP encoders may be outsourced, pre-trained models that create token embeddings (i.e., BERT - Bidirectional Encoder Representations from Transformers, or LLM embedding models) for news data and/or social media data. Baseline machine learning techniques that are compared against the proposed deep learning approach include support vector machines, logistic regression, random forest, and extreme gradient boosting.

The data analysis includes an exploration of model performances compared to simpler baseline methods, the central analysis of main and interaction effects of and between the experimental factors to model performance, model drift, and trading simulation performance. The temporal scope of the study attempts to cover as much of the entire span of minute-level technical data, news, and social media as available and feasible, starting from March 2025, up until April 2026, achieving as much intraday granularity as possible for maximal dataset sizes.

The study only focuses on predicting the Philippine stock market. While stock and economic data from other countries and global sources are included within the scope, they are included only as exogenous factors as opposed to targets for prediction. That said, the results of this study may not be generalizable to prediction tasks for other stock markets globally. For all information sources, private or proprietary information from data vendors and scrapers is included, but are nonetheless concentrated on the local landscape and locally relevant international contexts. The study at its core is about methods; findings of this study are focused on information-source-wise and time-wise effects on model performance and drift, and any other auxiliary findings that may shed light on other domains (such as the underlying business domain of the Philippine stock market) are intended only to explain those effects—as evidence of underlying model prediction

mechanisms—as opposed to evidence of economic, financial, or market-related realities. In this sense, the study does not claim that its findings are valid or generalizable to the study itself of the Philippine stock market, or any other background areas. The methods under consideration are limited only to multimodal deep learning and NLP methods, and references to any other form of stock prediction are only intended as points for comparison. That said, the study does not claim that its findings are generalizable or applicable to other stock prediction methods, namely non-deep machine learning methods, statistical models, rule-based methods, heuristic strategies, and portfolio optimization protocols. However, some generalizability to multivariate deep learning stock prediction methods that are single- or multi-output can be expected. Lastly, the study does not aim to forecast market shocks, although the goal includes evaluating the models’ performances and prediction mechanisms potentially during periods of shock.

In summary, the study does not aim to create a “state-of-the-art” model or aim to “beat” performance metrics established in the literature, since the main goal is only comparison between models. Moreover, the study also does not aim to investigate the profitability of models; only performance metrics core to the model are analyzed independent of any trading strategy. However, this does not prohibit the study from borrowing trading simulation methodologies to achieve the goal of studying model behaviors, but in doing so, only claims about model behavior (and not portfolio optimization strategies) are made.

Chapter 2: Review of Related Works

Predictability in Efficient Markets

The problems of stock market prediction are rooted in fundamental economic, market, and informational bases. One such basis is the concept of *efficiency*—the property of market prices reflecting all available information. As discussed in the following, the notion of efficiency challenges the notion of predictability in stock markets. However, shown in the following subsections as well is a discussion of conditions wherein predictability may survive in the context of market efficiency. The discussion is based on studies published in the fields of economic theory and financial market dynamics.

The Efficient Market Hypothesis

Efficiency can be demonstrated in the following example: when historical records indicate that a stock price rebounds after reaching a minimum value, and its current price is at that level, it cannot necessarily be inferred that the stock price will rebound as it did before. If so, investors will have already taken action, buying the stock until its price has rebounded. Essentially, all information that can be inferred is already “baked in” the current stock price, thereby rendering changes to stock prices hopelessly unpredictable, leading to random walk behavior. This is known as the efficient market hypothesis (EMH), where an efficient market is defined by Fama (1970) as “a market with a great number of rational, profit-maximizers actively competing, with each trying to predict future market values of individual securities, and where current important information is almost freely available to all participants”. Based on this definition, the unpredictability of the stock market is attributed to market competitiveness and its ability to nearly instantly process predicted value discrepancies, which is a stronger assertion of unpredictability than the vast complexity of interacting factors that influence prices—if true, it delivers a “final nail to the coffin”, asserting that even if one were able to capture all relevant information from the incredibly intricate network of low-signal influences, prediction still cannot be expected to garner profits, since the competitiveness of the market has more than likely already

processed it.

There are multiple forms of the EMH—the weak form, the semi-strong form, and the strong form. The weak form stipulates that the market efficiently absorbs technical information—all past stock prices and volumes—implying that technical analysis, namely directly analyzing stock behaviors, is useless for predicting future stock trajectories. The semi-strong form on one hand hypothesizes that the market efficiently absorbs all public information—not only technical information, but also economic information, news reports, and financial statements. This means that technical analysis and fundamental analysis—the process of analyzing the intrinsic value of a stock using all available information—are futile to predict stock trajectories. Finally, the strong form of the EMH states that the market absorbs all information, even privileged information, making any sort of analysis, even on secretly-held information among CEOs or board members, futile for prediction. Multiple studies have tried to test the hypothesis as well as its three distinct forms, but consensus remains elusive. Results are often controversial or inconsistent with other studies, highlighting methodological flaws, or intrinsic elusivity of the hypotheses (Tıtan, 2015).

Mechanics behind Financial Markets

There are different kinds of financial markets. There are stock markets, fixed income markets (government bonds), commodity markets (copper, oil), and foreign exchange markets, that all exhibit real-time evolution of financial assets. All financial markets exist fundamentally to optimize the allocation of capital within an economy, just like in other kinds of markets, where the main difference is only that the items traded are financial assets. That said, financial markets share the same principle of decentralized participation of independent individuals (retail investors, hedge funds, governments, etc.), each with different objectives, expectations, and information. The summarized interplay between these participants is baked into the traded assets' prices by way of price priority; due to this, the market also functions as an information aggregator, where prices reflect overall

investor sentiment (Bodie et al., 2023). A more detailed description of how financial markets work can be found in the following example for stock markets.

The stock market is mainly composed of the exchange (i.e. the Philippine Stock Exchange), and the participants (retail and institutional investors) in an iteration-based Continuous Double Auction mechanism. The exchange does not dictate market prices; they do not “own” the stocks in the market—instead, all stocks are owned and traded by the market participants themselves: retail investors, companies, etc., such that pricing is determined by the participants. Moreover, transactions are not executed independently; all buy and sell orders within a given timeframe are recorded in a Centralized Limit Order Book (CLOB) stored electronically within the exchange, and the exchange executes all transactions in a single event. This process repeats and completes in the order of milliseconds, if not microseconds—the auction system is limited by the speed of light in fiber optic cables.

There are essentially two types of buy or sell orders—limit orders and market orders. Limit orders are orders at a user-defined price called the bid price for buy orders and ask price for sell orders, and they are not executed immediately—rather, the exchange waits for another buy/sell order that meets the user-defined price to enter the auction, before the order is executed. This enables market participants to trade more cautiously and have full control over the amounts they are spending or earning. A priority system is set in place such that buy orders with the highest bid and sell orders with the lowest ask are executed first—if multiple orders have the same price, the orders that entered the auction earlier are executed first. The gap between the highest bid and the lowest ask prices is called the spread; for every auction, the exchange exhausts all limit buy orders with prices higher than or equal to the best ask prices and all limit sell orders lower than or equal to the best bid price—namely all orders that “cross the spread”. This always results in a nonzero and positive spread. On the other hand, market orders are “aggressive” buy or sell orders that execute on demand—regardless of the price. These orders do not come with a user-defined

price; the price of a buy order defaults to the lowest ask in the auction, and that of a sell order defaults to the lowest bid (Zaznov et al., 2024).

This mechanism works for every single stock on the exchange, giving rise to ever-evolving prices. Stocks are also organized into sectors (financials, holdings, industrials, etc.), each deriving an index from its constituent stock prices, which therefore contains information about the overall state of an industry (San Miguel, 2019). The exchange also exhibits its own overall index, which is normally a summary of the exchange's top companies' prices. For the Philippine Stock Exchange, this is the PSEi (Philippine Stock Exchange index), which summarizes its top 30 most actively-traded stocks (Gallardo & Viray, 2025).

Given that the stock market updates in the scale of microseconds (Zaznov et al., 2024) and that many applications view stock market prices in different trading period frequencies, charts and data tables must account for the fact that the full raw auction data is far too informationally dense and noisy for such applications. Certain aggregation is done for a chosen frequency of trading periods, namely by including only the opening, highest, lowest, closing prices of the stock, and the volume or number of stocks traded that occur within the interval of a single trading period (Schabacker, 2021). The aggregation of Open, High, Low, Close prices together with the Volume within a fixed trading period is otherwise known as an OHLCV candle.

A stock market participant can enter and leave the market in two ways. One can enter the market by buying a stock, and leave by selling that stock later on. This is known as entering (and leaving) a long position or “going long” for a certain asset. The other way involves entering the market by borrowing a stock from a stockbroker and selling that stock, to buy it later on and return it to the broker. This is known as entering (and leaving) a short market position (“going short”). Both methods are symmetric in the way market movements are leveraged to gain profits. When going long, the participant hopes that the price of the asset increases in order to achieve profits by selling; when going short,

the participant hopes that the price of the asset decreases in order to profit from buying the same asset (borrowed stocks must always be returned to the broker, thus buying is necessary). Shorting a stock in particular gives a way for participants to enter the market and expect profits even without spending capital, but bears greater financial risk when done without available capital to absorb losses (Reed, [2013](#)).

While all financial markets are fundamentally similar, the nature of the item traded differentiates them strongly (Mishkin, [2018](#)). Depending on the item traded, investor intentions and expectations differ, which brings about unique market behaviors.

For instance, markets for oil futures trade oil contracts, which specify a purchase of a certain amount of oil at a certain maturity date. This means that oil prices fluctuate based on global oil supply, and are thus most affected by supply cuts due to geopolitical affairs, production decisions, and oil supply transport conditions. This distinguishes it from the stock market where prices are determined largely by investor sentiment and company performance rather than global supplies; however, this is also why oil prices are relevant for stock markets—oil supply dictates transportation and production costs, providing a secondary effect on stocks (Park & Ratti, [2008](#)). Other studies however find that the effect of oil on stock markets is unremarkable (Apergis & Miller, [2009](#)), which may point out that these effects are not as pronounced compared to those of other types of markets.

Copper futures work nearly the same—contracts for purchases of copper are the assets traded, thus prices depend on global supplies of copper, which affect stocks particularly ones that rely on technological infrastructure and/or directly manufacture technological goods (since copper is an industrial metal). It also acts as a measurement of economic activity; copper demand rises during times of rapid industrial advancement, which therefore adds a tertiary global effect on stock prices (Ahn, [2019](#)).

Foreign exchange markets on one hand are the best examples of a decentralized market. Here, items traded are currencies themselves, whose values can fluctuate as a direct consequence of macro-scale differences in interest rates between the countries of the

currencies involved. Its relevance on stock prices is in how it indicates inflation differentials and global supply purchasing power—and by extension, company operational costs, especially on imported supplies (Phylaktis & Ravazzolo, 2005).

Government bonds are another good indicator of inflation (Mishkin, 2018). These are instruments that allow the government to borrow from willing investors, promising fixed repayments with interest over a maturity period, which can extend up to several years or decades, namely time scales where the purchasing power of subsequent fixed repayments can change significantly due to inflation, thereby affecting market desirability and valuation of bonds. Moreover, the percentage of a bond's market price constituting the fixed return is the bond yield, whose value reflects general market expectations on fixed returns; participants would sell bonds with returns lower than expectations to obtain assets with higher returns, reducing the bond's market price and thereby increasing its yield. Such expectations may be tied to climbing inflation rates, since central governments raise interest rates to combat inflation, making high-return opportunities more appealing for investors, and rendering low-return opportunities such as bonds issued long ago less capable of satisfying investors' requirements on returns given lower purchasing power; thus, bond yields also capture information related to inflation. Taken to another level, bond yields between different bonds of different remaining maturity periods can be compared by calculating yield spreads, whose directionality and magnitude encode information regarding the expected evolution of inflation over time (for instance, bonds reaching maturity in the far future with yields lower than that of bonds reaching maturity in the near future indicate an expectation of peak inflation in the near future—given that investors expect yields to reflect their requirements given changes to purchasing power—that would stabilize to lower inflation levels in the far future—since a rise in a far-future-maturing bond's yields would require longer-term perceived difficulty for fixed returns to match investor requirements, which is not the case with an expected eventual relaxation of inflation levels) (Fabozzi & Fabozzi, 2026). In summary, bond prices, bond yields, and yield spreads

contain a lot of macroeconomic information that globally affect stocks (Baele et al., 2010).

Finally, gold markets serve as a “safe storage of value” for investors (Ciner et al., 2013)—when uncertainty arises, such as poorly performing stocks, economic recession, wars, and etc., gold becomes desirable as its value remains intrinsically unaffected by economic shocks (since changes in intrinsic value would theoretically accompany changes in the overall scarcity of gold, which is unlikely) (Baur & Lucey, 2010). In such regimes, participants would sell unstable assets from other financial markets including stocks and buy gold. Conversely, when economic expectations shift more positively (companies will grow, economy stabilizes, etc.), participants would sell gold to restore positions in other financial markets. This theorizes a potential inverse relationship component between gold prices and asset prices from other types of markets including stocks.

The Fate of Prediction Algorithms

Studies on stock prediction using ML or DL techniques however, have an interesting place in the theory of efficient markets. A study that compared 55 different stock markets showed that market efficiency does not influence the accuracy of machine learning algorithms for stock prediction (Bustos et al., 2025). Lo (2004) theorized as an alternative to the Efficient Market Hypothesis his own Adaptive Markets Hypothesis, which states that market efficiency is not uniform and absolute; rather, it is dynamic—market inefficiencies can arise from gradual, rather than near-instantaneous adaptation of the market to newly available information. This is supported by a number of well-known market anomalies. The momentum effect (Jegadeesh & Titman, 1993), wherein rising stocks tend to continue rising and falling stocks tend to continue falling—even over long-term periods—used to be an exploitable feature of the market for any investor, albeit only up until the 1990s after which momentum returns since diminished (Bhattacharya et al., 2015). Moreover, the post-earnings-announcement drift—where stock prices continue to react weeks after earnings reports—also continues to be exploited by investors (Bernard & Thomas, 1989), and even institutional investors (B. Ke & Ramalingegowda, 2004).

Lastly, calendar effects (C. Y. Zhang & Jacobsen, 2020) still offer seasonal predictability, albeit to a diminished extent. This suggests that machine learning and computational approaches to predicting the stock market can take advantage of the edge cases where windows of time exist in which information has not yet been fully arbitrated. Indeed, (S. Gu et al., 2020) established that machine learning algorithms performed better than traditional approaches to stock market prediction. Evidence of gradual instead of near-instantaneous information absorption in intraday to daily scales has been demonstrated by (Khan et al., 2023) that 15-minute intervals for machine learning stock prediction performed better than prediction for 1-day intervals.

The existence of market anomalies and alternative explanations for stock market behavior suggest that market inefficiencies occur both at the levels of technical and fundamental data, thereby weakening the semi-strong and weak forms of the EMH. This subsequently gives rise to the viability of combining other data input modalities into the feature matrix of stock market prediction, and relationships between such features and the stock market have been observed in emerging markets such as the Philippines. Studies on stock market behavior at this level and on the Philippine landscape are scarce; much of such topics remains underexplored—however, a larger wealth of studies that analyze cases generally for lower-middle income countries can nonetheless provide insight. A study by (Enilov et al., 2021) has shown that causality moves from the world commodity index to stock prices for lower-middle income countries during the more recent years of the 21st century. A further breakdown revealed that for low to lower-middle income countries, greater causality from oil prices to stocks prevails in the same time period than vice versa, and causality from metal to stock prices exhibit greater levels than causality from oil to stocks. Moreover, (Aydin et al., 2023) found that for ASEAN countries, stock markets suffer as the currency weakens, establishing a relationship between exchange rates and stock market prices. This provides evidence that local macroeconomic indicators can provide significant signals for markets like that of Philippine stocks. Moreover, volatility

spillover from the stock market to government bonds has been observed in countries such as the Philippines and Thailand (Yurastika & Wibowo, 2021), showing how similarities between the Philippines and financial markets from other countries of similar economic development levels are not arbitrary—thus strengthening the credibility of such macroeconomic effects. While inconclusive, these results also suggest that macroeconomic indicators are a possible breeding ground for market inefficiency windows, where price discovery happens more slowly, driven by lower liquidity levels/slow market participation in the Philippine stock market. A more conclusive Philippine-based study by Sombrito et al. (2023), however, showcased a high-accuracy LSTM-based stock prediction model for the Philippine Stock Exchange trained on technical data, earnings reports, and news data, which suggests that the market may also be inefficient at absorbing fundamental data, and perhaps by extension news and social media data as well, giving rise to an opportunity for arbitrage through news-processing machine learning models. This also suggests that textual data and price data can be synergistic in the Philippine market—where earnings reports and news contribute significantly to prediction model accuracies. On a tangential note, having achieved a higher accuracy than other model architectures on a mix of fundamental and technical data and on other countries' financial markets suggests that the choice of model architecture and the market landscape being studied has a moderative impact on the effects of such data modalities on predictive power; optimal model architecture choices for the Philippine stock market may be unique. Reverting to predictive factors, another Philippine study has established that social media is a powerful tool for Filipino investors (Casao et al., 2025), who, given that they usually trade in lower frequencies compared to institutional-level or intraday traders, may provide sources of market inefficiencies driven by social media. Overall, inclusion of economic data such as commodity prices such as gold, metal, and oil, foreign exchange rates, and government bonds, and even textual data such as financial news data and social media posts thus have a favorable chance of reducing prediction uncertainty for the Philippine stock market. Indeed, inclusion of gold, metal, oil,

foreign exchange, and government bond/treasury yields as features for stock price return prediction is demonstrated explicitly by Lohrmann and Luukka (2018), with stock price and foreign exchange features exhibiting the highest importances.

Another layer of complexity to the favorability of certain factors in predicting stock prices is the taxonomy of sources of stock price movement at different time scales/prediction horizon levels. At the high-frequency (milliseconds to the next few minutes) scales, market orders move the stock prices—a large influx of market orders can exhaust the orders with the best prices, so the trade moves to the next best prices. This is the origin of a concept known as price impact (Du & Zhu, 2017), where the action of a trader moves the prices, even potentially against their own favor. Zaznov et al. (2024) also showed that order book imbalances, transaction imbalances, and past returns are highly relevant for high-frequency trading, which they established by including limit order book and order flow data among their features for a stock prediction algorithm. More specifically, behind the graph line of stock prices on stock exchange displays is an invisible interval of bid prices below it and an interval of ask prices above it. When an order book imbalance occurs, more limit buy orders exist in the market at a particular time than limit sell orders or vice versa. This may signal buying or selling “pressure”; in such cases, the market price of a stock is likely unstable, where limit orders that “cross the spread” may move the stock prices, which therefore means that not only market orders can move prices, but so can limit orders; it can be said that limit orders that cross the spread act indistinguishably from market orders. Moreover, the quantity of limit orders can also dictate the rates at which the prices move—lower volume, which is indicative of lower liquidity, can contribute to longer spreads, and therefore more price volatility. However, snapshots of the limit order book alone cannot suffice to reliably determine market movements. Another feature of limit orders is that they can be cancelled by users—essentially, users can withdraw from the market without making any transactions, which is thus an action that does not move stock prices. By simply looking at a snapshot of the limit order book, one cannot

determine whether an order book imbalance is caused by buying or selling pressure, or users leaving the market on one side more than the other (which does not directly impact stock movements). Moreover, market orders and limit orders that cross the spread do not appear in the limit order book. To address this, another instrument used for determining market movements is the order flow—a table which contains every transaction (market orders and executions of limit orders), including the exact timestamp, direction (buy/sell), volume of transaction, and prices. This table encompasses momentum information that is predictive for stock movements even at the micro- to millisecond scale (Zaznov et al., 2024). The same is true for prediction horizons spanning up to only a few minutes, as established by Aït-Sahalia et al. (2022) after which this kind of predictability rapidly diminishes. In such time scales, they also found that less actively traded stocks' returns were more predictable, as well as that simulated errors in trading significantly increased predictability. In terms of intraday seasonality, end-of-day prices were the most predictable. Other stock markets such as the Shanghai stock exchange feature the most correlation between stocks during the first five minutes of start-of-day prices (Tan et al., 2016). Sirnes and Dinh (2021) also established that order book imbalances, especially in highly liquid stocks, generate overreactions in stock prices—namely times wherein market prices jump significantly and reverse before stabilizing—likely as a result of investor panic from increasing buying pressure or selling pressure, providing a possible explanation that high-frequency predictability from order book imbalances are caused primarily by market inefficiencies. Such market inefficiencies may also be deliberately prolonged by predatory trading behaviors (Du & Zhu, 2017), wherein high-frequency traders exploit price discovery windows by slowing down their trades so as not to deliver price impacts that would drive prices to their perceived market value—essentially by exploiting uninformed trading decisions by others. This is especially more likely in highly liquid stocks with a higher auction frequency, providing evidence of predictability for high liquidity stocks despite the finding by Aït-Sahalia et al. (2022) that predictability improves for low liquidity stocks. On

one hand, in medium-frequency time-scales, Safari and Schmidhuber (2025) established that the market behavior shifts from persistent strong trends and reversals of weak trends (in higher frequencies, indicative of market inefficiencies due to the weak “corrective” effect of market consensus on overreactions), to persistent weak trends and reversals of strong trends (in lower frequencies, indicative of evolving market consensus towards structural, more abstract economic realities). This establishes that at these time-scales, market microstructure no longer takes hold of predictability; rather, evolving realities and evolving market-wide interpretations of such realities drive stock movements. This behavior can be seen in higher resolution in Moberg and Sucarrat (2007), where order flow data explain a significant portion of stock return variation in the 15-minute prediction frequency, further contextualizing this effect as order flows being a signal to the market that market consensus needs adjustment to better reflect internal economic realities. This also highlights that at the 15-minute frequency, movements start being dominated by market-wide interpretations instead of market microstructure push-and-pull mechanics. At even lower frequencies, Baker et al. (2025) established that stock market reactions to macroeconomic triggers, such as news on monetary policies and government spending, are observable in the daily frequency scale. Here, market-wide interpretations reflect internal realities, since stock price variation follows the arrival of new information. This is true even in the much longer scales, where even annual stock returns exhibit predictability, with net-of-inflation and combined earnings-by-price and long-short rate spread being the best predictors (Kyriakou et al., 2019). Moreover, stock market behavior in long time scales is driven by volatility regimes at different levels—in the industry level, country-level, and even global level—whose effects interact jointly, such that the effect of country-specific factors on stock returns drops during global volatility regimes (Catão & Timmermann, 2010).

To summarize, the difference between high-frequency and low-frequency stock price behaviors is in the nature of price movement drivers. In the high-frequency scale (milliseconds to a few minutes), market microstructure anomalies—namely order book

imbalances and exploitative trading behaviors—drive relevant price movements, due to market inefficiencies or the inability of the market to absorb information instantaneously. The stock prices themselves may not reflect the actual market consensus, and thus are unstable, undergoing reversions and fluctuations before stabilizing at the market-perceived supposed price, in a process called price discovery. In this scale, faster traders have an edge: knowing information quickly as they arrive gives the advantage of knowing the perceived direction of a stock, since this information has yet to propagate through the rest of the market. Market inefficiency therefore dominates predictability in such scales. On one hand, in the low-frequency scale (daily to longer time scales), changes to the actual perceived intrinsic value of a stock are what drives price movement—a “structural” reassessment of a stock’s intrinsic value as opposed to the “mechanical” push-and-pull of buying and selling pressure. Predictability still occurs, insofar as the trajectories of stocks’ intrinsic values can be inferred, but predictability occurs no longer due to market inefficiencies—the market is virtually efficient in such scales. Ultimately, market efficiency does not actually necessarily mean loss of all predictability. Finally, the most interesting to examine is the intermediate point—the intraday periods (minutes to hours). Based on studies reviewed above, movements in this scale are driven by a mix of the two—mechanics due to market inefficiencies as established by Du and Zhu (2017), and market consensus of stock prices evolving to match intrinsic realities manifesting as order flow pressures, as established by Moberg and Sucarrat (2007). In this frequency space, predictability is shared between technical data, news, and other fundamental data sources. This is thus an ideal frequency space for intraday data modality studies such as the current paper.

Review of Statistical, Machine Learning, and Deep Learning Methods

The pursuit of successful prediction in financial markets entails an overlap between multiple areas, namely statistical theory, data science, model engineering, and computational power. The random nature of equity markets has been modeled, over the course of the last century, by various techniques—from simple linear models to complex

deep learning architectures that attempt to emulate real-life human trading decisions (Sable et al., 2023). This move to more complex computational procedures is entailed by the growing challenges of time-series forecasting in financial markets, which include high volatility, random-walk processes, and low signal-to-noise ratios (Somkunwar & Srivastava, 2024). The following section provides an overview of the foundational literature on the several statistical, machine learning, and deep learning methods used for the current paper, described in their utility in financial prediction, as well as their origins and modern interpretations.

While the efficient market hypothesis posits that the market is hopelessly unpredictable (Beniwal, 2025), a growing body of literature provides evidence of market inefficiencies and anomalies that create tension with the hypothesis. This is the conceptual foundation of the pursuit of precision in stock forecasting—the “hope” that markets are not as efficient as posited, where windows of inefficiency can be arbitrated by computational processes that can absorb and process information faster than any human trader’s capacity. This is the theoretical justification behind the growing body of research booming in recent years that explores different, simple and convoluted ways of predicting the stock market algorithmically.

Financial stock prediction is presented in literature both as a regression and a classification problem. As a regression problem, financial data (in simpler and straightforward models) is structured as a uni- or multivariate time series where at any timestamp, the stock price or index value is a function of its lagged values (univariate) or of that and lagged values of multiple other exogenous variables (multivariate) in the same previous timestamps (Funde & Damani, 2023). When treated as a classification problem, the input data structure varies depending on the algorithm—for machine learning (non-deep learning) approaches, previous timestamps in a univariate prediction task are flattened into a single row of data containing the lagged values (Krauss et al., 2016), or transformed into a single timestamp of rolling technical analysis features (Patel et al.,

[2014]), where the target variable is transformed into the sign of the stock return. For complex deep learning architectures such as in LSTMs (Q. Chen, [2021]), not only are multivariate and multi-step input matrices acceptable as a data input format, but given the flexibility of deep learning architectures, the data configurations are nearly limitless. The transition from simpler to complex approaches has expanded the capabilities of financial prediction to include news features, investor sentiment, and macroeconomic indicators, all of which have an influence on stock price movements in modern exchanges.

Autoregressive Integrated Moving Average (ARIMA)

Financial forecasting began its roots from statistical modeling, with reliance on rule-based and/or intuitive mathematical bases that make assumptions about time series variables under consideration. One such method is the moving average (MA) model, which can be traced back to Slutsky ([1937]) with his use of such methods in modelling random shocks that were found to have hidden economic seasonalities—simply by mere rolling aggregation. At the same time, Yule ([1927]) introduced the autoregressive (AR) model to describe processes with correlated lag values—such as periodic phenomena like sunspot numbers.

An autoregressive process with order p , denoted as $AR(p)$, is defined as follows, where Z_t is the value at time t , ϕ_i are the autoregression coefficients, and a_t is a term for white noise.

$$Z_t = \phi_1 Z_{t-1} + \phi_2 Z_{t-2} + \cdots + \phi_p Z_{t-p} + a_t \quad (1)$$

On one hand, the moving average model of order q , or $MA(q)$, is defined in the following, characterized by a relationship between an observation and past errors. In the equation, θ_j are the moving average parameters.

$$Z_t = a_t - \theta_1 a_{t-1} - \theta_2 a_{t-2} - \cdots - \theta_q a_{t-q} \quad (2)$$

A unification of the AR and MA model, which is formalized by Box and Jenkins ([1970]), called the ARIMA (Autoregressive Integrated Moving Average) model, became a landmark

stage in statistical forecasting which introduced differencing in order to achieve stationarity. This is a necessary step in order to remove any confounding effects for further statistical analysis, and is therefore done before the AR and MA modelling. The differencing itself adds another parameter, d , namely how many times differencing has to be done recursively in order to achieve stationarity. Altogether, an ARIMA model is tuned by all three parameters p , d , and q . ARIMA has also shown success in forecasting financial markets, particularly in short-term prediction for NIFTY 50 and the Johannesburg Stock Exchange (Funde & Damani, 2023).

Exponential Smoothing Models

Exponential smoothing is yet another mathematical forecasting technique, which is more relaxed than ARIMA. It makes less complex and easier-to-verify assumptions about the time series in consideration, and is intuitive, like moving averages. It was first introduced by Holt (1957) and Winters (1960) by tackling sales forecasting with an approach different from moving averages. Unlike moving averages that assign arbitrary weights to individual observations, exponential smoothing assigns greater weights to more recent observations, in an exponentially decreasing fashion as observations go further back in time. Different versions of exponential smoothing are employed depending on the nature of the time series, as detailed in the following.

For time series data with no clear trend or seasonality, a simple exponential smoothing model suffices. This model uses a single smoothing parameter, α , to weight the most recent observation against the previous forecast.

$$S_t = \alpha y_t + (1 - \alpha)S_{t-1} \quad (3)$$

Where y_t is the actual value of the process at time t , and α is the smoothing coefficient ($0 \leq \alpha \leq 1$).

Now, for time series data with a clear upward/downward trend that is consistent and without seasonality, Holt's method for double exponential smoothing adds another layer to

handle the trend by creating intermediate level (L_t) and trend (b_t) series for integration in the final forecast (F_t). One of two versions are applied depending on whether the trend is additive (linear growth) or multiplicative (exponential growth), denoted in the following on each line (left: additive; right: multiplicative).

$$L_t = \alpha y_t + (1 - \alpha)(L_{t-1} + b_{t-1}) \quad \text{or} \quad L_t = \alpha y_t + (1 - \alpha)(L_{t-1} \times b_{t-1}) \quad (4)$$

$$b_t = \beta(L_t - L_{t-1}) + (1 - \beta)b_{t-1} \quad \text{or} \quad b_t = \beta \frac{L_t}{L_{t-1}} + (1 - \beta)b_{t-1} \quad (5)$$

$$F_{t+m} = L_t + mb_t \quad \text{or} \quad F_{t+m} = L_t \times (b_t)^m \quad (6)$$

where m is the forecast horizon.

Lastly, for time series data with both an upward/downward consistent trend and seasonality, the Holt-Winters method, also known as the triple exponential smoothing model, is applied. Here, a third intermediate series is created (s_t) to account for seasonality. Like double exponential smoothing, there are both additive (left) and multiplicative (right) versions, which correspond to linear and exponential trends and/or seasonality effects:

$$L_t = \alpha(y_t - s_{t-L}) + (1 - \alpha)(L_{t-1} + b_{t-1}) \quad \text{or} \quad L_t = \alpha \frac{y_t}{s_{t-L}} + (1 - \alpha)(L_{t-1} + b_{t-1}) \quad (7)$$

$$b_t = \beta(L_t - L_{t-1}) + (1 - \beta)b_{t-1} \quad \text{or} \quad b_t = \beta(L_t - L_{t-1}) + (1 - \beta)b_{t-1} \quad (8)$$

$$s_t = \gamma(y_t - L_t) + (1 - \gamma)s_{t-L} \quad \text{or} \quad s_t = \gamma \frac{y_t}{L_t} + (1 - \gamma)s_{t-L} \quad (9)$$

$$F_{t+m} = L_t + mb_t + s_{t-L+m} \quad \text{or} \quad F_{t+m} = (L_t + mb_t)s_{t-L+m} \quad (10)$$

where again, m is the forecast horizon.

Support Vector Machines

Now moving away from statistical techniques, the advent of machine learning models—namely models whose parameters can be adjusted by an optimization of a loss function given a dataset—began taking over the landscape of financial stock prediction. One of such machine learning models is the support vector machine (Cortes & Vapnik, 1995), which is designed to find the most ideal hyperplane that separates data points. This

hyperplane need not be a dimension less than the feature space; kernel functions can be used to transform data points non-linearly into higher dimensional spaces where the hyperplane search can take place instead—a concept known as the “kernel trick”—which is usually the case due to the fact that it’s almost always impossible to draw a hyperplane that perfectly separates classes. The ingenuity behind the design of SVMs sets them apart from other techniques that aim to minimize empirical risk (Beniwal, 2025), being widely used in stock return prediction studies (Sable et al., 2019).

Its regression counterpart, namely the support vector regression, works by fitting a tube (called the “epsilon tube” or -tube) that encapsulates as many data points as possible. This complexity allows the model to find complex patterns beyond the capacity and reach of simpler regression models. In financial markets research, SVR performs well with OHLCV data, since these features reflect the width of market fluctuation, and therefore investor sentiment as manifested by rapid price movements (Sable et al., 2019).

Logistic Regression

Logistic regression has also been a useful machine learning model apart from SVMs in financial market classification tasks like stock prediction (Jarmulska, 2021). It is unlike linear regression in that it does not output unbounded values, but rather a sigmoid function:

$$f(x) = \frac{1}{1 + e^{-x}} \quad (11)$$

which maps any real-valued input into a probability between 0 and 1—thereby making it appropriate for binary classification tasks. In a seminal paper, Ou and Penman (1989) demonstrated the use of logistic regression to predict earnings changes, which is a direct and early application of the technique on financial markets. This shows that while it is a simple technique, logistic regression can be successfully applied to the prediction of market movements through fundamental data.

Random Forest

Breiman (2001), in his highly cited seminal introduction of the random forest (RF), showcased that an ensemble of decision trees trained on bootstrapped samples of a dataset performed with breakthrough success in prediction applications when constructed with “bagging” methods—where the trees undergo a voting scheme to make a final prediction. In this manner, the RF was able to decrease variance without increasing the bias, which makes it a useful tool against overfitting.

Jarmulski (2021) also established the potential of random forests in forecasting market panic, market microstructure anomalies, and even in predicting entire market conditions. In a lot of studies employing both random forests and logistic regression models, random forest has been seen to outperform the latter, with prediction accuracies of greater than 80% for more complex prediction tasks such as debt crises (Aldasoro et al., 2025). Random forests also come with a regression counterpart, namely the random forest regression algorithm, where averaging is chosen as the bagging method.

Extreme Gradient Boosting (XGBoost)

Similar to random forests, extreme gradient boosting (XGBoost) makes use of decision trees, but no longer in an ensemble “bagging” mode. Rather, decision trees are applied “in series” wherein each tree corrects the errors of the previous tree—a behavior that can be optimized by minimizing the following objective function:

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k) \quad (12)$$

where l is a differentiable loss function and Ω is a regularization term that penalizes the decision trees’ complexity, offering ways to mitigate overfitting (T. Chen & Guestrin, 2016). Financial time series has benefited from the use of XGBoost algorithms due to its ability to handle complex data, but even counterintuitively performing better than more complex deep learning architectures, whereby its complexity matches the prediction requirements of financial markets, such as corporate bonds and cryptocurrency, even during extreme

volatility periods such as the COVID-19 pandemic (Y. Zhang & Chen, 2021).

Feed-forward Neural Networks

Deep learning, which started from the very first publication of a neural network (Rosenblatt, 1958) has taken an ever-growing role in financial markets research. Stock return prediction studies have been the subject of highly innovative deep learning architectures due to the sheer customizability of neural networks. When beginning any study into deep learning, an exposition towards one of the simplest configurations of a neural network is virtually imperative, involving that of a parameterized function $f_\theta : \mathbb{R}^{d_0} \rightarrow \mathbb{R}^{d_L}$ mapping d_0 -dimensional vectors to d_L -dimensional vectors through the following composition of L layer functions $f_1, \dots, f_L$:

$$f_\theta = f_L \circ f_{L-1} \circ \dots \circ f_1, \quad (13)$$

where each function $f_l : \mathbb{R}^{d_{l-1}} \rightarrow \mathbb{R}^{d_l}$, mapping d_{l-1} -dimensional vectors to d_l -dimensional vectors, is defined by

$$f_l(z) = \sigma_l(W_l z + b_l), \quad (14)$$

with weight matrices $W_l \in \mathbb{R}^{d_l \times d_{l-1}}$, bias vectors $b_l \in \mathbb{R}^{d_l}$, and activation functions $\sigma_l : \mathbb{R} \rightarrow \mathbb{R}$ applied for every element of the vector $W_l z + b_l$. The activation functions are typically nonlinear to achieve the best approximation effect. This particular configuration of a neural network is called a feed-forward neural network (with more than one layer, it is called a *Multi-Layer Perceptron* or MLP), and its versatility and utility derives from its ability to approximate any practically useful mathematical function in machine learning from any arbitrary input and output Euclidean spaces (Hornik et al., 1989). Achieving this approximation requires optimizing the set of parameters composed of all the weight matrices W_l and bias vectors b_l .

$$\theta = \{W_l, b_l\}_{l=1}^L \quad (15)$$

With a training dataset $\{(X_i, y_i)\}_{i=1}^N$ and a loss function $\mathcal{L} : \mathbb{R}^{d_L} \times \mathbb{R}^{d_L} \rightarrow \mathbb{R}$, the parameters θ are optimized to minimize the loss function, ideally, across all training samples

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_i), y_i). \quad (16)$$

Minimization of the loss function is done via gradient descent, where gradients $\nabla_{\theta} \mathcal{L}$ are computed using a backpropagation algorithm that leverages the multi-order differentiability of f_{θ} . However, this is only an idealization, as it is computationally expensive with the usual training dataset sizes. The practical method is to divide the training dataset into B randomly-sampled batches, and computing an approximation of $\mathcal{L}$ and the gradients per batch, nudging all the parameters for every batch. Training does not necessarily end when all batches have been exhausted in a full run of the training dataset—known as an epoch—and can continue for any arbitrary subsequent number of epochs (Rumelhart et al., 1986).

These simple neural networks have been investigated in financial stock prediction studies, particularly in de Oliveira et al. (2013), where a neural network was used to predict direct stock prices, achieving favorable out-of-sample errors, and in Shen et al. (2015), where a variant of the feed-forward neural network known as a *deep belief network* was used, showcasing the overall customizability of deep learning architectures.

Long Short-Term Memory (LSTM) Networks

One disadvantage of the machine learning techniques described is that they have no intended design for sequential modelling—albeit with data structure exploits, they may be used for such. In the domain of deep learning, sequential modelling has been able to make an even greater appearance than just in statistical models like ARIMA, during the advent of recursive neural networks (RNNs). As promising as the technology sounds at the time, it experienced a range of issues, such as the vanishing and exploding gradients, wherein information from distant time steps would decay, or otherwise grow uncontrollably during backpropagation. This issue was addressed with the introduction of the Long Short-Term

Memory (LSTM) networks by Hochreiter and Schmidhuber (1997).

The core advantage of the LSTM is the “cell state”, where a vector representation of the entire machine’s “long-term memory” resides. Information in an LSTM network is controlled by three “gates”, whereby information is sourced from and injected into the cell state, and flows into the next iteration as a “hidden state”, which acts as its short-term memory that is representative of the information processed through the LSTM that encodes relevant context from previous time steps.

- Forget gate: discards information from the cell state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (17)$$

- Input gate: injects new information into the cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (18)$$

The network also computes $\tilde{C}_t$, a candidate cell state containing potentially useful new information:

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (19)$$

The network then combines the cell state from the previous timestep filtered through the forget gate with the candidate cell state filtered through the input gate. This is the new cell state:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (20)$$

- Output gate: determines the next hidden state, based on the current cell state and input.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (21)$$

In this setup, the final hidden state is $h_t = o_t * \tanh(C_t)$.

A single LSTM forward pass consists of running the now-described transformations to each one of $x_1, \dots, x_T$ consecutive feature vectors within a time window of T timesteps sequentially. The final hidden state of the last timestep h_T is supposedly an informationally

rich vector that encodes all feature information across the entire input time window, which usually can be put through a linear transformation to obtain an output vector suitable for the prediction task. The weight matrices W_f , W_i , W_C , and W_o , as well as bias vectors b_f , b_i , b_C , and b_o , altogether constitute the learnable parameters of the LSTM.

The capacity of LSTMs to “remember” relevant information from long periods of time, even more so than RNNs, is what frees it from the limitations of exploding and vanishing gradients. Fundamentally speaking, they’re more advantageous than other machine learning algorithms due to their nature of being a sequential data processor—one that remembers long-term patterns.

The LSTM being a deep-learning model gives it the flexibility to be built into other systems, as well as be equipped with other architectures to form hybrids.

- **Stacked LSTMs:** For instance, LSTMs can be stacked upon themselves, which is a natural way to increase its model complexity, and be able to learn even more abstract and complex market realities. Stacked versions have consistently outperformed single-layer versions in predicting the NASDAQ and other Asian indices (Muruganandam & Arumugam, 2022).
- **Bidirectional LSTMs (Bi-LSTMs):** A version of LSTMs was created in order to perform sequential modelling not only forwards, but also backwards through time. This enabled sequential modeling in tasks like technical analysis, where the relationship between timesteps is bidirectional (Rudhistiar et al., 2025), as well as language modelling, where words can be inferred from surrounding context, as opposed to just preceding context (Rahman et al., 2021).
- **CNN-LSTM:** Several studies in finance have also tried predicting the stock market with a hybrid of CNN and LSTM, due to its capacity to extract features from localities in OHLCV data, where CNN architectures are advantageous, while leveraging the sequential nature of stock prices (Liu, 2025).
- **Multi-output LSTMs:** as opposed to single-output models that can predict only one

output at a time, a multi-output LSTM can simultaneously predict a number of related target variables, or even the next few time steps of a single variable under a fixed prediction horizon. The advantage of multi-output models over multiple single-output ones is a shared representation of the feature mix conducting predictions, theoretically improving consistency with structural co-movements and correlations between targets (Kerdprasop et al., 2025).

Transformers

LSTMs used to be the state-of-the-art technology for deep learning, even with applications in language modelling, since language is inherently sequential data (even bidirectionally so). This was the case up until the transformer architecture was introduced by Vaswani et al. (2017), with its core innovation being the *attention mechanism*, allowing the model to weigh token-level importances in language simultaneously instead of sequentially, dramatically improving the quality of contextual embeddings of phrases and sentences. Language tokens can be swapped for timesteps; transformers have also been found to be useful for time-series tasks (Jang, 2025).

The main advantage of transformers over recursive neural networks is how the attention mechanism does away with recurrence; cross-vector dependencies are computed simultaneously and independently irrespective of the “sequential distance” between vectors. Below is a rigorous mathematical description that summarizes the overall workings of the transformer architecture.

- Scaled dot-product attention: The attention function

Attention : $\mathbb{R}^{n \times d_k} \times \mathbb{R}^{m \times d_k} \times \mathbb{R}^{m \times d_v} \rightarrow \mathbb{R}^{n \times d_v}$ takes in three sequences of vectors called the *query vector matrix* $Q \in \mathbb{R}^{n \times d_k}$, the *key vector matrix* $K \in \mathbb{R}^{m \times d_k}$, and the *value vector matrix* $V \in \mathbb{R}^{m \times d_v}$, and outputs a matrix of as many transformed vectors as queries of the same dimensionality as the value vectors. It does so by computing an $n \times m$ matrix of *attention weights* (the softmax scores obtained in the following displayed equation) where each value supposedly denotes how much the

corresponding query vector “attends to” the corresponding key vector. A weighted sum of all the value vectors is computed for every query row on this matrix using weights on that row (elegantly equivalent to simple matrix multiplication of the attention weight matrix with V). In full, the attention function is given as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V \quad (22)$$

One caveat behind this computation is the scaling by $1/\sqrt{d_k}$, which is done in order to prevent the weight matrices from growing too large in magnitude.

- Multi-head attention: The described attention mechanism occurs in what is known as a *single head of attention*. In transformers, multiple heads are used in order to leverage learning from various ways of abstract vector representations of the same data. Essentially, the described process is done for every head, which is a function $\text{head}_i^{(l)} : \mathbb{R}^{n \times d_{\text{model}}} \times \mathbb{R}^{m \times d_{\text{model}}} \rightarrow \mathbb{R}^{n \times d_v}$ defined as follows (let h be the number of heads):

$$\text{head}_i^{(l)}(X, Y) = \text{Attention} \left(XW_i^{Q(l)}, YW_i^{K(l)}, YW_i^{V(l)} \right), \quad i = 1, \dots, h \quad (23)$$

where $X \in \mathbb{R}^{n \times d_{\text{model}}}$ and $Y \in \mathbb{R}^{m \times d_{\text{model}}}$ are two input sequences of d_{model} -dimensional vectors, and matrices $W_i^{Q(l)} \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^{K(l)} \in \mathbb{R}^{d_{\text{model}} \times d_k}$, and $W_i^{V(l)} \in \mathbb{R}^{d_{\text{model}} \times d_v}$ are learnable linear transformations that bring the input sequence vectors into each head’s unique subspace of vector representations. Typically, $d_k = d_v = d_{\text{model}}/h$ is chosen in order to match the computational cost of a single head of attention with d_{model} -dimensional input vectors, but this is not restrictive. In order to summarize the outputs from all attention heads, all outputs are concatenated and projected back into the model’s d_{model} -dimensional subspace using another learnable weight matrix $W^{O(l)} \in \mathbb{R}^{hd_v \times d_{\text{model}}}$. All in all, the function $\text{Multihead}^{(l)} : \mathbb{R}^{n \times d_{\text{model}}} \times \mathbb{R}^{m \times d_{\text{model}}} \rightarrow \mathbb{R}^{n \times d_{\text{model}}}$ wraps the entire multi-head attention process as follows:

$$\text{Multihead}^{(l)}(X, Y) = \text{Concat}_{2\text{nd dim}} \left(\text{head}_1^{(l)}(X, Y), \dots, \text{head}_h^{(l)}(X, Y) \right) W^{O(l)}. \quad (24)$$

Notice that $\text{Multihead}^{(l)}$ preserves the shape of X .

- Feed-forward network: A shape-preserving feed-forward neural network

$\text{FFN}^{(l)} : \mathbb{R}^{d_{\text{model}}} \rightarrow \mathbb{R}^{d_{\text{model}}}$ constitutes the next component of the architecture, defined by

$$\text{FFN}^{(l)}(z) = \max \left(0, zW_1^{(l)} + b_1^{(l)} \right) W_2^{(l)} + b_2^{(l)} \quad (25)$$

where $W_1^{(l)} \in \mathbb{R}^{d_{\text{model}} \times d_{ff}}$, $W_2^{(l)} \in \mathbb{R}^{d_{ff} \times d_{\text{model}}}$, $b_1^{(l)} \in \mathbb{R}^{d_{ff}}$, and $b_2^{(l)} \in \mathbb{R}^{d_{\text{model}}}$ are learnable weights and biases such that d_{ff} is larger than d_{model} (typically, $d_{ff} = 4d_{\text{model}}$).

- Residual connections and layer normalization: Both the multihead attention mechanism and the feed-forward network are wrapped in residual connection and layer normalization:

$$\text{Layer}^{(l)}(X, Y) = \text{LayerNorm}_2^{(l)} \left(X' + \text{FFN}^{(l)}(X') \right), \text{ where} \quad (26)$$

$$X' = \text{LayerNorm}_1^{(l)} \left(X + \text{Multihead}^{(l)}(X, Y) \right). \quad (27)$$

It is understood that residual connection, the layer normalization functions, and FFN are applied position-wise (since all operations map from $\mathbb{R}^{d_{\text{model}}}$ to $\mathbb{R}^{d_{\text{model}}}$) to recover $\mathbb{R}^{n \times d_{\text{model}}}$ matrices each time.

The $\text{Layer}^{(l)}$ function is known as a single transformer block (or transformer layer).

Composing N such layers as follows, each with different parameters, gives a near-complete picture of the architecture. Let $X^{(0)} = X$

$$X^{(l)} = \text{Layer}^{(l)} \left(X^{(l-1)}, Y \right), \quad l = 1, \dots, N \quad (28)$$

$$\text{CrossLayers}(X, Y) = X^{(N)}. \quad (29)$$

The application of these transformer layers can now be contextualized with example data inputs. A transformer provides versatility for either only one kind of data input, for which the *self-attention* mechanism is appropriate or two kinds, for which *cross-attention* is appropriate. Since self-attention can be seen as a special case of cross-attention, an

exposition of how to implement the cross-attention mode is given as follows: with two input sequences of feature vectors $X_{\text{in}} \in \mathbb{R}^{n \times d_x}$ and $Y_{\text{in}} \in \mathbb{R}^{m \times d_y}$ (which may stand for feature vector sequences of two different data modalities), mappings $M_x : \mathbb{R}^{d_x} \rightarrow \mathbb{R}^{d_{\text{model}}}$ and $M_y : \mathbb{R}^{d_y} \rightarrow \mathbb{R}^{d_{\text{model}}}$ are applied position-wise to obtain embedding vectors $X_e \in \mathbb{R}^{n \times d_{\text{model}}}$ and $Y_e \in \mathbb{R}^{m \times d_{\text{model}}}$. These vectors are further added position-wise with positional embeddings $P_x \in \mathbb{R}^{n \times d_{\text{model}}}$ and $P_y \in \mathbb{R}^{m \times d_{\text{model}}}$ respectively, that encode positional information about sequential order, since attention is permutation-invariant. Let X and Y be the output of these transformations:

$$X = M_x(X_{\text{in}}) + P_x \quad (30)$$

$$Y = M_y(Y_{\text{in}}) + P_y \quad (31)$$

One can now implement cross-attention by computing $\text{CrossLayers}(X, Y)$. One may also perform self-attention by computing $\text{CrossLayers}(X, X)$. However, the outputs of these functions are still sequences of d_{model} -dimensional vectors. Often in practice, one takes the last vector in the sequence, which, just like the final hidden state of an LSTM output, can be trained by connection to an output layer to aggregate all information across the entire sequence.

A small caveat worth mentioning in the given formalism is that X_{in} and Y_{in} can be of any positive sequence length; even though the given formalism defines functions like $\text{head}_i^{(l)}$ as mappings from $\mathbb{R}^{n \times d_{\text{model}}}$ for instance, the formalism rather describes these as families of functions, where in a family, there is one function for each possible n , that all share the same weights and biases $W_i^{Q(l)}$, $W_i^{K(l)}$, and so on. This completes the current exposition of the transformer architecture (Vaswani et al., [2017](#)).

As mentioned before, transformers are designed to handle sequential data, such as both natural language and time-series data. In the realm of language modelling, a kind of transformer capable of creating semantically rich contextualized embeddings for NLP tasks is known as the Bidirectional Encoder Representations from Transformers (BERT) model,

which was pretrained on a corpus designed so that text inference occurs both in the forward and backward directions (Devlin et al., 2019). The pretrained BERT model, however, is designed for fine-tuning on other datasets with specific tasks due to the design of its CLS token, making it suitable for specialization in the financial domain and even foreign languages. Implementations of the attention mechanism are largely varied due to its customizability, some of which are derivative forms of the transformer like some of the examples in the following.

- **FinBERT:** As a product of fine-tuning on a large corpus of financial communications, FinBERT is a BERT model capable of a deep understanding of financial texts, such as financial news, earnings reports, and financial research (Araci, 2019). It is packaged with sentiment analysis capabilities, and has been used in financial NLP-related stock prediction modelling (P. Chen et al., 2024).
- **Multilingual BERT (mBERT):** The Multilingual BERT model is, in self-explanatory terms, a BERT model that can process different languages, as it was pre-trained on Wikipedia articles in 104 different languages. However, when one might think that it can do well only on single-language tasks, it surprisingly does well on transfers between data in one language and a task in another, as well as understanding language with code-switching very well (Pires et al., 2019).
- **Hierarchical BERT:** The sentence-level Hierarchical BERT model is a specific way of implementing BERT that allows for document classification. It exploits the capacity of BERT models to summarize entire token sequences into a single token in order to obtain sentence tokens, which, in a hierarchical manner, be summarizes similarly to obtain entire document summary tokens. These summarized outputs can be used for downstream tasks like long document classification, which surprisingly gives promising performance scores even with minimal fine-tuning datasets (Lu et al., 2021).
- **Attention Pooling:** In processes that transform a number of fixed-sized

representational vectors into a single one (such as in summarizing many groups of pixels in an image into a smaller matrix, or many sentential contextual embeddings into a single one for a “paragraph” or document embedding), tools such as max pooling or average pooling are utilized for the necessary compression. However, this may lead to loss of important information. Inspired by how transformers use attention to preserve information, attention pooling was developed, employing a process that assigns weights to components learned through attention mechanisms (Ji et al., 2025). This allows the pooling process to “focus” on important information while dampening the effect of irrelevant data, thereby negating the “information blur” effect of uniform methods of pooling.

Financial time series forecasting has moved from statistical approaches, to machine learning algorithms, to deep learning algorithms, and to even more convoluted extensions, hybrids, and architectures that, in their vast quantity, even vast literature and active research cannot exhaust them. While the evolution of methods has introduced only better modeling frameworks and systems of prediction, classical models like ARIMA remain useful for financial markets today. With tree-based models like XGBoost and random forest being capable of capturing more complex relationships than linear, as well as the capacity of LSTMs and transformer models to store, process, and further enrich contextual information of substantial quality, the evolution of financial forecasting tools will continue to deepen and push the boundaries of analysis and prediction. That said, the trajectory of financial forecasting is starting to delve deep with hybrid models, trend towards spatial feature extraction, and deep learning augmentations given its powerful flexibility and uniqueness-factor generation for further literature and practical deployment.

Methodologically Informative Studies

As mentioned earlier, the evolution of financial forecasting followed the timeline of the development of statistical, machine learning, and deep learning methods. An emphasis is placed on deep learning algorithms at this point, in order to analyze studies with similar

goals to the current paper. The deep learning architecture on this topic is vast—unique architectures and simpler, mainstream architectures have been tested, and the methodological advantages of each have been identified. Methodological choices are not constrained by the choice of architectures only; the nature of the prediction task, performance metrics, and many other technical choices bear considerable significance. Below is a discussion of studies that have further informed the methodological choices made in particular for the current paper.

Data Structure and Preprocessing

One interesting methodological decision in machine learning-based stock prediction is whether the prediction target must be continuous (direct stock prices or stock returns) or categorical (predict positive, negative, and/or neutral returns). For continuous stock price/return prediction, several studies have demonstrated that deep learning architectures, such as even just a regular feed-forward neural network (FFNN), performed well with a combination of OHLC, technical, and fundamental features on predicting direct stock prices, with a POCID (Percentage of Correct Directional Prediction) score of 93.62% (de Oliveira et al., 2013). Mohan et al. (2019) has also established promising MAPE (Mean Absolute Percentage Error) scores of 2.03-2.17% on direct stock prices through various RNN-LSTM architectures that incorporate stock price information, and/or news data either as sentiment polarities or TF-IDF (Term Frequency-Inverse Document Frequency) contextual vectors. Even some transformer-based architectures were trained on a regression task S. Li et al. (2025). Other studies explore the option of using stock returns as the target. Shen et al. (2015) demonstrated the difference between predicting direct prices and returns—predicted prices appear only to follow the previous timestamp’s actual price fluctuation, giving the appearance that predicted prices merely lag the actual prices, which trivially gives it low MAPE scores; return predictions, on one hand, follow the fluctuations to a less extent, which may result in less favorable MAPE scores, but may provide more nontrivial buy/sell signals. Treating stock prediction as a regression task may provide, on

top of buy/sell signals, a scaling of magnitude of benefits or importance of signals. However, such signals may be useful only with less noisy stock prices—such as with daily prediction frequencies that de Oliveira et al. (2013), Mohan et al. (2019), and Shen et al. (2015) worked with. In the intraday setting, however, Zaznov et al. (2024) presents multiple reasons as to why the prediction target must be categorical. Firstly, at the minute-level frequencies, the market microstructure noise is less stable than the noise of daily closing prices—regression models would learn minute-level variations that do not correspond to meaningful trends (overfitting). If minute-level variations correspond to actual signals (such as market inefficiencies), such signals provide information on price movement direction, instead of exact magnitudes of price changes, which favors classification all the more. This is unlike the daily interval case, where market fluctuations are dominated by evolving sentiment, evolving volatility, and developing global events, which provide trajectory information (not only direction), making regression models appropriate. One would think therefore that classification should equivalently perform well in daily frequencies than in intraday frequencies, but interestingly, Khan et al. (2023) showed that 15-minute level prediction exhibited greater accuracy scores than daily prediction, thereby showing evidence that market inefficiencies may actually provide more directional information than daily signals. This may seem to be at odds with regression being more appropriate in the daily frequency case than in intraday cases, but this may be explained by the tradeoff between directional and trajectorial information between intraday and daily frequencies. In line with classification for intraday frequencies, classification provides more robust predictions; positive, negative, and neutral categories are less prone to errors than attempting to guess exact continuous values. Moreover, the less complex task of classification allows for faster computational times, therefore more rapid trading decisions for institutional and high frequency investment. Another consideration worth taking is whether the prediction task should be binary classification (positive return or non-positive return), or multiclass classification (negative, neutral, or positive return).

Studies like He and Gu (2021) support having a neutral class, where -0.75% to $+0.75\%$ returns were considered “neutral” for daily returns; other studies (R. Kim et al., 2019) that adopt a neutral class also feature giving a range of values centered at zero for the neutral class instead of just zero alone, where the width of the range depends on the frequency of the prediction task. However, Q. Chen (2021) provides support for binary classification, which was demonstrated to perform better than three-class classification, a choice argued to be appropriate only when there is a clear theoretical threshold for what constitutes a positive/negative price movement from a “neutral” movement. J. Chen et al. (2026) also showcases the use of binary classification without much justification, showing that it can be a arbitrary choice to make not limited by any strict rule or standard.

On top of modes of prediction, another important methodological decision is based on the treatment of market conditions during the few minutes after market open and before market close. Olorunnimbe and Viktor (2022) pointed out that high-frequency trading at those periods is risky, due to the significantly higher volatility and volume of transactions. The same concern also extends to lunch gaps for stock exchanges that exhibit recess periods; studies like Y. Wang et al. (2026) have shown that model accuracy irregularities do not happen only during the market open, but also during the afternoon’s opening period right after the lunch break, due to which the study dubs the period as a “second opening”. Essentially, conditions during these periods are unique, and are driven by a number of causes. Soebhag et al. (2024) established the effect of institutional investments driven by investors who “buy the dip” on losing stocks, and short-sellers buying back shares, during the closing period, causing a sharp reversal (sharp upticks) during closing periods that deviates heavily from intraday momentum. This is consistent with the fact that activities during market close are likely dominated by market exposure adjustments in anticipation of overnight volatility—from overnight news, movements of FX rates, commodities, oil prices, treasury yields, and new releases of fundamental information—while activity during market open is dominated by investors correcting their positions due to the actual

volatility observed overnight (Gerety & Mulherin, 1992). These all establish that stock prices during these periods behave erratically—reactions from intraday momentum turn into sudden reversals, and anticipatory behavior during market close may differ substantially from actual overnight volatility. Unusual trading behaviors during the market open and close periods can, however, be informative nonetheless for stock prediction hours or even days after such events; X. Wang et al. (2015) established that negative returns after overnight gaps correlated with volatility, in line with the same notion that as an asset’s value decreases, more investors “buy the dip”, only that this time, the effect being studied is a negative return that happens across the overnight gap causing volatility effects afterward, as opposed to Soebhag et al. (2024) where the effect being studied is a momentum of negative returns sharply reversing into an uptick. These effects indicate that even though market open and close periods are characterized by unusual patterns, machine learning may benefit from including such periods nonetheless during training, which X. Wang et al. (2015) have also demonstrated using an autoregressive model. They also established that the same effect occurs, albeit to a lesser extent, on lunch breaks—in some stock markets like the PSE where trading stops for a period during lunch break, return spikes similar to (but smaller than) that of the overnight gap occur over the break, and that just like the overnight case, a correlation between (negative) lunch break returns and subsequent volatility can be observed. Handling intraday stock price prediction therefore includes dealing with these boundary conditions; Huddleston et al. (2021) established, as they claim, the largest study on machine learning prediction of 5-minute interval stock prices, over 501 stocks. They adopted a method that compares model performance with technical indicators either allowed to or prohibited from running across the overnight gap, wherein findings suggest that technical indicators running across the gap offered no performance improvements due to auction-based volatility. They also included all of such data in training. However, for testing, they excluded the stock price returns during market open and close (particularly the first and last five minutes) due to auction-based volatility.

Other studies such as (Goluza et al., 2024) also deliberately designed implementations to prevent trading at the closing period for the same reason; technical indicators were also not allowed to be computed across overnight gaps. The same study also computed a “minutes until trading day ends” feature, which can help the model associate noisy periods to extreme values of the feature. The same treatment of technical indicators is found in other studies, such as in (Son et al., 2012), where only timesteps with complete input windows for feature generation are included within the training dataset. Son et al. (2012) also only includes timestamps with output predictions that do not cross into the bidding period at the end of the trading day or into the next trading day. In terms of lunch breaks, the same methodological adjustments should similarly work; in fact, despite findings of lunch breaks behaving similarly to market open and close conditions, less strict rules can be applied for these periods, since any effect would likely be less dramatic given the brevity of the lunch period compared to the overnight gap—not as much news articles get published—as observed by Ito and Lin (1992) on the Tokyo stock exchange, which also features a market shutdown during the lunch period. They argue that trading activity during the lunch period drives lunch period volatility; therefore, the lack of it due to a lunch break shutdown leads to lower volatility. A more visible account of the overnight and lunch break price jumps can be seen in Kong et al. (2020), where price jumps during market open, lunch break, and market close periods are apparent, with lunch break jumps being less in magnitude. The effects for both overnight and lunch break gaps were attributed to the buildup of fundamental information during such periods when the market is frozen. Other machine learning applications include that of Q. Chen (2021), where the effects of news articles published during overnight gaps are evaluated based on their 1-day effect on price returns—not the effect on the opening or slightly-after-opening prices of a trading day.

Crucial for the efficacy of technical data in financial prediction is the use of technical indicators, instead of only raw OHLCV values, as already exemplified by some studies mentioned (Goluza et al., 2024; Huddleston et al., 2021; Son et al., 2012). Most studies

support the use of multiple technical indicators derived by feature engineering from the raw OHLCV values, such as Basak et al. (2018), where a number of technical indicators, such as price momentum indicators—Relative Strength Index (RSI) and Stochastic Oscillator (SO)—and a trend indicator—namely Moving Average Convergence Divergence (MACD)—were used to make single-timestamp inputs informative of recent time steps (which can therefore be used as inputs for non-deep learning ML techniques such as logistic regression, SVM, and tree-based models). Lohrmann and Luukka (2018) also made use of volatility indicators such as Bollinger Bands, and volume indicators. A more complete review of technical indicators used in the literature is detailed in the following.

1. Relative Strength Index (RSI): Pioneered by Wilder (1978), the RSI is an indicator of price momentum and potential price reversal. It ranges from 0 to 100, and it can be interpreted as some kind of “rate of change” wherein the higher values indicate rising stock prices, and lower values indicate falling prices. Significantly higher values, however (> 70) indicate significant buying pressure, which may signal a potential market reversal. Similarly, lower values (< 30) indicate significant selling pressure, which may also result in a market reversal. By principle, it is a hybrid between a rolling metric, depending primarily on the previous n (traditionally 14) timesteps, and a metric with infinite memory, whereas as time accumulates, more distant time steps are nonetheless part of its calculation, albeit dampened. The calculation of the metric is done as follows:

$$\text{RSI}(n)_t = 100 + \frac{100}{1 - \text{RS}(n)_t} \quad (32)$$

where

$$\text{RS}(n)_t = \frac{\text{Average Gain}(n)_t}{\text{Average Loss}(n)_t} \quad (33)$$

with $\text{Average Gain}(n)_t$ and $\text{Average Loss}(n)_t$ being the mean of the positive (equal to 0 if the return is negative) and negative returns (0 if the return is positive) respectively over the n -period window inclusive of the current timestep, but only for

the first timestep t that the $\text{RSI}(n)$ can be calculated (assuming that timesteps begin at timestep 1, and thus with returns beginning at timestep 2, this corresponds to $t = n + 1$). For any subsequent timestep t , $\text{Average Gain}(n)_t$ and $\text{Average Loss}(n)_t$ are computed as follows:

$$\text{Average Gain}(n)_t = \frac{(n - 1) (\text{Average Gain}(n)_{t-1}) + \text{Gain}_t}{n} \quad (34)$$

$$\text{Average Loss}(n)_t = \frac{(n - 1) (\text{Average Loss}(n)_{t-1}) + \text{Loss}_t}{n} \quad (35)$$

where Gain_t and Loss_t are understood to be the positive and negative return, respectively, of the current timestep t . This ensures that recent timesteps dominate the $\text{RSI}(n)$ while taking into account all past timesteps indefinitely. This recursive and infinite-memory method of calculating rolling window aggregates is often attributed to Wilder himself within the literature Gurrib (2018).

2. Stochastic Oscillator: Introduced by Lane (1984), the Stochastic Oscillator is a set of two technical indicators, namely $\%K$ and $\%D$, which signals the momentum of a stock price (Thorp, 2000). $\%K$ is a measure of the relative position of the closing price between the highest and lowest prices within a chosen number of timesteps k , while $\%D$ is a slower-moving indicator of this relative position, as it is a 3-point simple moving average of $\%K$.

$$\%K_t = \frac{p_t - \min(l_{t-k+1}, \dots, l_t)}{\max(h_{t-k+1}, \dots, h_t) - \min(l_{t-k+1}, \dots, l_t)} \quad (36)$$

$$\%D_t = \frac{\%K_t + \%K_{t-1} + \%K_{t-2}}{3} \quad (37)$$

where p_t , h_t , and l_t are the close, high, and low prices respectively. In this setup, k typically can take a value of 5 (Thorp, 2000). A number of relevant signals is discussed in (Lane, 1984), namely *divergence*—where the local minima or maxima of $\%D$ (or $\%K$) trend in the opposite direction as the corresponding local minima or maxima (respectively) of the stock price, which indicates intra-bar price movements opposite the inter-bar trend, hence the start of an overbuying/overselling-induced

reversal, *hinge*—where %K or %D experiences a slowdown in momentum, indicating potential approach to a minimum or maximum before reversal, *warning*—where %K experiences a sharp reversal, *crossover*—where %K and %D experience a reversal simultaneously but %K outpaces %D, and %K *reaching the 0% or 100% extremes*—which all signify a sharp reversal of closing prices, as well as *failure*—where %K rebounds after a short-lived reversal, indicating a continuation of the stock’s current momentum. Other implementations include two %D indicators, the described being the *fast* %D oscillator, and the other called the *slow* %D indicator, which is simply the simple moving average of the fast %D (Kong et al., 2020). This provides additional information for identifying overbought or oversold market conditions.

$$\text{Slow \%D} = \frac{\%D_t + \%D_{t-1} + \%D_{t-2}}{3} \quad (38)$$

The same principles apply no matter the frequency of the timesteps—weeks, day, minutes (Thorp, 2000)—Kong et al. (2020) explored the efficacy of stochastic oscillators on the prediction of intraday jumps of stock prices, highlighting the use of stochastic oscillators on more granular frequencies.

3. Price Log Returns: Usually taken as the core signal, price log returns measure the per-interval change in closing prices, which is crucial as it functions as a differenced feature. Financial variables are known to exhibit random walk behavior (Fama, 1970), meaning that they exhibit high autocorrelation with recent timestamps, making it crucial to transform financial features through differencing. Moreover, since returns exhibit highly erratic behavior, logarithmized returns offer more stable, more normally distributed values. Studies like Kong et al. (2020) use log returns to capture the highest granularity of price momentum, where return values of high magnitude may indicate persistence or reversal. It is computed as follows:

$$r(n)_i = \ln p_i - \ln p_{i-n} \quad (39)$$

4. Price Log Cumulative Return: This captures the cumulative intraday return for every

interval, taken as the difference between the current closing price and the closing price of the previous day's trading session (Kong et al., 2020). It also captures information about price momentum, which can be useful for anticipating reversals, and also gives every timestamp a direct measure of overnight gap effects from before the current trading session. Let i be the current trading day and n be the last interval of every trading day. The cumulative return is given as:

$$R_{i,t} = \ln p_{i,t} - \ln p_{i-1,n} \quad (40)$$

5. Price Rate of Change: Also known as PROC, these are differences between the current price and prices from a number n of periods ago (Kong et al., 2020), which is analogous to price log returns, just relative to a different lag. It is a less sensitive measure of price momentum, which can capture trends across longer horizons. It is computed as follows:

$$\text{PROC}(n)_t = \frac{p_t - p_{t-n}}{p_{t-n}} \quad (41)$$

6. Moving Averages: Several kinds of moving averages are often used in technical analysis, which are methods that find central tendencies of the most recent n timestamps, for arbitrary n . The simplest kind is the Simple Moving Average, which provides equal weight to all recent timestamps (Tanaka-Yamawaki & Tokuoka, 2007):

$$\text{MA}(n)_t = \frac{1}{n} \sum_{i=0}^{n-1} p_{t-i} \quad (42)$$

Moving averages provide a more stable series of the closing prices, which can be used both as a technical indicator on its own and as a reference for other momentum-based technical indicators (Son et al., 2012). Another kind is the Exponential Moving Average, which, as opposed to the Simple Moving Average, gives more weight to more recent timesteps. It is computed recursively, as follows (Kang, 2021):

$$\text{EMA}(n)_t = \frac{2}{n+1} (p_t - \text{EMA}(n)_{t-1}) + \text{EMA}(n)_{t-1} \quad (43)$$

7. Moving Average Convergence Divergence: Also known as MACD, this is a widely-known momentum indicator with a well-established set of buy and sell signals like the RSI and Stochastic Oscillators. It has also been used much in the literature to examine market efficiency (Kang, 2021). It is composed of three components, namely the *main MACD line*, which is computed by taking the difference between a short-term EMA with period n_1 and long-term EMA with period n_2 , the *signal line*, which is an EMA (with a different period, say n_3) of the main MACD line, and the *histogram*, which is the difference between the main and signal lines.

$$\text{MACD}(n_1, n_2)_t = \text{EMA}(n_1)_t - \text{EMA}(n_2)_t \quad (44)$$

$$\text{Signal}(n_1, n_2, n_3)_t = \frac{2}{n_3 + 1} (\text{MACD}(n_1, n_2)_t - S) + S \text{ where } S = \text{Signal}(n_1, n_2, n_3)_{t-1} \quad (45)$$

$$\text{Histogram}(n_1, n_2, n_3)_t = \text{MACD}(n_1, n_2)_t - \text{Signal}(n_1, n_2, n_3)_t \quad (46)$$

The choice of n_1 , n_2 , and n_3 is up to the trader, but traditional choices include $n_1 = 12$, $n_2 = 26$, and $n_3 = 9$. These three components can be used together to obtain the following buy and sell signals, under trading rules known as the *zero crossover* and the *signal line crossover*.

$$\text{zero crossover} = \begin{cases} \text{Buy when } \text{MACD}(n_1, n_2)_t > 0 \\ \text{Sell when } \text{MACD}(n_1, n_2)_t < 0 \end{cases} \quad (47)$$

$$\text{signal line crossover} = \begin{cases} \text{Buy when } \text{Histogram}(n_1, n_2, n_3)_t > 0 \\ \text{Sell when } \text{Histogram}(n_1, n_2, n_3)_t < 0 \end{cases} \quad (48)$$

Either trading rule can be used, although the latter is more popular in recent literature (Kang, 2021).

8. Price Oscillators: These indicators derive from the moving averages by taking the rate of change from the long-term (with period n_2) to the short-term (with period n_1)

moving averages (Son et al., 2012):

$$\text{OSCP}(n_1, n_2)_t = \frac{\text{MA}(n_1)_t - \text{MA}(n_2)_t}{\text{MA}(n_2)_t} \quad (49)$$

$$\text{EOSCP}(n_1, n_2)_t = \frac{\text{EMA}(n_1)_t - \text{EMA}(n_2)_t}{\text{EMA}(n_2)_t} \quad (50)$$

These indicators, together with the moving average indicators, have been shown to be effective for predicting intra-day stock prices (Tanaka-Yamawaki & Tokuoka, 2007).

9. Price Disparity: These indicators also derive from moving averages by taking the rate of change from the moving average (with period n) to the current closing price.

These, together with Price Oscillators and Moving Averages, provided the best setting for intraday stock prediction for some studies (Son et al., 2012; Tanaka-Yamawaki & Tokuoka, 2007).

$$\text{DISP} = \frac{p_t - \text{MA}(n)_t}{\text{MA}(n)_t} \quad (51)$$

$$\text{EDISP} = \frac{p_t - \text{EMA}(n)_t}{\text{EMA}(n)_t} \quad (52)$$

10. Psychological Line: One of the lesser known indicators, this indicator provides a means of anticipating reversals by counting the number of closing price upticks within the last n periods, where theoretically, a large number would indicate an upcoming reversal due to overbought conditions, and vice versa (Tanaka-Yamawaki & Tokuoka, 2007).

$$\text{PL}(n)_t = \frac{\# \text{ of upticks}}{n} \quad (53)$$

11. Rank Correlation Index: Also known as RCI, this indicator measures the “strength” of the market by measuring the directional persistence of stock closing prices over the lookback window. The indicator performs this by ranking the closing prices within the window, taking the Spearman correlation between the price ranks and the temporal ordering of the stocks. This means that high, positive values (maximum of 1) indicate near-monotonically increasing prices, low, negative values (minimum of

−1) indicate near-monotonically decreasing prices, and values close to 0 indicate almost no persistent trends (Tanaka-Yamawaki & Tokuoka, 2007). Since indicator correlates ranks, it is mostly agnostic to the magnitudes of the closing prices, inasmuch as they do not affect the rankings.

$$\text{RCI}(n)_t = 1 - \frac{6 \sum_{i=0}^{n-1} (\text{Rank}(p_{t-i}) - (n-i))^2}{n(n^2 - 1)} \quad (54)$$

12. On-balance Volume: Also known as OBV, this is a volume indicator that describes how volume is influenced by the directionality of returns, where it accumulates volume by adding a timestep's volume when the return is positive, and subtracting when negative. It is given by

$$\text{OBV}_t = \text{OBV}_{t-1} + \begin{cases} v_t & \text{if } p_t > p_{t-1} \\ 0 & \text{if } p_t = p_{t-1} \\ -v_t & \text{if } p_t < p_{t-1} \end{cases} \quad (55)$$

The OBV indicator has been shown to be effective in enhancing trading strategies in Chinese stock exchanges (Tsang & Chong, 2009).

13. Relative Volume: Also called RVOL, this is another volume indicator that describes the volume of the current timestep in relation to those of the last n timesteps. This has been found to indicate deviations in trading intensity, which can be influenced by shifting market expectations, making it an informative indicator (Karpoff, 1987).

$$\text{RVOL}(n)_t = \frac{v_t}{\frac{1}{n} \sum_{i=0}^{n-1} v_{t-i}} \quad (56)$$

14. Bollinger Bands: This is a set of three indicators that function as volatility indicators as well as monitor whether conditions are overbought or oversold. Given a lookback period, it computes the upper and lower bands, which are two standard deviations away from a moving average, and it takes the relative position of the current closing

price within the bands as the third indicator (Deep et al., 2024). With $\sigma(n)_t$ denoting the standard deviation of the closing prices from the previous n timesteps ending at t , it is computed as:

$$\text{BBU}(n)_t = \text{MA}(n)_t + 2\sigma(n)_t \quad (57)$$

$$\text{BBL}(n)_t = \text{MA}(n)_t - 2\sigma(n)_t \quad (58)$$

$$\text{BB}\%(n)_t = \frac{p_t - \text{BBL}(n)_t}{\text{BBU}(n)_t - \text{BBL}(n)_t} \quad (59)$$

The latter indicator $\text{BB}\%$ can be used to identify overbought/oversold conditions with the help of other indicators. For instance, if $\text{BB}\%$ is higher than 1 or less than 0 (indicating that it's outside the BBU and BBL bands), it signals rapid price movements, which, with the help of other technical indicators, can forecast future directionality of returns. The BBU and BBL bands can further be used to determine trend-setting conditions. They are direct measurements of volatility magnitudes, which can amplify during the start of trends, and even forecast future volatility (Bollinger, 2001).

15. Parabolic SAR: Also known as PSAR, this indicator makes use of the price information to define uptrend and downtrend conditions, as well as pinpoint reversals. Specifically, it recursively computes

$$\text{PSAR}_t^{\text{proj}} = \text{PSAR}_{t-1} + \text{AF}_t(\text{EP}_t - \text{PSAR}_{t-1}) \quad (60)$$

$$\text{PSAR}_t = \begin{cases} \min(\text{PSAR}_t^{\text{proj}}, l_{t-2}, l_{t-1}) & \text{if Trend}_t = \text{Uptrend} \\ \max(\text{PSAR}_t^{\text{proj}}, h_{t-2}, h_{t-1}) & \text{if Trend}_t = \text{Downtrend} \end{cases} \quad (61)$$

where l_t and h_t are the within-candle low and high prices, EP_t is the price extreme within the current trend (highest high in uptrend conditions, lowest low during downtrend conditions), and AF_t is the acceleration factor that initializes at a certain value (traditionally 0.02), and increases by a certain amount (0.02 traditionally) every time EP_t changes to a new extreme (unchanged otherwise), with a maximum,

traditionally, of 0.2. The trend is determined recursively as well, given the following conditions:

$$\text{Trend}_t = \begin{cases} \text{Uptrend if } \text{PSAR}_t^{\text{proj}} < h_t \text{ and } \text{Trend}_{t-1} = \text{Downtrend} \\ \text{Downtrend if } \text{PSAR}_t^{\text{proj}} > l_t \text{ and } \text{Trend}_{t-1} = \text{Uptrend} \\ \text{Trend}_{t-1} \text{ otherwise} \end{cases} \quad (62)$$

During times that the trend switches, some rules override the now-described definitions of PSAR_t , and AF_t . Particularly, whenever the trend changes at time t :

$$\text{PSAR}_t = \text{EP}_{t-1} \text{ (price extreme of previous trend)} \quad (63)$$

$$\text{AF}_t = \text{Initial Value (traditionally 0.02)} \quad (64)$$

Since the current trend is new, EP_t would naturally renew as well given its definition. Since PSAR features trend information and reversal forecasting, it stands as an informative indicator for price prediction (Wilder, 1978), and has been used as a technical indicator in some ML studies (Abolmakarem et al., 2024).

16. Average True Range: Introduced by Wilder (1978), the average true range (ATR) is a volatility indicator—a measure of the width of price fluctuations. It is based on the True Range (TR), namely the maximum of the absolute differences between the current period's high and low prices, the current high price and the previous period's closing price, and the current low price and previous period's closing price. The calculation of the $\text{ATR}(n)_t$ then is a Wilder-style average over a chosen window length n —Wilder (1978) chose 7 or 14 periods.

$$\text{TR}_t = \max(|h_t - l_t|, |h_t - p_{t-1}|, |l_t - p_{t-1}|) \quad (65)$$

$$\text{ATR}(n)_t = \begin{cases} \sum_{k=2}^{n+1} \text{TR}_k & t = n + 1 \\ \frac{(n-1)\text{TR}_{t-1} + \text{TR}_t}{n} & t > n + 1 \end{cases} \quad (66)$$

Both the True Range and the Average True Range can be used as features for stock prediction simultaneously (Abolmakarem et al., 2024).

17. Average Directional Movement Index: Also known as ADX, this is an indicator that measures trend strength regardless of direction (Wilder, 1978). Studies have noted its utility in stock market return prediction based on machine learning. It is a Wilder-style period sum of the DMI (Directional Movement Index) based on two directional indicators (+DI and -DI) which denote the strength of the uptrend and downtrend, respectively, of the price ranges (not returns). The calculation of the ADX is complex, as detailed in the following (Gurrib, 2018). Two range movement indicators (+DM and -DM) are calculated, whose Wilder-style n -period sums (+SDM and -SDM) are taken.

$$+DM_t = \begin{cases} h_t - h_{t-1} & \text{if } h_t - h_{t-1} > l_{t-1} - l_t \text{ and } h_t - h_{t-1} > 0 \\ 0 & \text{otherwise} \end{cases} \quad (67)$$

$$-DM_t = \begin{cases} l_{t-1} - l_t & \text{if } l_{t-1} - l_t > h_t - h_{t-1} \text{ and } l_{t-1} - l_t > 0 \\ 0 & \text{otherwise} \end{cases} \quad (68)$$

$$+SDM(n)_t = \begin{cases} \sum_{k=2}^{n+1} +DM_k & t = n + 1 \\ +SDM(n)_{t-1} - \frac{+SDM(n)_{t-1}}{n} + (+DM_t) & t > n + 1 \end{cases} \quad (69)$$

$$-SDM(n)_t = \begin{cases} \sum_{k=2}^{n+1} -DM_k & t = n + 1 \\ -SDM(n)_{t-1} - \frac{-SDM(n)_{t-1}}{n} + (-DM_t) & t > n + 1 \end{cases} \quad (70)$$

The n -period Wilder-style sum of True Range (TR) is also used to calculate $ADX(n)$, as follows:

$$ATR(n)_t = \begin{cases} \sum_{k=2}^{n+1} TR_k & t = n + 1 \\ ATR(n)_{t-1} - \frac{ATR(n)_{t-1}}{n} + TR_t & t > n + 1 \end{cases} \quad (71)$$

Finally, the calculation of the directional indices +DI(n) and -DI(n) follows, from which the DMI(n), and therefore the ADX(n) are computed, as follows:

$$+DI(n)_t = 100 \frac{+SDM(n)_t}{ATR(n)_t} \quad (72)$$

$$-DI(n)_t = 100 \frac{-SDM(n)_t}{ATR(n)_t} \quad (73)$$

$$DMI(n)_t = 100 \left| \frac{(+DI(n)_t) - (-DI(n)_t)}{(+DI(n)_t) + (-DI(n)_t)} \right| \quad (74)$$

$$ADX(n)_t = \begin{cases} \sum_{k=n+1}^{2n} DMI(n)_k & t = 2n \\ ADX(n)_{t-1} - \frac{ADX(n)_{t-1}}{n} + DMI(n)_t & t > 2n \end{cases} \quad (75)$$

The use of 14-period windows for ATR, $+/-$ SDM, and ADX is standard. After calculation of the ADX, a “strong trend” may be inferred during periods where $ADX > 20$ (Gurrib, 2018). Studies have noted the use of this indicator in machine learning contexts, particularly both in daily and intraday stock market data. Saud and Shakya (2022) provided evidence that machine learning algorithms based primarily on directional movement indicators ($+DI$, $-DI$, DMI , and ADX) perform significantly better than buy-and-hold trading strategies on daily stock price data. This study particularly noted the utility of the trading signals offered by $+DI$ and $-DI$: a crossover with $+DI > -DI$ (suggesting an uptrend in price ranges) indicates a buy signal, a crossover with $-DI > +DI$ (suggesting a downtrend in price ranges) indicates a sell signal, and a hold signal otherwise. The inclusion of $+DI$ and $-DI$ indicators apart from ADX is a viable strategy to infer direction other than strength of a trend; however, in applications with other price momentum indicators already present, such as in Khan et al. (2023) with the inclusion of RSI, ADX is sufficient. Khan et al. (2023) also supports the use of the indicator in intraday (15-minute interval) applications.

Technical indicators are useful for machine learning not only concerning stocks, but for all financial markets. Lohrmann and Luukka (2018) provided an example wherein stocks, bonds, oil, gold, materials (incl. industrial metals like copper), and foreign exchange were all included for training data, and technical indicators for every modality were computed, suggesting that not only are adjacent financial markets’ price information relevant enough to be candidate features for stock return ML prediction, but as well as their respective

technical indicators. Studies like Y. Wang (2014) also suggest that features derived sector indices of a stock can be used in such ML tasks. One consideration worth noticing is whether technical indicators and raw price features really should belong to the same feature vector (one data modality), and this is where studies differ. Some implementations treat them as a single modality (J. Chen et al., 2026; Dashtaki et al., 2025), while some treat them as separate modalities (Fataliyev & Liu, 2023). This is however not a great cause of concern, since all such studies regardless of implementation show promising results.

Since computing technical indicators can significantly increase feature dimensionality, relevant as well to the discussion of processing financial data are methods for feature selection. It is a well-known method in financial data prediction tasks to generate an initially extensive list of arbitrarily-chosen technical indicators with arbitrary periods for which feature selection would be later applied to narrow down into an optimal set. Gunduz et al. (2017) used a chi-square-based feature selection method and ranked features based on the number of times they were selected relevant for predicting hourly returns for a certain stock, to create a global set of features for 100 stocks on the Istanbul Stock Exchange. Kong et al. (2020) used mutual information and mRMR (Minimum Redundancy Maximum Relevance) to capture both target-relevant features and optimal noncollinear features. Lohrmann and Luukka (2018) used fuzzy similarity and entropy measures (FSAE) to capture the most relevant and least redundant technical indicators. Between all these methods, it is clear that the choice of feature selection methods is often algorithmic rather than manual and exploratory, although some studies prefer the manual approach, using correlation filtering and SHAP analysis instead (Dashtaki et al., 2025).

Equally important to the extraction of features for technical data is feature extraction and selection for news data. For multimodal financial data prediction tasks, one can rely on a number of natural language processing tools, including sentiment analysis and semantic textual embeddings. Studies have shown the effectiveness of simple sentiment analysis, where sentiment levels for “positive”, “neutral”, and “negative” labels are

generated by a chosen model (W. Gu et al., [2024](#)). In such cases, temporal alignment with the stock price data is often carried out by summarizing the collected sentiment data for every time interval (such as days) into averages (Halder, [2022](#)). Such studies have found that incorporating news data in such a way provided the better-performing models than using technical data alone. Even intraday applications benefit from the inclusion of news data in this manner, as seen in Ravat et al. ([2025](#)) where models were trained over trading data at a 5-minute frequency. However, more evidence points out that the specific implementation of news needs not be specific in order to extract better model performance. Instead of sentiment analysis labels, other studies have opted for full semantic textual embeddings, commonly using models fine-tuned to the financial domain like FinBERT, or even just regular pretrained models such as BERT. Fataliyev and Liu ([2023](#)) found that among three different text encoders (GLoVe, BERT, and FinBERT), FinBERT provided the best advantage for stock return prediction tasks, and studies like P. Chen et al. ([2024](#)) also utilize only FinBERT. However, studies like Q. Chen ([2021](#)) and Yadav ([2024](#)) utilize news headline embeddings from BERT only, and the methods vary yet still interestingly provide improved performances. As for P. Chen et al. ([2024](#)), news headlines for every day were pre-filtered such that only one news headline—namely the one with the highest polar sentiment score, was filtered to represent all news of a single day. This contrasts with other implementations that use mean pooling. On one hand, Q. Chen ([2021](#)) provided a method that uses every news article as a direct training or evaluation sample, with its BERT embeddings as the features and the market-adjusted return (stock price returns minus the corresponding sector index returns in order to isolate news/investor sentiment effects) direction as the prediction target. Moreover, BERT embeddings per word of the article headline were concatenated to obtain the summary embedding of the headline instead of the BERT’s CLS token, providing further evidence that the specific embedding method can be versatile. Filtering was also done for the training set by pre-relating every news article to a certain stock, and including only news articles with the top and bottom 15%

market-adjusted returns, providing stronger evidence for filtering news articles given its selective impact on certain stocks. This highlights how categorization of news data is also crucial when handling multi-stock predictions; in J. Chen et al. (2026), both local and global news were used—local ones that affect only a specific stock and are therefore only included in that stock’s prediction task, and global ones that are included in all stocks’ prediction tasks. An overall similarity between all these studies is the fact that only news headlines were used rather than the news article body. This is further supported by Shi et al. (2018), who explicitly tested model performance between including only headline text and actual full article contents, finding that only news titles provided a better advantage, perhaps due to the semantic noise of news article bodies compared to the summative relevance of a news headline.

Feature extraction for social media data is also actively researched, and shares a lot of similarities with the way news data is handled. Studies also vary between extracting only sentiment features or full semantic embeddings for the textual contents, with examples like Karadaş et al. (2025), Saravanos and Kanavos (2024), Ravat et al. (2025), P. Zhang et al. (2025), G. Sun and Li (2025), and W. Sun et al. (2026) focusing on labeled sentiment scores similarly using FinBERT’s financial sentiment classification capabilities, lexicon-based approaches such as VADER, or even just the pretrained BERT model, as well as examples for full semantic representations shown in He and Gu (2021), many of which show superior model performances compared to machine learning baselines and non-multimodal applications. While such studies share similarities with how news data features are extracted, the full implementation for social media often includes significantly more steps due to the nature of social media posts. For instance, social media data may contain user mentions and links, for which denoising must be applied by converting all such mentions and links into appropriate and uniform placeholder texts (Dashtaki et al., 2025). Moreover, given that news articles are from a professional media outlet while social media posts mostly come from regular users, there is a credibility gap worth filtering for (P. Zhang

et al., (2025). Social media posts also feature user influence and post impact features (such as author following, post likes, replies, etc.), that not only help quantify the veracity of posts, but also provide more useful feature dimensions than content alone. As such, stock prediction ML studies that utilize social media data also utilize these influence and impact features in various ways, and this is where the aforementioned studies differ significantly. Karadaş et al. (2025) and W. Sun et al. (2026), for instance, provided methods for using the authors' following and post impact features to provide weights for the content sentiment scores, allowing impact and influence scores to “moderate” the textual content's overall effects on the prediction. This makes sense—viral posts are often more influential and have a greater effect on overall market sentiment than obscure, low-engagement posts. Some studies implement this moderative effect more strictly and use influence and impact features to actually filter out posts (P. Zhang et al., (2025), deliberately shrinking the social media datasets. On one hand, studies like Saravanos and Kanavos (2024) experiment with extracting as many social media features as comprehensively possible, using a wide range of natural language processing techniques such as parts-of-speech tagging, n-grams, and sentiment analysis, which have been shown to improve machine learning performance. W. Sun et al. (2026) attempts something similar, featuring keyword mention counts, but focuses more on feature engineering like that involved in technical analysis—deriving follower-weighted mean sentiment scores, viral propagation coefficients, as well as other features to complete 20 sentiment features from news and social media as well as 10 popularity features. This study also highlights that for the sake of time-alignment in the case of multiple social media posts within a single timestep, temporal aggregation methods are reliable, similar to the case of news data. Also among such derived features within the literature include single-feature sentiment scores, obtained by subtracting the negative sentiment score from the positive sentiment score (Ridhawi et al., (2026)).

Based on the review of literature on the taxonomy of price movement drivers categorized by different frequency spaces discussed earlier, price movement by technical

indicators sits at horizons that are a few minutes at most, while language-driven effects have a different horizon profile. While Kerstenfischer and Schmeling (2024) stated that, in the U.S., up to 35% of bond and stock price movements were attributable to news, H. Lee et al. (2014), in one of the first papers that established the relevance of linguistic features such as news in the task of stock prediction, stated that the effect of textual features on stock prices diminishes quickly. However, studies like Q. Chen (2021) shed more light on the time scales in consideration, namely that they are within minutes up to a day, which are longer than horizons for only non-textual price features. Particularly, the market-adjusted returns for the prediction target were computed over a horizon of 30 minutes for news published within trading hours providing evidence for news effects that last up to 30 minutes after optimizing on a hyperparameter set of 5, 10, 20, and 30 minutes. For news published outside the trading hours, the returns were computed over 1 day, which was chosen after testing on horizons of 1, 2, 3, and 5 days, highlighting how the effects of overnight news due to market closure accumulates and cascades onto the next trading day. This suggest how, while news effects do diminish quickly, including news articles from a longer lookback window (such as 1 day) is crucial in order to capture overnight effects. W. Sun et al. (2026) also deliberately includes news data from the overnight gap due to these considerations. An analogous case can also seen for social media data; namely, investor sentiment during the overnight period has been found to be more relevant for prediction than intraday-period sentiment (G. Sun & Li, 2025), so much so that separating during-trading-session investor sentiment and post-trading-session investor sentiment into two different modalities generated superior model performance results, thus providing strong evidence for including 1-day lookback periods for social media data as well. Moreover, price returns at 5-minute intervals (Huddleston et al., 2021) and 15-minute intervals (Khan et al., 2023; Moberg & Sucarrat, 2007) were found to be optimal choices for machine learning prediction using technical features, outside the high-frequency (micro-to millisecond) scales. Vogel and Kraus (2025) on one hand were able to determine that

microstructure noise dominates in the 1-5 minute scales, whereas event-based fluctuations determine movements in the 30 to 60-minute scales. These results are informative both for prediction horizon choices as well as lookback periods given a matrix of choices for data modalities including technical, news, and social media data.

Modelling

The extensive discussion on data structures and preprocessing methods motivates choices for intraday stock return classification with prediction horizons in the scale of minutes, augmented by an extensive generation of technical indicators (with corresponding design for subsequent feature selection), with a wide variety of choices of implementations for the inclusion of news and/or social media data, and with precautions for implementation with respect to trading day boundary conditions. With that, the discussion now moves towards considerations for the model-building approach.

Modules designed to process technical data alone are comparatively the widest variety of machine learning and deep learning approaches with respect to other relevant data modalities. Traditional machine learning implementations that ingest price features with technical indicators are commonly tree-based methods like random forests (Lohrmann & Luukka, 2018) and gradient-boosted trees (Lohrmann & Luukka, 2018). Studies often show that tree-based models beat simpler models like logistic regression (Basak et al., 2018; Jarmulski, 2021) for financial forecasting, but studies particularly for intraday frequencies show otherwise, such as findings from Khan et al. (2023) that logistic regression performed better in 15-minute-interval settings than a random forest, all featured gradient-boosting methods, and even a single-layer feed-forward neural network. Other implementations include support vector machines (Beniwal, 2025) that have been found in some studies to perform better than even LSTMs and convoluted neural networks (Arora et al., 2019), which make it a comparable candidate for traditional ML. In terms of neural networks, de Oliveira et al. (2013) and Shen et al. (2015) have established promising performance results for simple feed-forward networks; however, Patel et al. (2014) and Krauss et al.

(2016) have shown that tree-based models can still outperform such models. A shift towards models with better input representation capabilities such as CNNs and LSTMs is required to meaningfully begin achieving greater results, evidenced by higher trading simulation performances than tree-based models (Ghosh et al., 2022), but even then, traditional machine learning can still provide a better advantage in terms of raw model performance scores. This is where there are no more established rules for which kind of model is “better” other than trends in the literature that are nonetheless falsifiable. Proposed models where deep-learning methods provide greater advantage usually feature more complicated architectures where the effect of the technical data module can no longer be isolated. Nevertheless, choices for technical data modules found in the literature are methodologically motivating. Such choices frequently include LSTM-based architectures. Examples include P. Chen et al. (2024), Halder (2022), W. Gu et al. (2024), T. Kim and Kim (2019), Sarkar and Shahid (2025) and Karadaş et al. (2025), where LSTM models take advantage of its sequential capabilities such that every forward pass is no longer limited to a single timestep of technical indicators. Yao et al. (2018) also uses an LSTM for high frequency (minute-level) data. A noteworthy observation is that even with such multi-timestep lookback windows, each timestep’s feature vector can still contain technical indicators defined with their own input windows, such as in P. Chen et al. (2024). This is further supported by CNN-LSTM architectures that process timesteps using the CNN before sequential processing by the LSTM Liu (2025), such that every timestep in the LSTM sequence input is already a temporally aggregated feature vector itself. While CNN modules usually complement other architectures such as LSTMs and transformers Mandal et al. (2023), CNNs can also be used for stock prediction as a standalone architecture, especially for high frequency data Zaznov et al. (2024), where its spatial aggregation capacity is generalized to multi-feature time-series applications.

A fair share of studies, however, focus on using time-series transformers or self-attention modules that simultaneously and independently compute dependencies

between different timesteps in an input sequence of feature vectors, overcoming problems produced by exploding and vanishing gradients across time in recursive neural networks. S. Li et al. (2025) and Vogel and Kraus (2025) give examples of training time-series transformers/self-attention modules for processing price and technical data series at the intraday scales (hours and minutes), while J. Chen et al. (2026) and Bustarviejo and Bousoño-Calzón (2026) give examples for daily trading frequencies, showing that transformer-based applications are also versatile in terms of trading frequencies. Unlike other models, transformers offer significantly a lot more modifiable components. Normally, attention heads are *bidirectional*—attention modules compute attention weights between input feature vectors without regard for directionality, such that, say in the case of time-series query and key sequences, any query vector can attend to any key vector from the relative past, present, or future. Since temporal relationships are causal—i.e. only the past and present affect the future and not vice versa, a method known as *attention masking* can be done, which renders causally nonviable attention cells within the query-key matrix inert, allowing only causally possible interactions to affect the residual computations. This is a method seen in D. Wang et al. (2025), which proposes an architecture involving such a causal method of building time-series transformers. Some studies also perform complex mathematical transformations to price data such as a discrete wavelet transform, which takes the data into the frequency domain, before using attention mechanisms (J. Chen et al., 2026), albeit this is more unconventional. Other unconventional implementations include that of Vogel and Kraus (2025), which modified individual attention heads within the multihead attention block to process information at different time scales, as well as including scoring function in the attention computation based on market microstructure and econometric features, showcasing that extensions of the well-established attention mechanism by Vaswani et al. (2017) are not only harmless, but also provide innovative value. However, transformers, and other deep learning architectures, are known to be parameter-heavy and therefore prone to overfitting, which is the case for datasets with lots

of non-predictive noise. Some studies adjust for this, namely by filtering out samples with exceedingly small stock return values, such as in Albada and Sonola (2025) where returns with magnitudes of half a percent are removed from the training dataset.

As for textual data, multimodal architectures are necessary in order to take its effects in conjunction with price/technical data into a unified representation for downstream prediction tasks. Similar to the large variety of text features discussed earlier, the choices for model architectures are largely vast, ranging from simple to complex. Perhaps the simplest way to “fuse” textual data and price data is by concatenation of price features and derived sentiment, influence, impact, or other NLP features from textual data, which is commonly done for machine learning algorithms, as shown in Saravanos and Kanavos (2024), which also implements the same concatenation for deep learning models like LSTMs and CNNs. Much of the studies implementing LSTM models for the technical data features mentioned earlier actually concatenated the obtained sentiment scores from models like FinBERT with the price features (W. Gu et al., 2024; Halder, 2022; Karadaş et al., 2025), showcasing how simple concatenation of preprocessed features already provides research-level viability. More complex fusion implementations also showcase concatenation; when the representation of the textual data is that of full semantic embeddings, concatenation is also usually done in two different ways: *early fusion*, where different data modalities’ representations are fused before much of the steps before the forward pass, or *late fusion*, where different modules process the modalities, the transformed output representations of which are concatenated in the final layers before the prediction output. Studies that showcase early fusion include Albada and Sonola (2025)—where feed-forward neural networks receive price features concatenated with the full 768-dimensional embedding vector from a BERT model for social media data—and W. Sun et al. (2026)—which includes both news and social media data, where the high-dimensional news embedding vector from BERT is concatenated with price data, and social media content, influence, and impact features. Studies that perform late fusion on one hand include

P. Chen et al. (2024), where financial features are processed through an LSTM, and only the resulting final hidden state is concatenated with the FinBERT-derived semantic embedding of a representative news article headline (headline with the highest positive/negative sentiment score) before projection into a final output layer, resulting in performance scores that statistically outperformed multiple baselines; for the ones that it did not outperform, the differences were not statistically significant. Interestingly, the model also computes the derived FinBERT embedding by feeding all the word token embeddings into a Bidirectional LSTM layer, showcasing flexibility in the treatment of textual embeddings, where other studies simply use the CLS token (J. Chen et al., 2026; Liao et al., 2026), or mean pooling over word embeddings (Bustarviejo & Bousoño-Calzón, 2026), depending on conventions of the chosen model and which method garners the best performance scores, albeit this is normally a nonrestrictive choice. Implementations also slightly differ for implementing the FinBERT/BERT textual encoders, where some studies (Q. Chen, 2021) fine-tune the model further with the present textual data, while some studies (Bustarviejo & Bousoño-Calzón, 2026; J. Chen et al., 2026) keep such encoders frozen, showing that fine-tuning can also cause catastrophic forgetting, where the model downgrades from pre-training performance as a result of noisy loss landscapes from small fine-tuning datasets or different pre-training and fine-tuning domains. As for temporal aggregation of textual data, different studies implement different aggregation methods for summarizing textual data published within regular time periods in order to ensure data alignment, with some studies performing mean pooling (W. Sun et al., 2026), and others utilizing something akin to max pooling (P. Chen et al., 2024); however, other studies experiment with more complex pooling methods, namely a process called *attention pooling*, which designates learned weights for each item, similar to the attention mechanism. Such studies include (Liao et al., 2026), which tested different attentive pooling methods to summarize hundreds of news articles within a single time window, and those that performed the same later in the forward pass such as Arora et al. (2019) and W. Sun et al.

(2026), namely on LSTM hidden sequences. Other studies, however, prefer approaches that enable direct unaggregated text data to interact with other data modalities through attention mechanisms, exploiting the capacity for such mechanisms to process sequential data for two modalities at once. Examples include He and Gu (2021) for cross-attention between price and social media data, Fataliyev and Liu (2023) for combining cross-attention between news and price data with self-attention for each modality, where price sequences were first transformed by an LSTM, and Albada and Sonola (2025), Bustarviejo and Bousoño-Calzón (2026), P. Chen et al. (2024), Dashtaki et al. (2025), and Liao et al. (2026) for exhaustive pairwise attention fusion between different data modalities including news, social media, and/or technical data (where an attention module exists for each unique permutation or combination of the query and key designation of the modalities). A number of studies (Jang, 2025; Ridhawi et al., 2026; T. W. Wang et al., 2026) agree that cross-modal attention is superior to simple concatenation of embeddings from different modalities, supporting the recent motivation behind attention mechanisms. T. W. Wang et al. (2026) also adds that in cross-modal fusion, sequences from the two interacting modalities can be of different time scales, providing flexibility for attention mechanisms in handling modalities of different time frequencies. A lot of studies also perform within-modality attention mechanisms to allow data within each modality to interact with itself through self-attention, or even allow different kinds of data within the same modality to interact. J. Chen et al. (2026) provides an example of using ProbSparse self-attention for each of local news and global news embeddings, before performing cross-attention between both sub-modalities under news data. Moreover, He and Gu (2021) experimented with a two-layer transformer architecture where in one of the layers, the impact features of a sequence of social media posts within a lookback window were treated as keys and queries for self-attention, while the value vectors were the corresponding social media content embeddings, enabling the model itself to learn relationships between impact and content features instead of the author imposing formulaic assumptions. Other studies

attempt the same goal but with different architectures, such as P. Zhang et al. (2025), who experimented with using graph neural networks to model user influence and post impact that can in turn moderate sentiment analysis features on-the-go within a forward pass.

Self-interaction within a single modality is also particularly useful for tasks like filtering news and social media articles for each prediction instance. Jang (2025) particularly solves this problem using a learnable filter—a filtering module added before the multimodal cross-attention heads that dynamically selects top K items from a sequence of news data for a certain prediction instance that works by learning a fully-connected layer that takes each news embedding concatenated with overall sequence representative embedding to its “relevance score”, which is used to select the top k news articles. Jang (2025) found that $K = 5$ yielded the most optimal results. Since the top- K selection method is non-differentiable, this would normally be unable to become part of the parent learnable architecture. Instead of a hard top- K selection, it uses a differentiable, soft top- K selection mechanism based on perturbed optimizers (Berthet et al., 2020): the scores are perturbed with Gaussian noise hundreds of times (Monte Carlo samples), and one-hot vectors over the news sequence indicating the selected items for the K positions is computed for each of the perturbed samples. Taking the average of these one-hot vectors across all samples results in a vector of probabilistic selection scores instead of hard, discrete scores. Since each Monte Carlo sample exposes a noise value and a binary selection outcome, the averaged product of these two values across all Monte Carlo samples yields an empirical estimate of the gradient of the selection probability, since it captures the notion of measuring how a directed perturbation affects selection probability. This renders the entire selection processes differentiable end-to-end, without attempting to backpropagate through an essentially non-differentiable function. Differentiable top- K optimizers are not limited to this specific implementation; in an unrelated domain, it has been used in computer vision for patch selection, in a study (Cordonnier et al., 2021) that implements the optimizer during training such that a parameter known as σ , which controls the noise

level of the Gaussian perturbations, is annealed across training time from a starting point so that the model learns to adapt to increasingly stricter selection probabilities, which is ideal for inference.

A worthy implementation detail to note is whether only attention heads are used, or attention modules with subsequent feed-forward neural networks are utilized (such as in a full transformer layer), and this is also where implementation differs between studies. Some use the full architecture (Dashtaki et al., 2025; Fataliyev & Liu, 2023; Mandal et al., 2023), while others use only the representations from attention heads as inputs into further modules (Albada & Sonola, 2025; Jang, 2025). Other studies not only use full transformer layers, but also the full encoder-decoder stack (Albada & Sonola, 2025; Xu et al., 2025), highlighting the viability of even more complex transformer architectures for stock prediction. In terms of the number of heads to use for multihead attention, a number of 8 attention heads is commonly found in cross-modal multihead attention implementations (J. Chen et al., 2026; Jang, 2025), although this is not restrictive.

When using attention heads for the price data modules or cross-modality fusion modules, it is often imperative to carefully design the input embedding layers before the first attention block. This involves transforming raw input features into an embedding space and adding/concatenating positional/temporal embeddings. For textual data, much of the embedding work has been done by the textual encoding modules; however, for social media data, influence and impact features are usually also transformed into vector embeddings before fusion as described above. In He and Gu (2021), the raw impact and influence features were first converted into a learnable embedding using a linear layer before later processing, which is a standard way of transforming raw features into embedding spaces. Other papers support simply concatenating these linear-projected social impact vectors to content embeddings, such as in L. Wang et al. (2022) and in Daniluk et al. (2021). The treatment of positional embeddings also differs for textual data in some implementations; in such case, positional embeddings that encode the publication

timestamps (He & Gu, 2021) are preferred, since news articles and social media posts are published at irregular timestamps, and normal positional embeddings only encode the order of input vectors. This is also not restrictive, with some papers using only normal positional embeddings (Dashtaki et al., 2025), but it is more theoretically sound given the described argument. Positional embeddings are often added to the embedding vectors, but some studies support concatenation instead: in X. Zhang and Gao (2024) timestamp information is processed through a linear layer and concatenated with the post content embedding, allowing the model to learn the time embeddings itself instead of pre-calculating temporal information. For price features on one hand, embedding methods are diverse; at the simplest, linear projection can suffice (Omranpour et al., 2024). However, some studies prefer encoding price features with multi-layer perceptrons (J. Chen et al., 2026; Jang, 2025), whereas some utilize CNNs (Mandal et al., 2023). Other studies use LSTM hidden sequences as the embedding inputs into attention modules (Fataliyev & Liu, 2023; L. Sun et al., 2021; W. Sun et al., 2026), and even transformer outputs (Bustarviejo & Bousoño-Calzón, 2026; Dashtaki et al., 2025). Positional embeddings for high frequency data such as per-minute stock prices, on one hand, also require special implementations due to irregularities caused by inactive trading minutes and lunch/overnight gaps. Some studies implement adaptive temporal embedding methods deliberately designed to handle irregularities (Vogel & Kraus, 2025), where temporal embeddings are rather learned than calculated, similar to the case of embeddings for textual data. S. Lee et al. (2026) proposed ATENet, also a method for learnable temporal positional encodings that designed for time irregularities; however, it is in itself a wholly complex architecture. In the case of simple modular applications, preferred methods include the Time2Vec method, which have been used for transformer-based stock price prediction (J. Y. Lee & Yoo, 2025). Time2Vec is designed both for regular and irregular timestamp data, which is a method that takes a single time feature (such as elapsed time), and composes a vector with a single linear component and multiple sinusoidal components

using learnable parameters—namely a coefficient for the linear component, frequency parameters for the sinusoids, and phase shift parameters for all components. It is given by

$$\tau_i(t) = \begin{cases} \omega_i t + b_i & \text{if } i = 0 \\ \sin(\omega_i t + b_i) & \text{if } 1 \leq i \leq k \end{cases} \quad (76)$$

where t is the raw time variable, ω_i and b_i are learnable frequency and phase shift parameters for every dimension i , and k is the number of output dimensions (Kazemi et al., 2019). The final vector is given by

$$\Phi(t) = [\tau_0(t), \dots, \tau_{k-1}(t)]. \quad (77)$$

Unlike regular positional embeddings, Time2Vec is meant to be concatenated with the embedding vectors. It is also meant to be a stand-in for positional embedding, although this is not restrictive (Ughi et al., 2023).

A key implementation detail is the design of the output prediction layer. With attention modules or transformer architectures, only the last timestep’s vector representation was used for the prediction layer (Bustarviejo & Bousoño-Calzón, 2026; Jang, 2025), motivated by how recent timesteps influence the prediction more than timesteps longer ago. Some studies however (Dashtaki et al., 2025) use mean pooling over the transformer’s output vectors for use in the prediction layer in order to take into account all timesteps’ contribution in the prediction. The choice is nonrestrictive as well, and can be made based on experimental testing of its effects on model performance. All studies however agree on *concatenating* the fusion modules’ outputs (J. Chen et al., 2026; Dashtaki et al., 2025; Fataliyev & Liu, 2023), which would all be processed through a dense layer into output neurons, which would normally exhibit one neuron for every prediction class in the case of multiclass classification Q. Chen (2021). One might think that a single neuron only would be needed for binary classification, but J. Chen et al. (2026) designed two output neurons even with a binary classification task, showcasing harmless extension of the rule to binary classification.

One important kind of interaction that also must be taken into consideration, particularly for multi-stock prediction tasks, is inter-stock interactions. Often the challenge is that a single stock already contains a large number of features (owing to extensive technical analysis), and including even just one more stock in the data matrix can increase the dimensionality two-fold. Certain studies have devised a way for stocks to interact with each other without encountering this problem, in certain innovative ways. Q. Li et al. (2020), for instance, modeled stock interactions and co-movements driven by news data by converting news and technical data tensors for every stock-timestamp pair into a shared tensor space, and using stock correlations based on common news mentions, those representations are adjusted, before passage into an LSTM model. This highlights a way to incorporate stock correlations by “perturbing” the features of a single stock based on others before model intake. This is evidence that an “early fusion” method works. Other implementations however use methods that are more involved in deep learning, such as that of Mandal et al. (2023), which utilized a multigraph convolution network to model different kinds of relationships between stocks. Essentially, the model uses the multiple graphs to integrate other stocks’ representation vectors into the representation of the target stock, where the representation vectors are the stocks’ multimodal cross-attention fusion outputs. This essentially works like a “late fusion” method for inter-stock interactions. R. Kim et al. (2019) also featured a graph-based approach, where a hierarchical graph attention network using a multitude of different company relation types was used to aggregate stock representations. However, a disadvantage of these graph-based implementations however is that the graphs are often user-defined, where independently learned relationships instead would be preferable. The attention mechanism offers a way to make this process entirely self-contained within the model, as shown in Xu et al. (2025), where stocks are simultaneously processed in the model, and after forming informationally rich vector representations of every stock, a simple self-attention module allows every stock’s representation to attend to others within each timestamp.

Since stock forecasting potentially involves predicting returns for multiple stocks, it is often very common to come across model architectures in the literature that train and test a single model across multiple stock datasets, even as separate time-series. Models in the literature like that of J. Chen et al. (2026), Yao et al. (2018), Fataliyev and Liu (2023), Bustarviejo and Bousoño-Calzón (2026), Mandal et al. (2023), Xu et al. (2025), and W. Gu et al. (2024) may either be single-output, namely that the prediction target is just a single stock’s return, where it is possible to train a single model per stock or train a single one for all, or multi-output, wherein multiple stock returns constitute the prediction target—where it is inherent that a single model is trained on all stocks. In such multiple-stock, single-model cases, the model is known as a *global deep learning model* or a *multi-task learning model*, where different prediction tasks share the same parameters. This method uniquely has advantages of its own—Zanotti (2025) highlights that global models are stable, at least in retail industry modelling, in the sense that continuous retraining is not necessary and less-frequent periodic retraining is sufficient to maintain a stable accuracy, as opposed to local models. In terms of stock forecasting, Guennioui et al. (2024) showcased that a global model outperformed individual LSTM models trained locally on single stocks. This is perhaps due to the fact that despite different stocks being different time series, the prediction task is similar between stocks, which, according to Y. Zhang and Yang (2017), is conducive for achieving good global model performance, as opposed to the case where the prediction tasks were unsimilar. Some studies challenge this, showing that such multi-task learning models can still perform with unlike prediction tasks: Jiang (2025) showed that a global model predicting stock closing prices, stock transaction volumes, and volatility targets simultaneously retains viable performance. Overall, there is great motivation to design models trained on multiple stocks rather than training multiple single-stock models, perhaps in most cases.

Evaluation

The methods for evaluating stock prediction algorithms are varied. For machine and deep learning applications, the usual metrics—such as MAPE for regression tasks (Mohan et al., 2019), and accuracy/F1/MCC scores for classification tasks (Khan et al., 2023)—are applied. However, multiple studies (P. Chen et al., 2024; Q. Chen, 2021; Z. T. Ke et al., 2019) note that such metrics may fail to fully capture some key performance aspects of a prediction method, especially for classification, where prediction values collapse stock returns of any magnitude into a single category. Instead, another method for capturing nuanced performance measurements is trading simulations, where a trading strategy based on model predictions is tested for profit generation. The length of the test period can be short; some high-frequency trading studies (Son et al., 2012; Zaznov et al., 2024) adopt test windows of only one week in length, whereas low-frequency trading studies (P. Chen et al., 2024) feature testing windows that can extend for a month, or across years, and no matter the time scales, trading simulations have been used to capture some aspect of model performance. Most obviously, trading simulations can measure the actual profitability of the model—although this is confounded by the particular trading strategy. Q. Chen (2021), for example, used two different trading strategies, which have shown different results, showing that trading strategies are not implementation-agnostic and thus cannot be used to measure *absolute* model profitability. However, *relative* profitability can be measured by running the same trading simulation settings on different models, wherein, since the confounding trading strategy has been controlled for, the resulting profit scores can be reasonable comparisons of profitability between different models. Q. Chen (2021) also demonstrates this, where both trading strategies resulted in more or less the same rankings of the models being compared. Generally, in using trading strategies that measure profitability, transaction costs are often simulated as well, such as in S. Li et al. (2025), which designed their trading simulation to buy only if the price return exceeds a certain threshold in order to mitigate the effect of transaction costs. Yao et al. (2018), on one

hand, bakes the transaction cost information during training itself, by classifying return signals as buy/sell only if the profit is more than the transaction fee, so that the trading simulation itself is implemented without trading fees. This is motivated by the high frequency nature of their prediction task (dataset timestamps are only 3 seconds apart). Such studies that measure profitability also use a certain metric known as the Sharpe ratio (Q. Chen, 2021), which can be thought of as the cumulative profit of a trading strategy adjusted for risk. To intuitively understand the metric, one can imagine two trading strategies that both generate the same cumulative return, but one takes a noisier path than the other—the Sharpe ratio penalizes the noisy strategy since it is *riskier*, such that higher scores would be attributed to more stable trading strategies. Q. Chen (2021) defines it as follows for an example trading frequency of 1 day: the annualized cumulative return of the trading strategy is defined as

$$\frac{1}{N} \sum_{t=1}^N r_t \times D \quad (78)$$

where N is the number of trading days in the sample, D is the number of trading days in a single year, and r_t is the ratio between the day t return and day t total position. With that, the annualized Sharpe ratio is given by

$$\frac{\bar{r}}{\sigma(r)} \times \sqrt{D} \quad (79)$$

where $\bar{r}$ and $\sigma(r)$ denote respectively the mean and standard deviation of $r_1, \dots, r_N$.

Essentially, the Sharpe ratio provides information on what trading strategy is *most preferable* for investors who want the most gains but the lowest risk. In terms of the specific strategy used, the literature also exposes a certain kind found most commonly especially for studies involving multiple stocks, namely the *long-short investment strategy* (Mandal et al., 2023), where for every prediction instance, all stocks are ranked based on model prediction probability or some other reliable ranking metric (Q. Chen, 2021). The strategy then performs investment by going long on the top K stocks and going short on the bottom K stocks, for a certain number of K . This strategy is also found in Ghosh et al. (2022).

Another interesting aspect of stock prediction evaluation is model drift—how much the accuracy of a model decays over time elapsed since the cut-off for training data. Some studies on stock prediction have taken this effect into account by exploring incremental/offline-online learning techniques (Singh et al., 2022), explicitly stating how machine learning models require tuning in order to stay up-to-date with market data. The occurrence of different volatility regimes (Catão & Timmermann, 2010) can also significantly change how markets behave, thereby contributing to concept drift. It is therefore interesting to explore whether the inclusion of certain data modalities (news and/or social media) on top of financial data can impact model drift, which can be done with the help of a certain tool, known as Adaptive Windowing (ADWIN), introduced by Bifet and Gavalà (2007). ADWIN itself does not generate model drift scores—it is a process that iteratively takes a window of an input series (in this case, prediction errors), and adds a new timestep—one by one—until it encounters a way to split the window between which the difference in means is statistically significant (indicating an occurrence of drift), after which the earlier split is dropped (in order to preserve the latest information), and the process repeats with the new, shorter window until the same occurs, and so on, flagging the timesteps where drift was detected. Each timestep can be given a calculation of the current window’s average value, which may jump sporadically given that the window changes shape abruptly upon detection of drift. With this rolling metric, high-resolution model drift scores can be obtained with a method used by Garcia-Rubio et al. (2026) that aggregates deviations between prediction errors and this ADWIN-maintained average. One might question how this method is able to measure drift given that the window average essentially follows the data, but the crux lies in the delayed detection of drift; ADWIN is conservative and requires strong evidence for drift detection, enabling brief moments where drift occurs without triggering the detection yet. This is the signal that contributes to the drift score, and by design, it quantifies change—even if the prediction errors drift back to original levels, it would still register as a positive contribution to the overall score.

Summary of Gaps in the Literature

Despite the extensive discussion vastly covering the topic of stock prediction in the literature, it can still be said that certain points of interest remain underexplored by studies in general. Perhaps the major pain point is that most of the studies seek to pursue state-of-the-art performance—the “best model” or the “most profitable” one—and not an in-depth analysis of which factors actually contribute to performance. While a considerable number of the mentioned studies do perform ablation analyses in order to determine the effects of removing certain model components, they do not consider *all* model configurations with a certain component removed, and all configurations with it retained. As such, single-component ablation does not give a complete picture; it misses out on interaction effects, and takes for granted the rest of the combinatorial space of model configurations. This motivates the current paper’s objective to study main and interaction effects using a factorial analysis set-up.

The studies also hardly attempt to measure *model drift*, which is particularly relevant for stock price prediction due to constantly shifting market regimes. While certain studies address drift in stock data by retraining models (Pulicharla, 2019), direct attempts to quantify drift are hard to find. For studies that consider drift, the well-established fact itself that market behaviors shift over time is studied more than the susceptibility of models to such intrinsic drift mechanisms. The question of which kinds of models are more or less robust over volatility regime shifts is understudied. This motivates the objective to study model drift as a function of model configurations, since studies hardly quantify whether it interacts with such model modification variables.

The treatment of trading simulations, while rightfully purposed for quantifying the profitability of certain trading strategies, has rarely been explored for its potential to quantify certain other core performance metrics of a prediction model. For instance, the long-short investment strategy described in Mandal et al. (2023)—where for every timestamp, the decision to long or short a stock is based on the ranking of stocks according

to the predicted probabilities—provides a relative measurement of how correctly a model *ranks* stocks in terms of profitability. While this is not necessarily a “gap” in the literature, since trading simulations reasonably should measure profitability, this motivates the objective to use the long-short investment strategy for a goal aside from quantifying model profitability: namely the capacity of a model to rank the profitability of going long or short on stocks.

All in all, even with an extensive list of references from the most recent years of literature, there are still a number of quite interesting directions of study that are rarely, if not never exposed in the literature, that the current study mostly aims to achieve. This establishes the main rationalization behind the current study’s research objectives, using methods that are also guided by the literature in order to maintain relevance with current practices in the field of deep-learning-based stock return prediction. These methods have been carefully crafted, and are summarized in the following chapter.

Chapter 3: Methodology

Research Design

The research employs an experiment on model design factors. This entails the detailed construction of each experimental setting from the structure of data inputs, the individual steps of a forward pass for each model architecture, to the preprocessing, training, and evaluation pipelines that generate the experimental outputs for statistical analysis. This design is pursued since the objective of all the research questions is to generalize findings and since the settings for experimentation are fully controlled. This applies for investigating both descriptive (feature importances), relational (understanding the effect of certain variables on model performance and model drift, and comparing deep learning models vs. heuristic methods), and diagnostic (mechanisms affecting feature importance and attention distributions) types of analyses.

In summary, the main conceptual framework of independent and dependent variables under analysis is defined as follows.

1. Independent variables: variables whose main and interaction effects on the dependent variables are studied; namely:
 - *Overall pipeline complexity*: a binary variable determining whether the overall deep learning architecture is transformer-based ($= 1$) or a multi-layer perceptron ($= 0$), whether the text representation method is full semantic embeddings ($= 1$) or sentiment features ($= 0$), whether the multimodal fusion method is cross-attention with dynamically filtered text documents ($= 1$) or simple concatenation of aggregated features of unfiltered text documents ($= 0$) and whether the model supports inter-stock correlations to inform the prediction by design ($= 1$) or otherwise ($= 0$). The designs of the transformers and multi-layer perceptrons are described in later sections. These pipeline choices are made in order to provide a comparison between a solution with a relatively high representational capacity (due to the presence of sequential processing

capabilities, attention mechanisms, intended design for cross-modal fusion, etc.) and a lower-capacity, minimally simple design, from which effects on each of the dependent variables (exposed in the following) are potentially due to a difference in pipeline complexities.

- *Inclusion of news data:* a binary variable determining whether or not news data is included ($= 1$) for training, validation, and testing runs of the model, and the model correspondingly therefore incorporates architectural adjustments to accommodate the modality. In the case that the deep learning architecture is transformer-based, the architecture includes a news encoder and cross-modal fusion transformer to be described in later sections, where the news encoder handles conversion of news headlines into embeddings, and the cross-modal fusion transformer both incorporates publication timestamp information and handles fusion between news data and price data. In cases that the deep learning architecture is a multi-layer perceptron, the architecture simply includes extra input neurons for sentiment indicators derived from a financially-tuned sentiment classifier's output activations given the news headlines translated into English using a machine translation model.
- *Inclusion of social media data:* a binary variable indicating whether or not social media data is included ($= 1$) for training, validation, and testing runs of the model, and the model correspondingly incorporates additional architecture to accomodate the modality. If the deep learning architecture is transformer-based, the architecture, similar to the case of including news data, includes a social media data encoder that both embeds the content of each post, and a cross-modal transformer that incorporates publication timestamp information, combines content embeddings with learned representations of the corresponding posts' impact and influence features, and handles fusion between social media data and price data. In cases that the deep learning architecture is a multi-layer

perceptron, the architecture simply includes extra input neurons for sentiment, influence, and impact indicators derived from a multilingual sentiment classifier’s output activations and raw post metadata.

- *Prediction horizon*: a binary variable indicating whether the chosen classification target for the model is the 30-minute return direction ($= 1$) or the 10-minute return direction ($= 0$).

2. **Dependent variables**: the variables on which the effects of the independent variables are studied. All dependent variables are measured using the same trained models—only one set of trained models is created for all dependent variable experiments. The dependent variables are summarized as follows:

- *Model performance as a traditional machine learning evaluation metric*: a continuous variable denoting the Matthew’s correlation coefficient (MCC) performance of the model on a single stock. An MCC score for a single stock counts as one observation. The MCC is given by:

$$\text{MCC}(\tau, s) = \frac{\text{TP} \cdot \text{TN} - \text{FP} \cdot \text{FN}}{\sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}} \quad (80)$$

where TP (true positives), TN (true negatives), FP (false positives), and FN (false negatives), are elements of the model’s confusion matrix on a single stock on the testing period.

- *Model performance as ranking ability*: a continuous variable denoting the cumulative return of the model on a simulated long-short investment strategy on the testing period that at every interval, ranks stocks according to predicted probabilities, goes long on the top K stocks, and goes short on the bottom K stocks, given a value for K and an offset along the trading interval indicating when the simulation begins (counting as one observation). While trading simulations are normally used to estimate the trading profitability of a model, this implementation aims not to measure the model’s profitability, but rather measures how well a model *ranks* stocks according to trading profitability.

- *Model drift*: a continuous variable denoting a combination (product) of two normalized scores (ranging from 0 to 1) for every stock, namely a normalized score denoting the mean absolute differences between the model’s instantaneous loss values and the expanding-window averages of the model’s loss over the current adaptive windows defined by an ADWIN detector for that stock (signifying loss function instability within local regimes, hence drift), and a normalized score denoting the complement of the adaptive windows’ average length for the same stock, so that small lengths yield higher scores (signifying the overall instability of the loss function across regimes, hence drift).

The independent variables are understood to interact with each other (namely, that certain independent variables “moderate” relationships between other independent variables and a dependent variable); thus, both *main* and *interaction* effects are considered for the study.

As mentioned before, the study entails not only descriptive and relational analyses involving the variables, but also diagnostic analyses that shed light into why such relationships arise. Given these requirements, the research design is mainly a factorial experiment to analyze the model performance and model drift metrics in terms of the four given independent variables (overall pipeline complexity, inclusion of news data, inclusion of social media data, and prediction horizon), which also takes into account two-way interactions. Each of the dependent variables correspond to a separate analysis of main and interaction effects; however, these analyses are made on top of the same set of trained models, as mentioned before. This means that the results from the analyses are not independent results, albeit the results hold. Another phase of this section of the research design is also implemented, featuring descriptive and post-hoc analyses for every dependent variable to understand which experimental settings provide the best performing models on each metric. Finally, a final phase involving a comparison between the best performing deep learning model and baseline machine learning methods (logistic regression, support vector machines, random forests, and an extreme gradient boosting) trained on the same

model architecture and data configuration settings as the deep learning model is performed for each of the dependent variables on corresponding similar tests. Research questions [1](#), [2](#), and [3](#) are all addressed by the described factorial analyses.

Constructing the models involves rigorous pipelines, which are specific for each experimental setting. To summarize, each experimental setting corresponds to a unique configuration of the variables model architecture (MLP/Transformer) inclusion of news data (true/false), inclusion of social media data (true/false), and prediction horizon (10-minute and 30-minute lookahead predictions). The choice of prediction horizons is guided by the discussion on the taxonomy of market movement drivers depending on the prediction frequency—a mix of market inefficiencies and evolving market interpretations of reality drives 10-minute stock movements, while mostly longer-term effects dominate 30-minute stock movements. Below is a summary of the pipeline for each model considering the various data modality combinations present in the prediction framework.

1. For experimental settings where the model architecture is transformer-based, a time-series transformer layer accepts as input all the economic data features. This serves as the sequential time-series component of the model to predict stock prices and indices. The time-series transformer has a lookback window of 60 trading minutes—in order to leverage intermediate price momentum effects as well as still be able to capture market microstructure-driven movements. The time-series transformer is equipped with a Time2Vec module for learning temporal positional embeddings. For experimental settings where the model architecture is an MLP, there is no time-series processing component other than summarized rolling technical indicators—input neurons accept the most recent timestamp’s feature vector. Preprocessing includes the creation of the aforementioned economic features; however, before doing so, missing data are dealt with through a forward-fill in the case of closing price features, bid and ask prices, and bid and ask yields, zero-fill for volumes, stock returns, refresh rates, and net bid change (absolute and percentage) features,

and a fill with the latest available closing price/bid price for opening, high, and low price features. These imputation methods are used since missing trading minutes in LSEG simply indicate zero liquidity, hence zero volumes, rates of change, and also therefore constant price data. The transformed features are standardized on the training split. Care is taken to prevent test data leakage into standardization scaling.

2. In all experimental settings including news data, an encoder is used based on the architecture. For transformers, *Cohere’s embed v4.0*, an LLM embedding model, was used to transform news headlines into 1024-dim Matryoshka embeddings (only the first 256 dimensions are used by truncating the originally requested 1024-dim Matryoshka embeddings in order to reduce dimensionality while prioritizing the most informative dimensions without increasing model parameter count). For MLPs, the *facebook/nllb-200-distilled-600M* neural translation model was used to translate headlines into English first, and *tabularisai/ModernFinBERT* was used to obtain sentiment scores. The NLLB-200 model is a machine translation model with a mean chrF++ performance of 52.33 (chrF++ is a text translation quality metric ranging from 0-100 that measures the F1 score over matching character and word n -grams between a reference translation and the candidate translation) across all languages for translation to English (AI at Meta, 2022), whereas ModernFinBERT is an improvement over the FinBERT model, achieving a sentiment analysis accuracy score of 80% on the FiQA (Financial Opinion Mining and Question Answering) dataset, with an F1 score of 61%, precision of 64%, and recall of 88% (TabularisAI, 2025). Table 2 showcases the chrF++ translation quality scores of the 19 most widely spoken languages (plus Tagalog) to English using the Facebook NLLB model. Scores are taken from the model card of *facebook/nllb-200-distilled-600M* (AI at Meta, 2022).

Table 2*chrF++ Translation Quality Scores (to English)*

Language	chrF++
<i>Top 20 Most Spoken Languages</i>	
Mandarin Chinese	52.9
Hindi	62.5
Spanish	57.1
French	65.4
Modern Standard Arabic	62.2
Bengali	57.9
Russian	58.6
Portuguese	69.0
Urdu	57.8
Indonesian	63.9
German	65.3
Japanese	50.2
Nigerian Hausa	52.6
Marathi	58.9
Telugu	61.2
Turkish	60.0
Tamil	55.8
Yue Chinese (Cantonese)	51.3
Vietnamese	58.1
<i>Additional Language</i>	
Tagalog (Filipino)	63.7

These are mutually exclusive—transformer models do not use FinBERT sentiment scores, and MLP models do not use Cohere embeddings. Aligning the news data with the time-series financial tickers for synchronicity despite the irregular nature of news articles is straightforward: for every financial ticker timestamp, all news articles published within the most recent 270 trading minutes or most recent 24 hours (whichever is longer) serves as the lookback window, inclusive of the prediction timestamp, irrespective of the prediction horizon setting. This is to ensure that the model sees news in a window that captures a consistent amount of market activity, even in the presence of overnight and weekend gaps. For instance, if the past 270 trading minutes crosses into the previous trading day, news articles from then, including those that were published during the overnight gap, are included in the lookback window, since these are relevant to the current market activity that only more recently had the chance to react to this information. In case the gap between the current and previous trading days is a weekend or a holiday break, no matter how long, the treatment is the same—all news articles published in that period are captured in the lookback window. With that, the input data is a sequence of news articles, complete with news headlines and timestamp of publication. For MLP models, the derived sentiment indicators (described above) are only added as extra input neurons (early concatenation); however, for Transformer models, a news-fusion transformer is constructed, which performs cross attention between the time-series transformer’s outputs and the news article sequences. The news-fusion transformer is also equipped with a learnable top-K differentiable filter that learns the top 5 most relevant news articles for every stock at every timestamp, before passing onto the cross attention module. The choice of 270 minutes is motivated by the suspicion that the market absorbs news information much more slowly, as this has been the case for other countries’ stock exchanges. Timezones are also carefully handled in order to ensure alignment across all modalities by converting all timestamps into UTC+08:00

(Asia/Manila time) format.

3. For all experimental settings including social media data, a similar methodology is also used as in news data, where for transformer models, Cohere embeddings are extracted for the social media content, but for MLP models, a multilingual BERT model (*tabularisai/multilingual-sentiment-analysis*) is used to extract sentiment scores. The same mutual exclusion between transformer models and MLPs regarding the use of Cohere embeddings and sentiment scores applies. For transformer models, the embeddings are concatenated with a learned embedding of the raw impact and influence features through a fully-connected layer, whereas for MLP models, the impact and influence features are used to construct the set of derived sentiment, influence, and impact indicators used simply as early-concatenated input neurons. For transformer models, a social-media-fusion transformer is used similar to the news-fusion transformer, which is also equipped with a learnable top-K filter. The past 270 trading minutes is used as the window for extracting sequences of social media posts to align with the time-series economic data.

For all transformer models, an additional inter-stock correlation transformer is appended to the architecture before the prediction layer in order to learn inter-stock correlations. This is done by simply transposing the stock and timestamp dimensions of the time-series transformer outputs (or the news-fusion and/or social-media-fusion transformer outputs, concatenated if both present), before intake into another self-attention module with an FFNN layer. Only the outputs from the latest timestamp are used in the actual prediction layer, which is a simple fully-connected network with two output neurons per stock. The transformer models are designed to input all stocks at once and output return predictions of all stocks at once as well for a single forward pass, for a total of 60 output neurons (two for each stock), whereas the MLP models are designed only to input data for just one stock, and output return predictions for that single stock only for a single forward pass. Each model however is trained on all stocks, in line with global forecasting methods.

For transformer models, 8 attention heads and only one layer is used for every present transformer module (time-series transformer, news-fusion transformer, social-media-fusion transformer, and inter-stock transformer). For MLPs, 5 hidden layers with a dimensionality of 384 are used before the output prediction layer of similarly two output neurons.

The actual value that defines the loss function for training is the softmax probability outputs from the two output neurons, for the categorical outcomes (positive, nonpositive). The task is a classification problem instead of a regression problem. This is due to the fact that the market is noisy and illiquid; predicting the stock return percentage would end in predictions flatlining at zero, trivially, because of an abundance of timestamps with zero stock price movement. That said, the loss function utilized for training and optimization is binary cross-entropy. In experimental settings with only economic numerical data as the single data modality, the multimodal fusion layers are skipped for transformers, and input neurons reserved for news and social media for MLPs are simply retracted.

For each experimental setting, the entire architecture, including the transformer/MLP, news encoder, and social media encoder branches (if applicable) is regarded as a single model trained at once altogether. The Cohere, NLLB-Modern-FinBERT and Multilingual-BERT encoders for embeddings and sentiment scores however, are frozen for computational efficiency, and since the datasets for news and social media data may not be suited for fine-tuning. This training method implies that a single training observation encompasses a single sequence of consecutive timestamps of economic numerical data, and if the experimental setting includes news and/or social media data, the sequences of articles and/or social media posts within the respective aforementioned lookback windows are also part of the single data observation. Batching is done carefully to implement this data observation structure.

Given that each independent variable (model architecture, inclusion of news data, inclusion of social media data, and prediction horizon) is a binary variable, the experimental setup is completed with sixteen distinct models—one for each unique

combination of independent variable settings. The described rules are applied to construct every setting individually and uniquely.

Data Generation

With the sixteen distinct trained models, each are trained on a 80%-10%-10% train-validation-test split. For each experimental setting, training is done until loss values on the validation set begin to diverge from training loss, after which the model with the lowest validation loss is chosen. The next task thereafter is to generate each model's performance metrics. For each experimental setting, MCC scores are measured for each of the 30 stocks on the test set, using optimized thresholds that are optimized starting from the validation set, and are re-optimized after passage through every tenth of the test set, in chronological order. Model drift scores per stock are also measured using the ADWIN-maintained process described in the Research Design section. Furthermore, trading simulations are also done to obtain cumulative profit scores using the process described in the Research Design section as well.

Data Treatment

As mentioned in the previous section, the MCC scores, model drift scores, and trading simulations generate cumulative profit scores for every experimental setting. The goal of this is to statistically compare such performance ratios between experimental settings in the fashion of an experimental factorial design, namely through main and interaction analyses. An OLS regression model with cluster-robust standard errors (to take into account correlations due to measurements coming from the same model and same stocks) is used to determine statistical significance of the main and interaction effects of the independent variables. The three dependent variables are studied separately, each having their own study of main and interaction effects study with a unique OLS regression instance.

After ascertaining the best-performing model out of the sixteen settings for each of the three dependent variables (highest MCC, lowest drift, and highest cumulative trading profits), resulting in three potentially different deep learning models, each are compared to

trained machine learning baselines (Logistic Regression, Random Forest, XGBoost, and Support Vector Classifiers) on the same tests (MCC comparison, model drift comparison, and trading simulations), using OLS regression with cluster-robust standard errors to determine statistical significance as well.

Chapter 4: Data Understanding

The data requirements for this research are highly specific, and mostly require proprietary sources. Each data modality follows a format, described in the following subsections.

Financial Time-series Datasets

The financial time-series datasets are multivariable datasets that update at a minute-level or daily frequency, composed of the stock prices of each blue-chip company in the PSE (the top 30 most highly recognized and most actively-traded stocks), the six sectoral indices, the PSEi (composite index), 1-minute-frequency economic indicators such as the PHP to USD exchange rate, the PHP to gold exchange rate, continuous front-month Commodity Exchange (COMEX) High-grade Copper futures (with “continuous front-month” meaning the nearest active contract, rolling over to the next month whenever such contract reaches expiration), continuous front-month Intercontinental Exchange (ICE) Brent Crude Oil futures, and daily-frequency Philippine government bonds maturing in approximately 20 years, 10 years, 7 years, 5 years, and 2 years respectively, at the time of data collection.

Table 3 showcases all of the financial time-series datasets used in the study.

Table 3

All Financial Datasets Constituting All Technical Time-series Data

Abbrev.	Name
<i>Stocks: Financials</i>	
BDO	BDO Unibank, Inc.
BPI	Bank of the Philippine Islands
CBC	China Banking Corporation

Continued on next page

Table 3*All Financial Datasets Constituting All Technical Time-series Data (Continued)*

Abbrev.	Name
MBT	Metropolitan Bank and Trust Company
<i>Stocks: Holding Firms</i>	
AC	Ayala Corporation
AEV	Aboitiz Equity Ventures
DMC	DMCI Holdings, Inc.
GTCAP	GT Capital Holdings, Inc.
JGS	JG Summit Holdings, Inc.
LTG	LT Group, Inc.
SM	SM Investments Corporation
SMC	San Miguel Corporation
<i>Stocks: Industrial</i>	
ACEN	ACEN Corporation
CNPF	Century Pacific Food, Inc.
EMI	Emperador Inc.
MER	Manila Electric Company
JFC	Jollibee Foods Corporation
MONDE	Monde Nissin Corporation
SCC	Semirara Mining and Power Corporation
URC	Universal Robina Corporation
<i>Stocks: Property</i>	
ALI	Ayala Land, Inc.
RCR	RL Commercial REIT, Inc.

Continued on next page

Table 3*All Financial Datasets Constituting All Technical Time-series Data (Continued)*

Abbrev.	Name
AREIT	AREIT, Inc.
SMPH	SM Prime Holdings, Inc.
<i>Stocks: Services</i>	
CNVRG	Converge Information and Communications Technology Solutions, Inc.
GLO	Globe Telecom, Inc.
ICT	International Container Terminal Services, Inc.
PGOLD	Puregold Price Club, Inc.
PLUS	DigiPlus Interactive Corp.
TEL	PLDT, Inc.
<i>Stocks: Sectoral Indices</i>	
PSFI	Financials
PSHO	Holding Firms
PSIN	Industrial
PSMO	Mining & Oil
PSPR	Property
PSSE	Services
<i>Stocks: Composite Index</i>	
PSEi	Philippine Stock Exchange Index
<i>Exchange Rates</i>	
PHP	PHP to USD Exchange Rate

Continued on next page

Table 3*All Financial Datasets Constituting All Technical Time-series Data (Continued)*

Abbrev.	Name
XAU	PHP to Gold Exchange Rate
<i>Energy Futures</i>	
LCOc1	Continuous Front-month ICE Brent Crude Oil Future Contracts
<i>Government Bonds</i>	
PHGV20	Philippine Treasury Bond at 20 Years to Maturity
PHGV10	Philippine Treasury Bond at 10 Years to Maturity
PHGV7	Philippine Treasury Bond at 7 Years to Maturity
PHGV5	Philippine Treasury Bond at 5 Years to Maturity
PHGV2	Philippine Treasury Bond at 2 Years to Maturity
<i>Commodity</i>	
HGc1	Continuous Front-month COMEX High-grade Copper Future Contracts

These datasets is composed of the data columns found in Table 4, which mainly consist of full or partial OHLCV data, and/or bid and ask prices, net change variables, or refresh frequencies.

Price and volume charts of every involved financial time-series instrument are given in the following. Figure 1 shows the evolution of the PSEi closing prices and volumes over time.

The PSEi generally has no overall growth or decline over the entire year; however, a drop in the index value occurs during the latter half of 2025, rebounding up during the first two months of 2026, before dropping again starting at the boundary between February and

March 2026 due to Iran war shocks. This general pattern is shared across all market indices, starting with financials (Figure 2), where the overall pattern is shared as well among all constituent banking companies.

Table 4

Relevant Raw Variable Columns for All Time-series Datasets

Column	Description
<i>Stocks (All Stocks, Sectoral Indices, and the Composite Index)</i>	
LocalDate	Date of the current timestamp in local time (UTC+08)
LocalTime	Time of the current timestamp in local time (UTC+08)
Close	Closing price (last traded price) of the current minute
Net	Net change in closing price between the previous and the current minute
%Chg	Percent change in closing price between the previous and the current minute
Open	Opening price (first traded price) of the current minute
Low	Lowest traded price of the current minute
High	Highest traded price of the current minute
Volume	Number of stocks traded in the current minute
<i>Exchange Rates (USD and XAU Datasets)</i>	
LocalDate	Date of the current timestamp in local time (UTC+08)
LocalTime	Time of the current timestamp in local time (UTC+08)
Bid	Best bidding price (highest price at which any buyer is willing to spend on the asset, namely 1 US dollar for the USD dataset or 1 troy ounce of gold for the XAU dataset) of the current minute
Ask	Best asking price (lowest price at which any seller is willing to sell the asset) of the current minute

Continued on next page

Table 4*Relevant Raw Variable Columns for All Time-series Datasets (Continued)*

Column	Description
High	Highest traded price of the current minute
Low	Lowest traded price of the current minute
Open	Opening price of the current minute
RefreshRate	Number of quote refreshes during the minute
BidNet	Net change in bid price between the previous and the current minute
<i>Energy Futures (ICE Brent Crude Future Contracts)</i>	
LocalDate	Date of the current timestamp in local time (UTC+08)
LocalTime	Time of the current timestamp in local time (UTC+08)
Close	Closing price of the current minute
Net	Net change in closing price between the previous and the current minute
%Chg	Percent change in closing price between the previous and the current minute
Open	Opening price of the current minute
Low	Lowest traded price of the current minute
High	Highest traded price of the current minute
Volume	Number of stocks traded in the current minute
Bid	Best bidding price of the current minute
Ask	Best asking price of the current minute
<i>Copper Futures (COMEX Copper Future Contracts)</i>	
LocalDate	Date of the current timestamp in local time (UTC+08)
LocalTime	Time of the current timestamp in local time (UTC+08)
Bid	Best bidding price of the current minute

Continued on next page

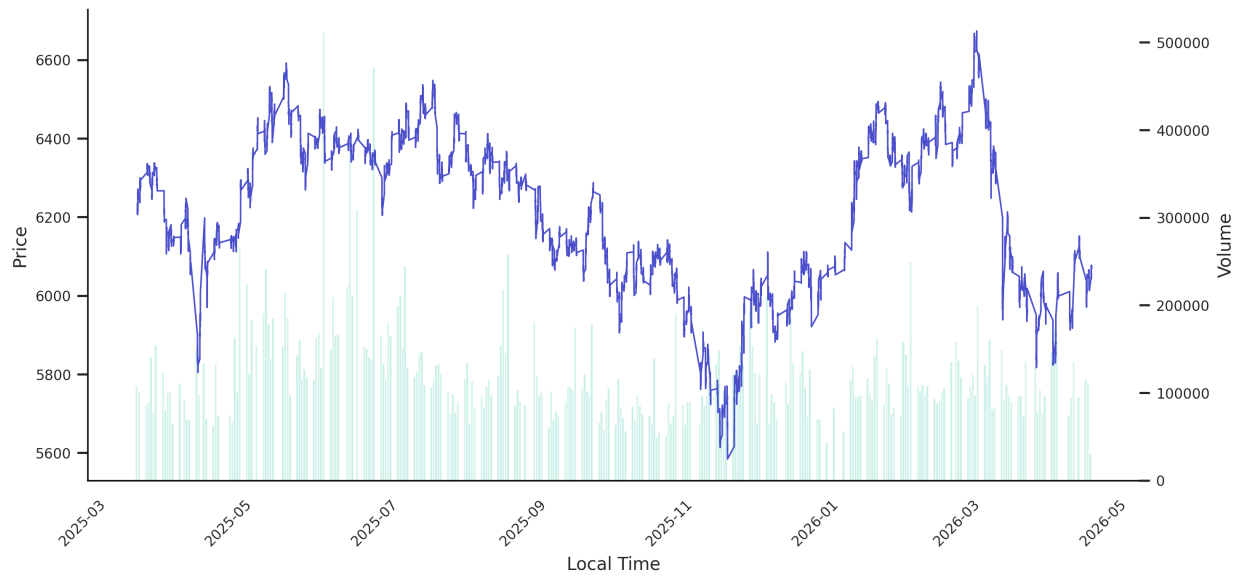
Table 4*Relevant Raw Variable Columns for All Time-series Datasets (Continued)*

Column	Description
Ask	Best asking price of the current minute
<i>Government Bonds (PHGV Datasets)</i>	
ExchangeDate	Date of the current day
Bid	Best bidding price of the current day
Ask	Best asking price of the current day
BidYld	Bond yield implied with the bond price assumed to be the best bidding price of the current day
AskYld	Bond yield implied with the bond price assumed to be the best asking price of the current day
BidYChg	Net change in bond yield implied by the best bidding price between the previous and the current day

Figure 3 shows a more varied sector (holding companies), where some stocks generally rise while others fall or remain stagnant, in deviation from the main downtrend and uptrend patterns of the PSEi.

The industry sector also has varied behaviors among constituent stocks, as shown in Figure 4. The sector index itself (PSIN) retains some of the downtrend and uptrend regimes present in the PSEi, albeit constituent stocks individually having unique behaviors.

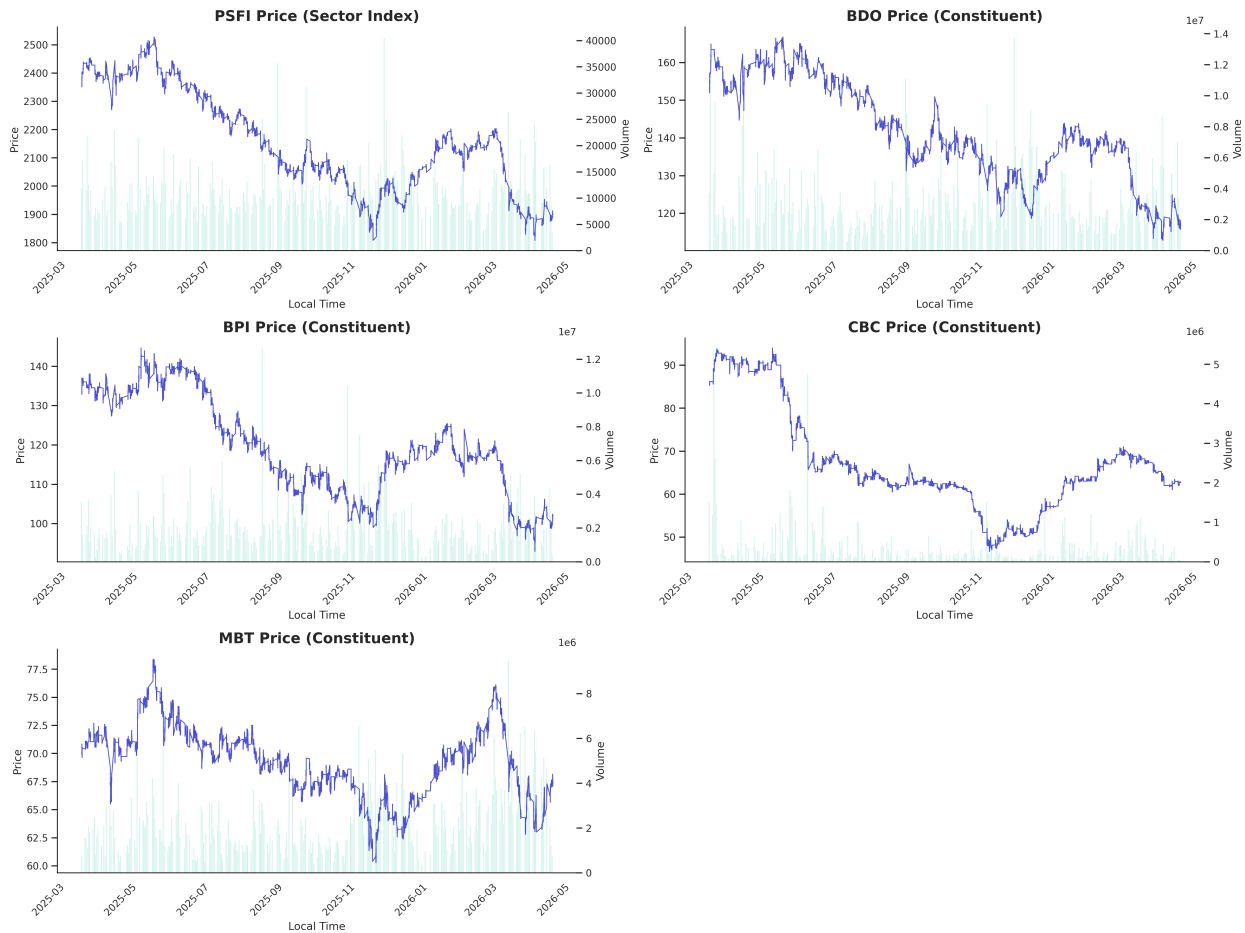
The mining industry (PSMO) index is shown in Figure 5, and is one of the sectoral indices that exhibit the least similarity with the trends found in the PSEi. This is probably due to the fact that PSMO companies are less actively traded—no PSMO-listed company is among the 30 blue chip stocks of the PSEi.

Figure 1*PSEi Price and Volume*

Property companies, as shown in Figure 6, are similar with financial sector companies in that they all share similar trending behaviors with the PSEi and the sector index PSPR.

The services industry most distinguishably deviates from the overall trending behaviors found in the PSEi, as seen in Figure 7; each company has unique growth/decline/stagnation behaviors that do not line up with the sectoral index.

These datasets are indexed by minute-level timestamps, or daily timestamps that can be recast into minute-level scales (shown in the preprocessing steps detailed later on). The only timestamps in the final data frame, which are thus the data observations, are the trading minutes—from 9:31 AM to 12:00 PM, and 1:01 PM to 3:00 PM, on Monday to Friday, with the exception of non-working days—which are the only minutes with data available from the stock price and index datasets. Other financial datasets (commodities, gold, etc.) actually contain data that exceeds just the trading minutes, but only data within the trading minutes are used. The chosen source for this data is the London Stock Exchange Group (LSEG) Workspace, a world-class provider of real-time and historical

Figure 2*PSFI and Constituent Stocks' Price and Volume*

financial data, licensed to the University of Asia and the Pacific (UA&P), and available for use in the UA&P Public Access Computing lab. While the source is greatly encompassing, availability restrictions with the current license allow only for one year of minute-level data. With data collection done in multiple iterations, data from March 12, 2025 to April 15, 2026 were successfully collected—as such, only data from this range are used for training, validation, and evaluation of models. Dataset sizes vary due to different missing data patterns—a uniform dataset size after preprocessing is reported in the next chapter.

Other financial datasets generally behave uniquely, where the only main similarity with the PSEi, for some of the financial datasets, is the presence of market shocks during

Figure 3

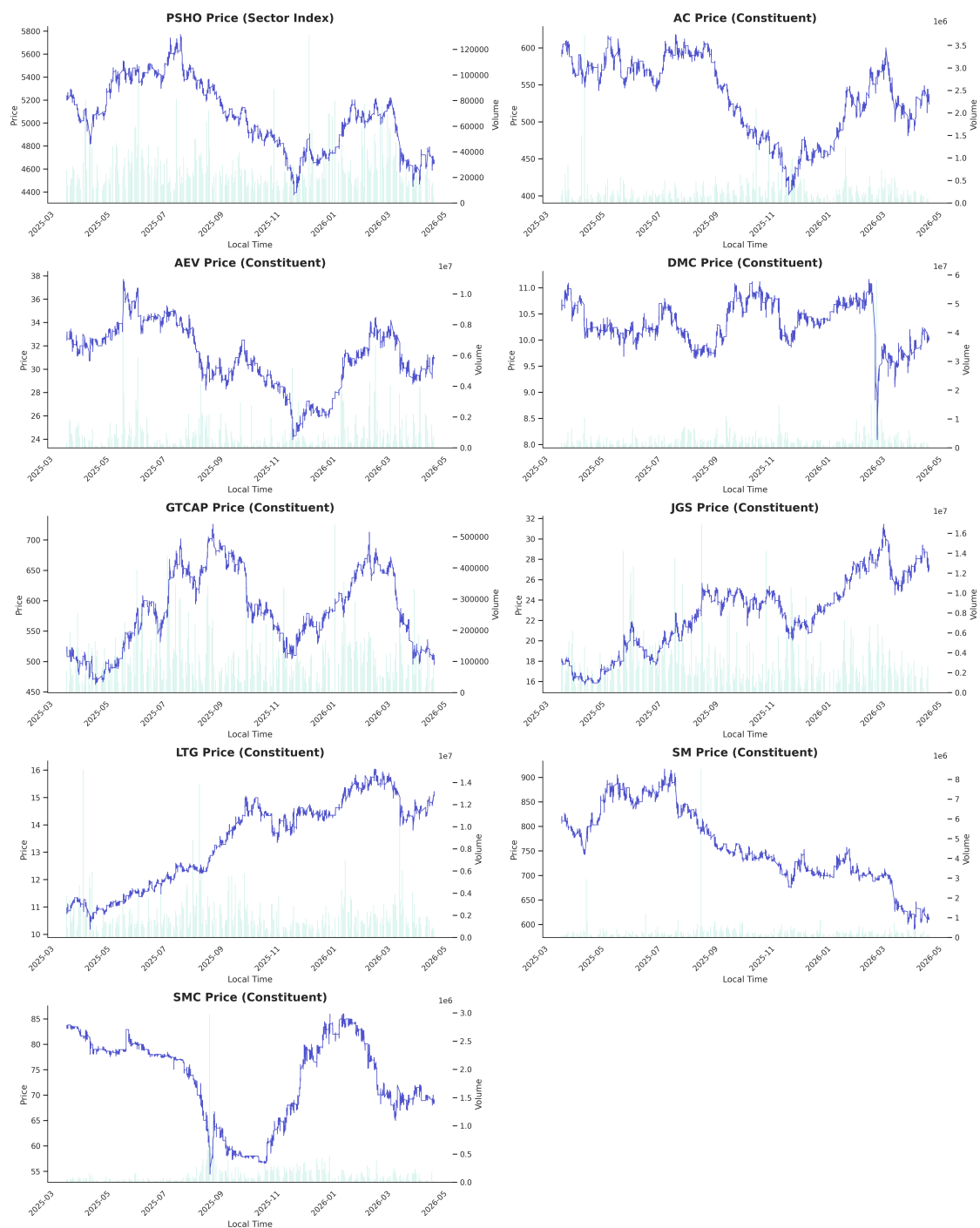
PSHO and Constituent Stocks' Price and Volume

Figure 4

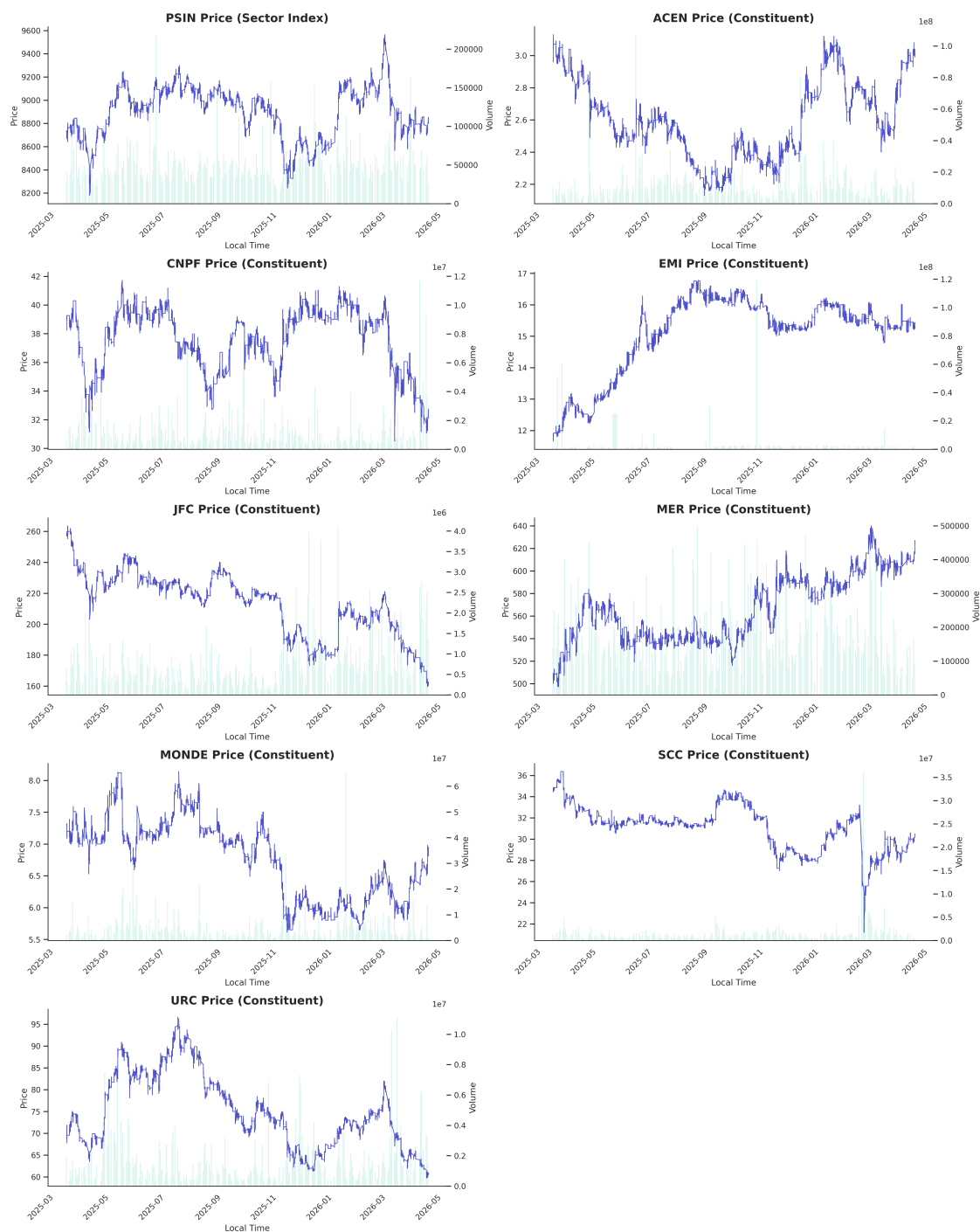
PSIN and Constituent Stocks' Price and Volume

Figure 5
PMSO Price and Volume

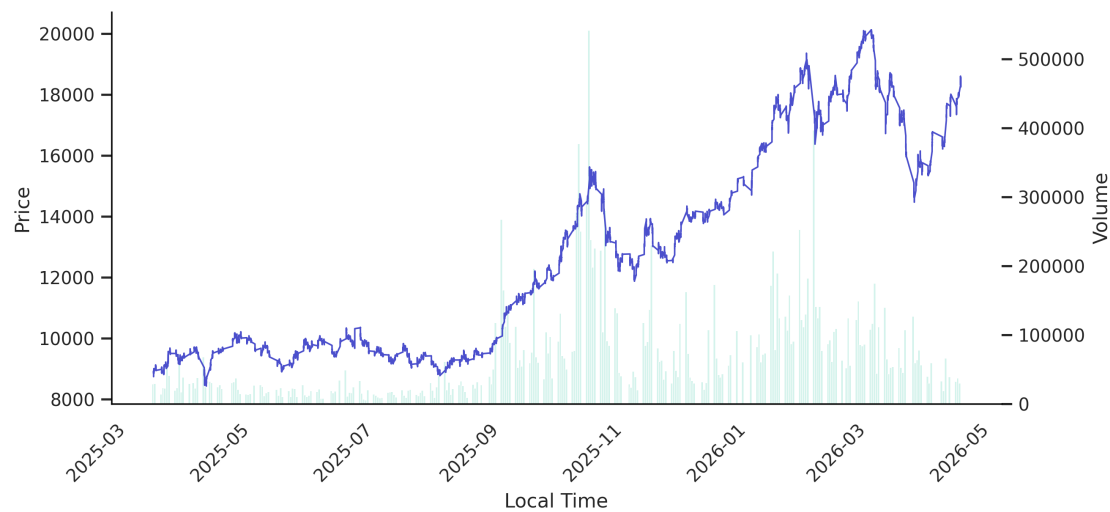
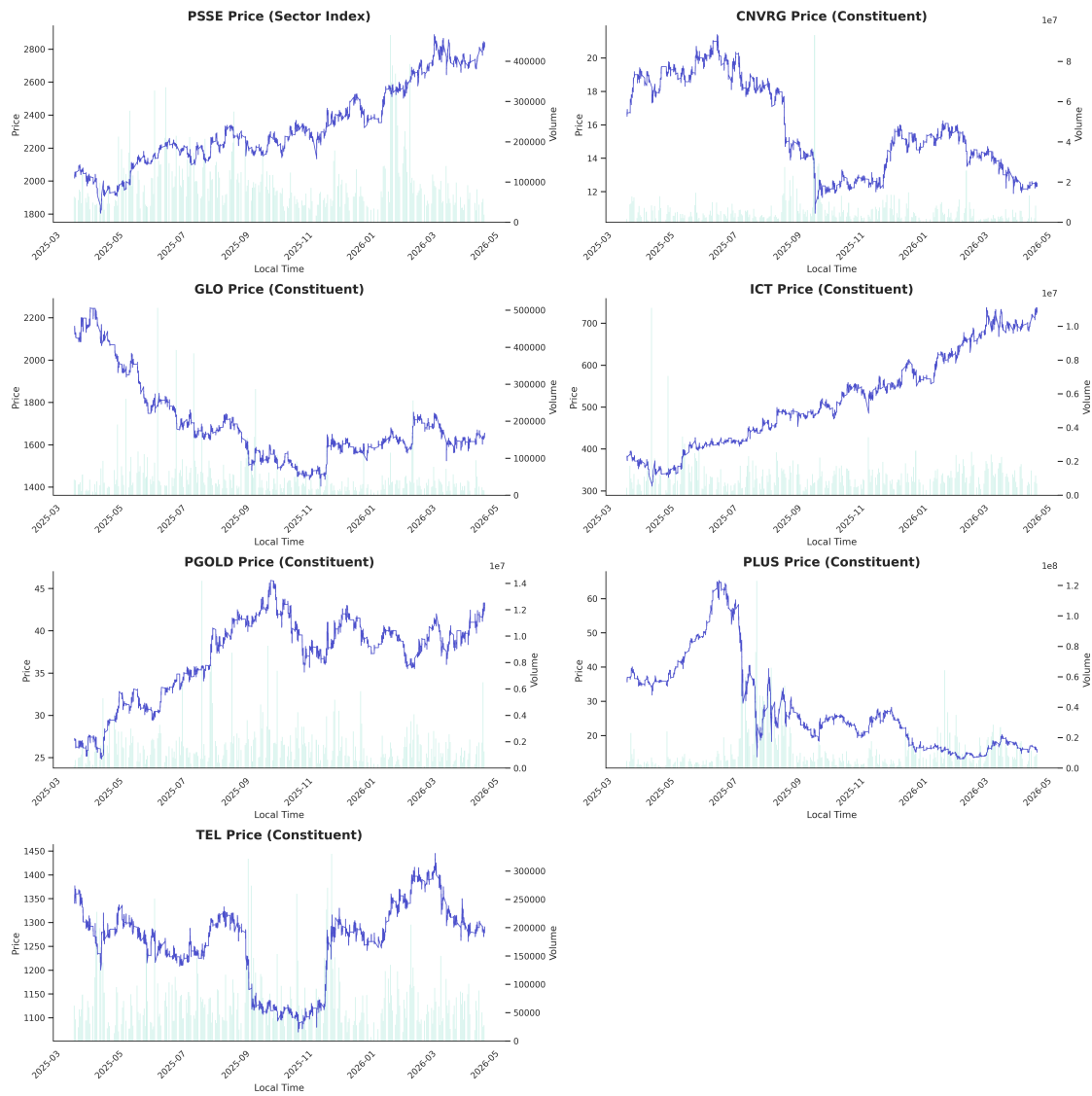


Figure 6
PSPR and Constituent Stocks' Price and Volume



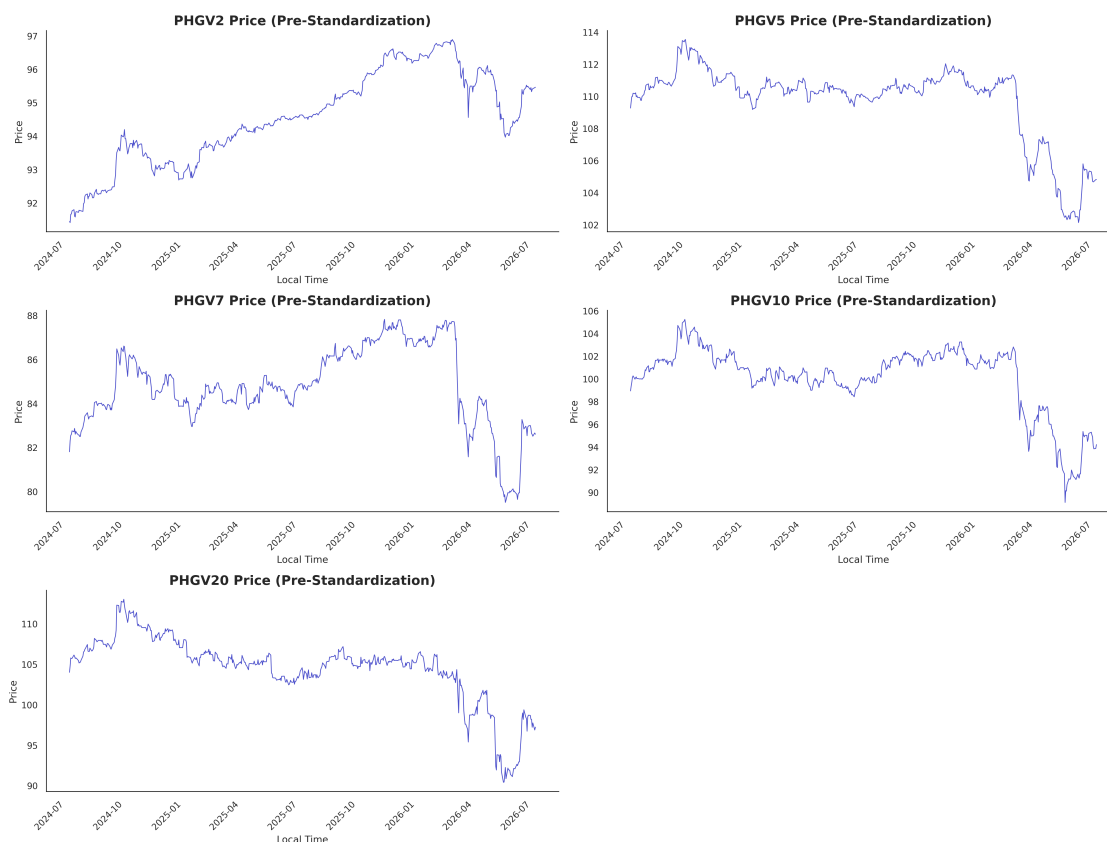
Figure 7
PSSE and Constituent Stocks' Price and Volume



the onset of the 2026 US-Iran war. Government bond prices, shown in Figure 8 for example, reflect a significant drop in value during the event, similar to stocks.

Figure 8

Government Bonds' Prices



Copper prices interestingly, however, seem unaffected by the 2026 war shocks, as seen in Figure 9.

Oil prices, intuitively, receive the greatest shock due to the mentioned geopolitical conflict, as shown in Figure 10.

The PHP-USD exchange rate reflects the Philippines' ongoing rising inflation levels, with a consistent uptrend of USD prices, that may have also been amplified by the US-Iran war, shown in Figure 11.

Finally, gold prices fluctuate with the least volatility, shown in Figure 12, receiving minimal shocks for the second half of 2025, and experiencing higher volatility, albeit

Figure 9

COMEX Continuous Copper Futures' Prices

**Figure 10**

Brent Crude Oil Continuous Futures' Prices

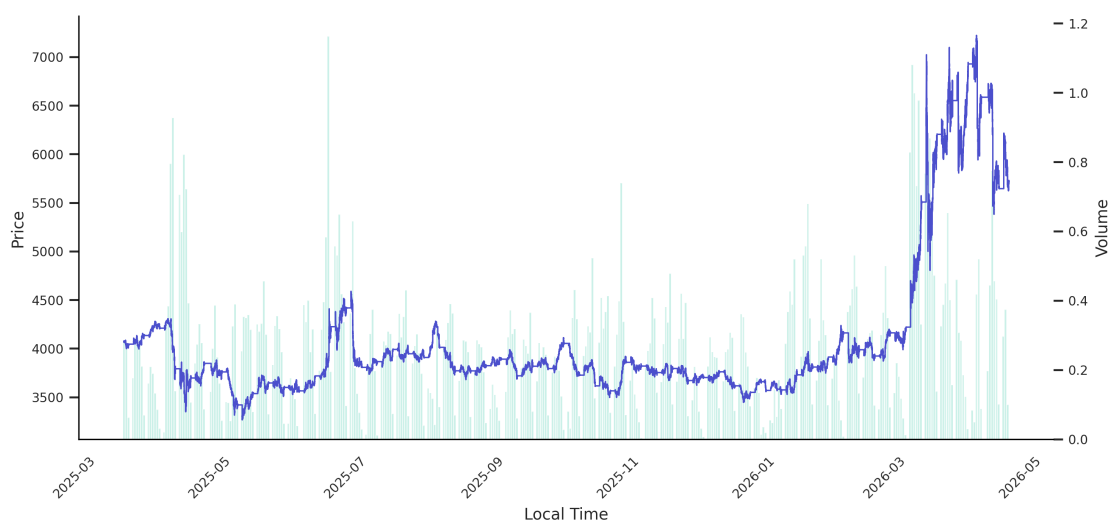
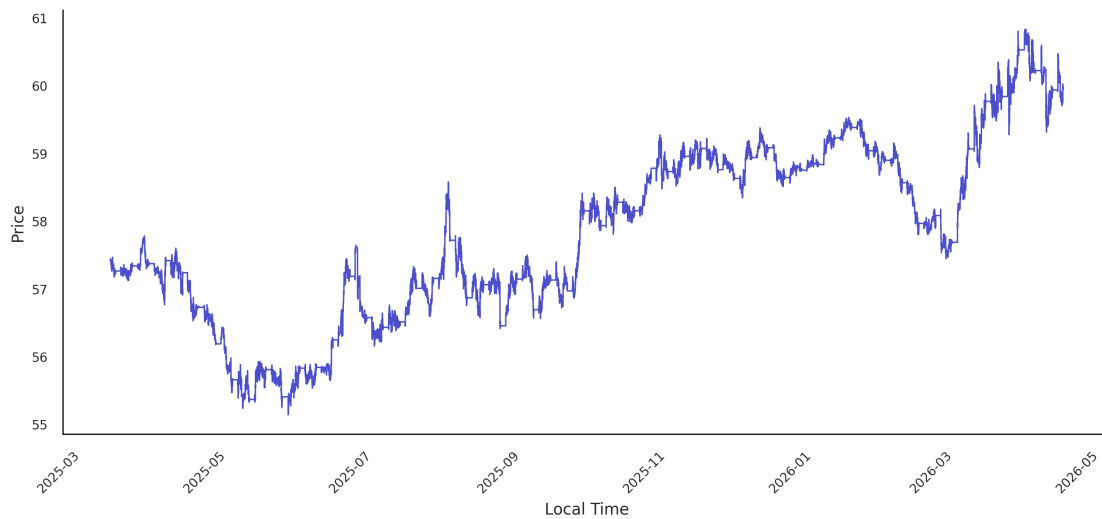


Figure 11*PHP to USD Exchange Rate*

retaining much of its stability, during the first three months of 2026.

News Data

News data were collected from both Philippine-based and international sources filtered down only to the ones that describe the Philippine financial domain and macroeconomically relevant international events. Each data observation consists only of an article headline indexed by its timestamp in local time (UTC+08). The date range of the news dataset mirrors that of the financial time-series datasets for training, validation, and testing requirements. The main source of this data is also LSEG Workspace, particularly its News Monitor 2.0 feature, where a consolidated, comprehensive search query was used to collect only the most important headlines. Only journalistic news headlines about real world events were included, and other kinds of news updates (such as corporate filings and stock commentary) were excluded. Because the LSEG News Monitor is a global source of news articles, a portion of the dataset comes from news outlets in other countries, resulting in non-English news headlines of varied languages. Figure 13 shows the frequency of the most common non-English languages found in the dataset.

Data inspection revealed that the timestamps collected from LSEG are not fully

Figure 12
PHP to Gold Exchange Rate

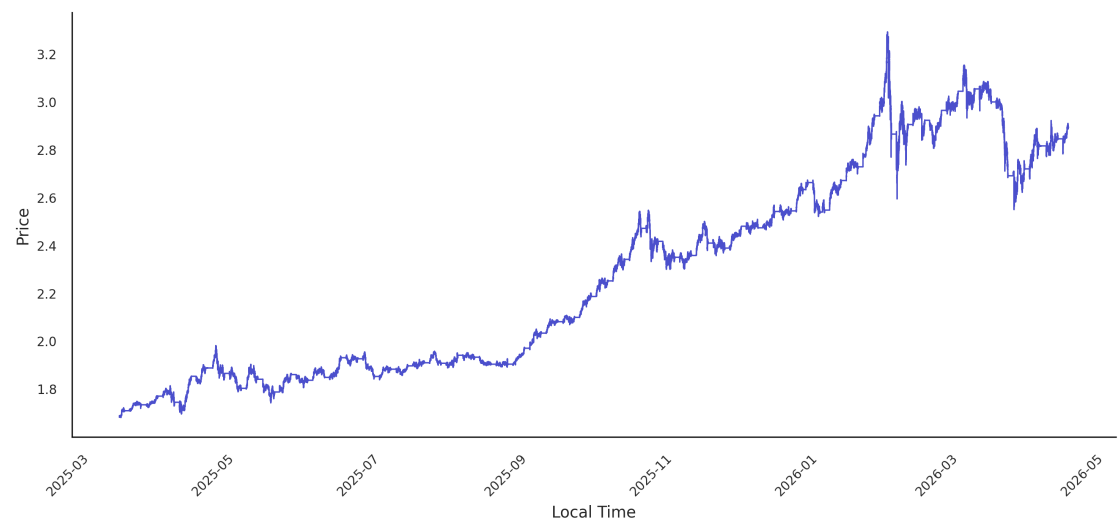
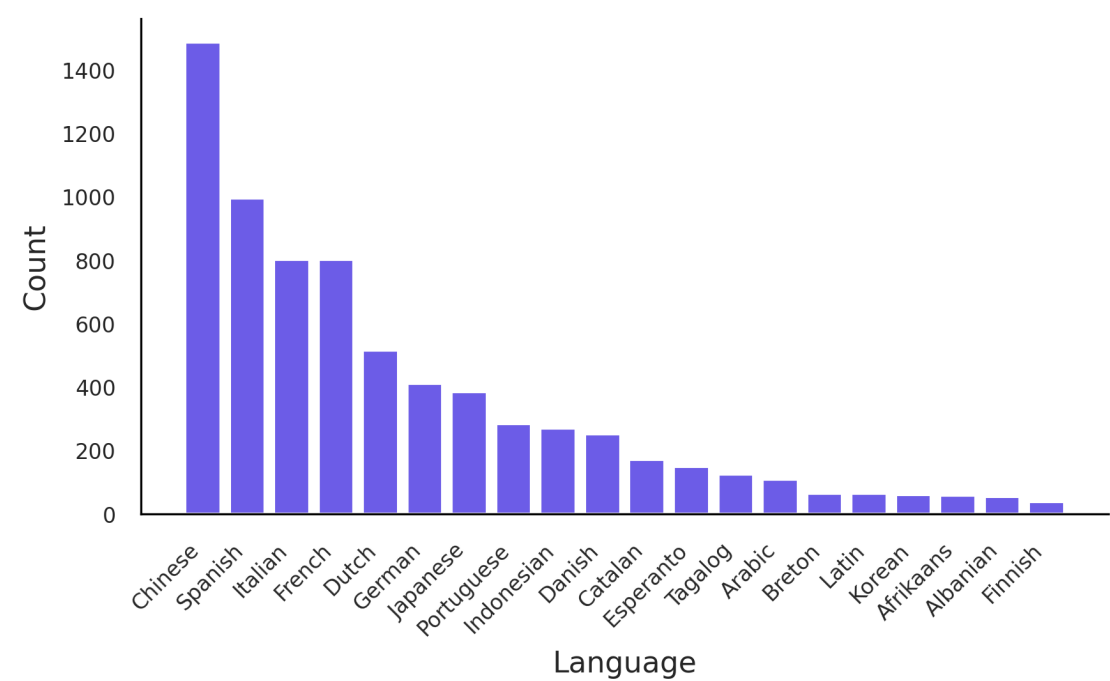


Figure 13
Top Non-English News Headline Languages

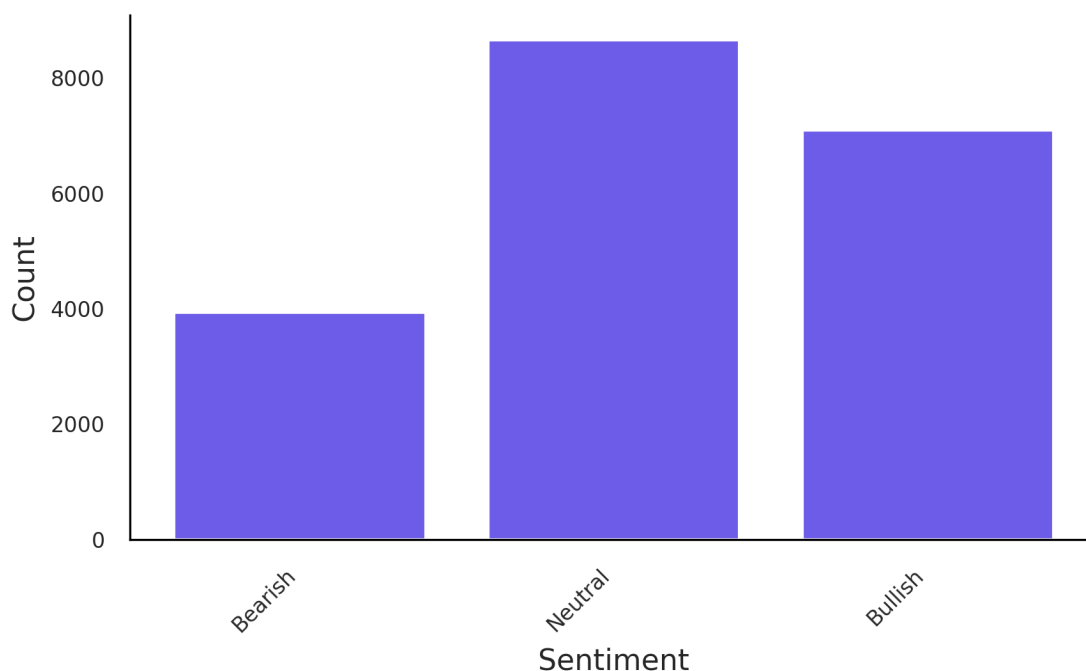


accurate, and may be off from the actual news publication timestamps by up to a few days. Given this, additional web scraping was done with the help of Serper.dev, a Google Search API, to recover publication timestamps directly from the news article pages themselves by attempting searches on Google News or Google Search for web articles with either the exact or similar titles, and scraping the articles' HTML source code for the publication timestamp metadata, or otherwise the displayed publication timestamp on the page headers. This problem afflicts only the less popular news outlets, particularly the local sources, whereas global news outlets such as Reuters have fairly accurate timestamps that are off by only about a few minutes up to an hour. As such, Reuters timestamps were preserved, whereas only timestamps from the rest of the news sources in the dataset were replaced, in order to reduce search API costs. Scraped timestamps were cast into local time (UTC+08) to match the temporality of the preserved Reuters timestamps. This entire operation is, although unfavorably, a potentially unhygienic multi-filter process; search queries may not lead to exactly the same articles, leading to publication timestamps that reflect a similar article published around the same time instead of the exact same news article (which nonetheless is permitted into the dataset since the general meaning is preserved, and for machine learning goals specifically, a pairing of such content to a relevant news publication timestamp is sufficient); moreover, some article URLs also cannot be scraped, and timestamp metadata can be missing, which all reduce the size of the dataset. The final dataset size is reported in the next chapter.

Figure 14 showcases the distribution of investor sentiment classifications of the news article headlines from the Modern-FinBERT model. News stories are varied in sentiment, corresponding to a high concentration of financially relevant news articles.

Social Media Data

Social media data from X were collected, queried using search terms that describe investor sentiment, commentary, and news updates within the Philippine financial domain (namely on stocks), as well as macroeconomically relevant international events. X posts are

Figure 14*News Sentiment Distribution*

structured by a hierarchy: a single thread contains posts and replies, each of which may consist of one or multiple sentences, but a single unit/data observation for this modality is only a single post, not a thread.

Figure 15 showcases the distribution of sentiment classifications of the social media posts from the *tabularisai/multilingual-sentiment-analysis* model. As can be seen, majority of the posts are in the neutral category, showing a higher concentration of posts with less relevance to the financial domain as compared to news data.

Table 5 showcases the relevant columns found in the raw social media datasets. The variables RetweetCount, ReplyCount, LikeCount, QuoteCount, ViewCount, and BookmarkCount are the *post impact features*, whereas the variables prefixed by “Author” are the *influence features*. The current source for social media data is the Twitter Scraper V2 Apify Actor, which is an online pre-built web scraper with event-based pricing. The total dataset size is to be reported in the next chapter.

Figure 15
Social Media Sentiment Distribution

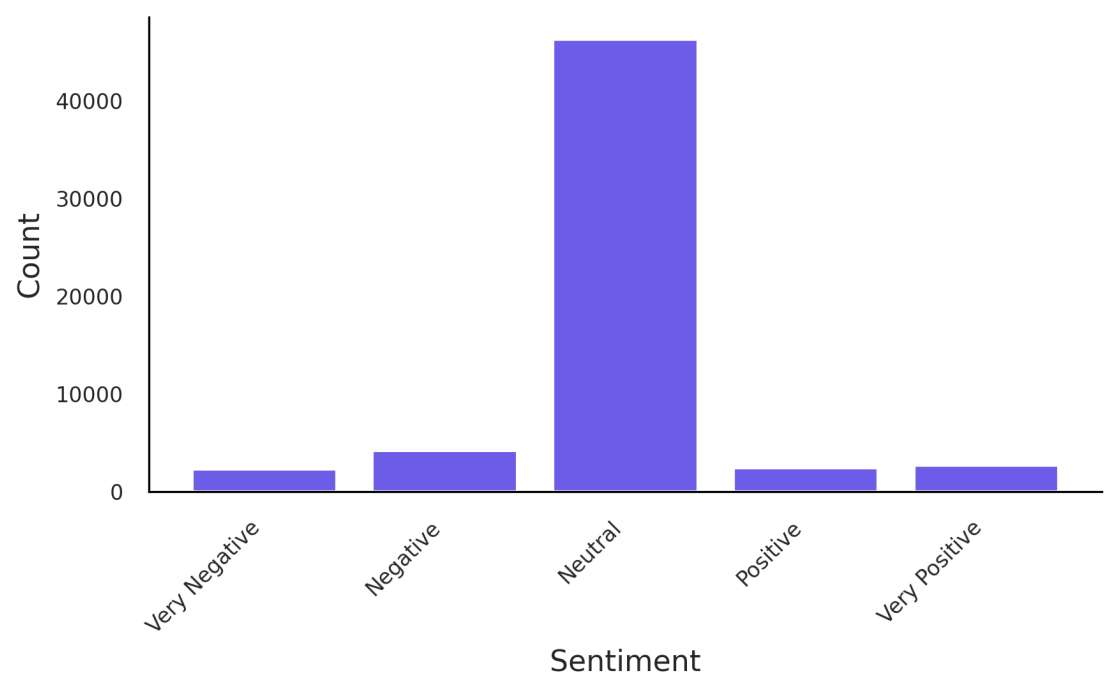


Table 5
Relevant Raw Variable Columns for Social Media Data

Column	Description
CreatedAt	Date and time of the current timestamp in universal time (UTC+00)
Text	Content of the X post
RetweetCount	Number of retweets of the post
ReplyCount	Number of replies under the post
LikeCount	Number of likes of the post
QuoteCount	Number of quotes of the post
ViewCount	Number of times the post has been viewed

Continued on next page

Table 5*Relevant Raw Variable Columns for Social Media Data (Continued)*

Column	Description
BookmarkCount	Number of times the post has been bookmarked
AuthorIsBlueVerified	Whether the post author is blue verified
AuthorFollowers	Number of followers of the post author
AuthorFollowing	Number of users followed by the post author
AuthorFavouritesCount	Number of posts saved by the post author as favorites
AuthorMediaCount	Number of media items (photo and video) posted by the author
AuthorStatusesCount	Number of statuses of the post author

Chapter 5: Data Preprocessing

The experimental setup consists of the completed models per experimental setting under the specifications dictated by each unique independent variable configuration. This means that the goal of the setup for every experimental setting is to construct a stock prediction model that can predict the return directions of, respectively, the 30 blue-chip stocks, on the test dataset, respecting equivalent training conditions for comparability. Each model's feature matrix includes the following:

- Technical indicators of every stock: feature engineering was done so that on top of the raw stock data columns identified above, the stock's features include *closing price indicators*, namely Logarithmic Returns $r(1)_t$, $r(5)_t$, $r(10)_t$, and $r(30)_t$, the Cumulative Intraday Logarithmic Return $R_{i,t}$, the Relative Strength Index $RSI(14)_t$, Price Rates of Change $PROC(5)_t$, $PROC(10)_t$, and $PROC(20)_t$, Moving Average Convergence Divergence indicators $MACD(12, 16)_t$, $Signal(12, 16, 9)_t$, and $Histogram(12, 16, 9)_t$, Moving Averages $MA(10)_t$ and $MA(50)_t$, Exponential Moving Averages $EMA(10)_t$ and $EMA(50)_t$, Price Oscillators $OSCP(10, 50)_t$ and $EOSCP(10, 50)_t$, Price Disparity indicators $DISP(10, 50)_t$ and $EDISP(10, 50)_t$, the Psychological Line $PL(12)_t$, the Rank Correlation Index $RCI(9)_t$, and the Bollinger Band indicator $BB\%(20)_t$ (defaulting to 0.5 if $BBU(20)_t = BBL(20)_t$), *high-low-closing price indicators*, namely the Stochastic Oscillators $\%K_t$, Fast $\%D_t$, and Slow $\%D_t$, Parabolic SAR indicators $(Close_t - PSAR_t)/Close_t$, $Trend_t$ (1 for Uptrend, 0 for Downtrend), and $Reversal_t$ (1 during the start of a trend reversal, 0 otherwise), Average True Range $ATR(14)_t$, and Average Directional Index indicators $ADX(14)_t$, $+DI(14)_t$, and $-DI(14)_t$, as well as *close-volume indicators*, namely Relative Volume $RVOL(20)_t$, and change in On-balance Volume $OBV_t - OBV_{t-1}$.
- Similarly, closing price indicators, high-low-closing price indicators, and close-volume indicators for the corresponding sector-level index for the target stock, and the PSEi.
- Bond indicators (spread, midspread, midyield and bid-ask yield spread), as well as

pairwise yield spreads (differences in bid yields between two different government bonds) for the PHGV datasets.

- Bid-ask indicators (spread, midprice, relative spread based on midprice, and 10-, 30-, and 50-period moving averages of spread) and close indicators for copper futures.
- Bid-ask indicators, closing price indicators, high-low-closing price indicators, and close-volume indicators for oil futures.
- Bid-ask indicators, high-low-closing price indicators, and close-volume indicators for foreign exchange datasets (PHP and XAU).
- Sinusoidal encodings for the cyclical nature of each trading day, day of the week, day of the month, and month of the year (only for MLP datasets; for transformers, only a single scalar elapsed time variable is constructed, for input into a Time2Vec module)

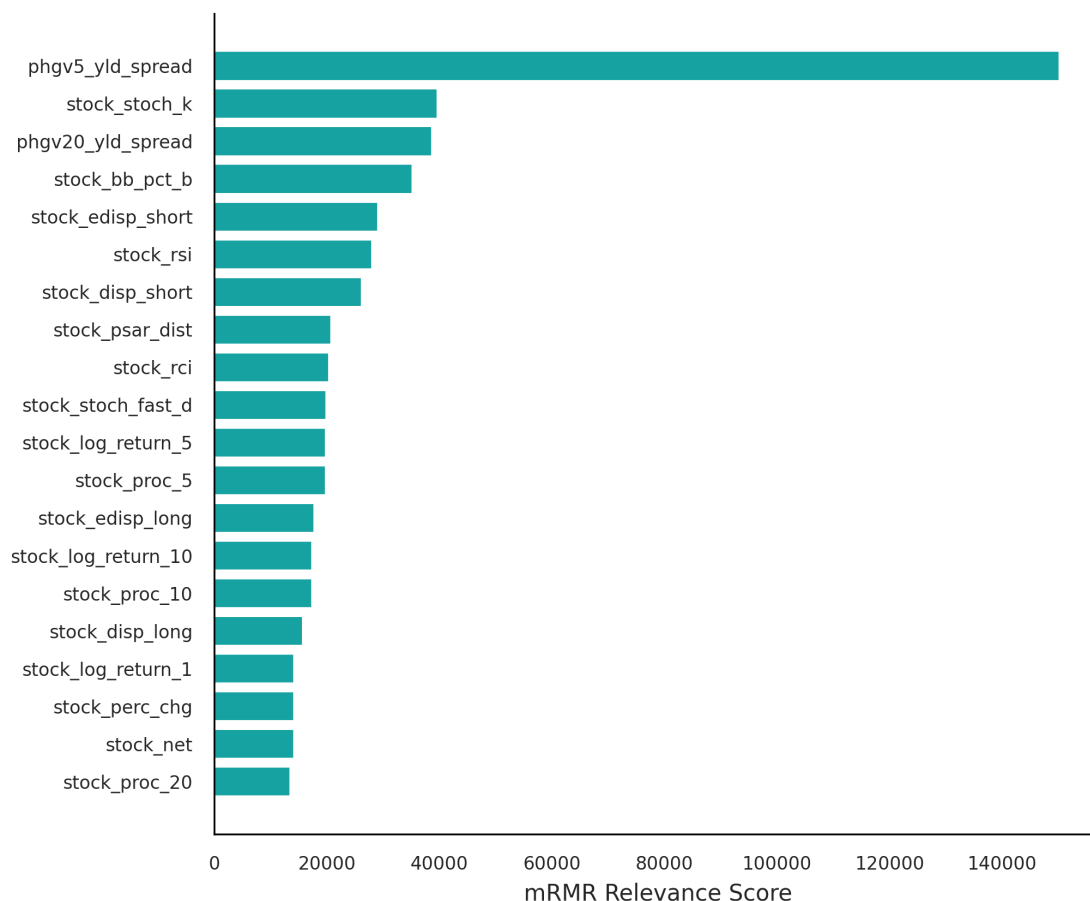
These features make up the economic features of the model data input. Among these features, an mRMR model is used to select the 100 most relevant and least redundant technical features. Since the datasets for the 30-minute and 10-minute prediction horizon models differ, separate mRMR feature selections are done for each. Figure 16 shows the most relevant financial time-series features for the 10-minute horizon models.

The top features are mostly stock technical indicators with some features from the government bond datasets. Similar features were selected for the 30-minute horizons, as shown in Figure 17

The datasets for 30-minute and 10-minute models differ due to the rule that no prediction horizon window should cross stock price boundaries. For instance, for 10-minute models, timestamps 11:51 AM to 12:00 PM are removed, since their 10-minute returns, respectively, are at 12:01 PM and 12:10 PM, at which the Philippine Stock Exchange is under recess, when trading is halted until 1:00 PM. This implies that more dataset samples are removed from 30-minute datasets than from 10-minute datasets. A total of 65,388 trading minutes compose the final stock datasets for each stock on the 10-minute prediction horizon, where each stock dataset has 101 features for transformer models (the 100 features

Figure 16

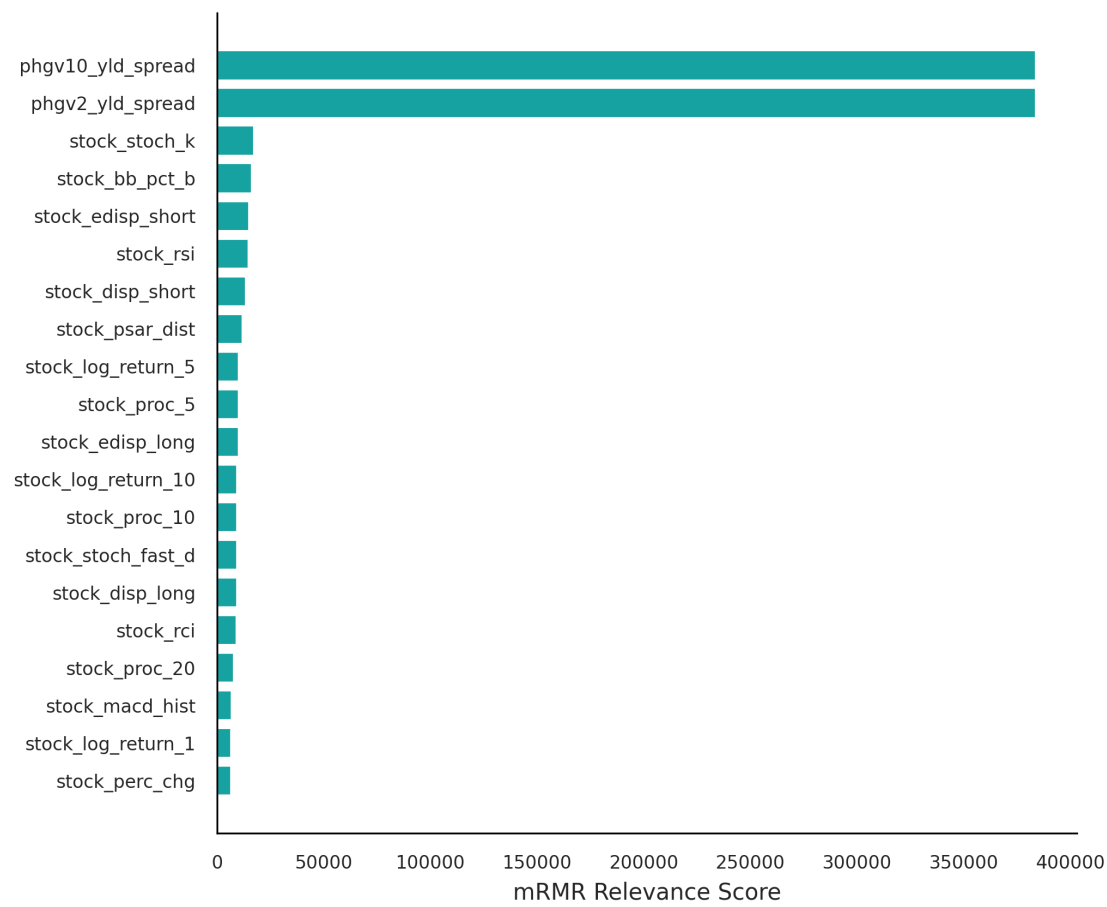
Top Relevant Features from mRMR for 10-minute Prediction Horizon Models



selected by mRMR and an elapsed time variable for the Time2Vec input), or 110 features (100 mRMR selections and time-derived features). On one hand, a total of 54,928 available trading minutes compose the stock datasets on the 30-minute prediction horizon.

Other data modalities (news data and social media) take on separate data streams from the numerical data, and from each other. News and social media data also adopted a number of sentiment indicators, and for social media data only, indicators derived from impact and influence features (these additional indicators for text data are only used for MLPs; transformers rely fully on the content embeddings and for social media, the raw impact and influence features). After implementing the before-mentioned procedure for recovering timestamps for news data, a total of 19,677 timestamped news headlines were

Figure 17
Top Relevant Features from mRMR for 30-minute Prediction Horizon Models



obtained. For social media on one hand, a total of 57,703 X posts were obtained. All of these dataset sizes encompass the training, validation, and testing splits.

Chapter 6: Modelling

The discussion in the following describes in full mathematical detail all the necessary components used to construct the deep learning models used for every experimental setting, as well as the process for configuring data inputs into the models.

Notation

Throughout, tensors are written with explicit index sets to make batching unambiguous. The following items are either model design hyperparameters (when values are specified) whose values are *the same* across all experimental settings unless otherwise specified in the following, or intermediately derived tensor dimension sizes.

- $B = 32$ — batch size. Both transformers and MLP models use the same batch size (32).
- $S = 30$ — number of stocks (assets) processed jointly in a batch element.
- $T_s = 60$ — length of the stock time-series window.
- T_n — number of auxiliary (news articles or social media posts) items available in a batch element. This is determined by the number of such items within the lookback window defined by the most recent 270 trading minutes or most recent 24 hours (whichever is longer), so T_n can vary between batches due to irregular publication timestamps.
- $d \in \{100, 110\}$ — dimensionality of the raw per-timestep stock feature vector (100 features for transformers, 110 for MLP due to additional timestamp-derived features).
- $d_{\text{tdim}} = 1024$ — dimensionality of the Cohere embeddings (original embeddings before further preprocessing) extracted for every news headline and social media post.
- $d_{\text{news}} = 15$ — dimensionality of the concatenated news sentiment input dimensions for the MLP models (if present).
- $d_{\text{msoc}} = 15$ — dimensionality of the concatenated social media sentiment and impact/influence dimensions for the MLP models (if present).
- $d_{\text{tsoc}} \in \{5, 7\}$ — dimensionality of the concatenated social media impact/influence

input dimensions for the transformer models (if social media is present): equal to 5 for 30-minute horizon transformers, or 7 for 10-minute horizon transformers.

- $d_m = 384$ — dimensionality of the hidden layers of the MLP models.
- $E_s = 128$ — stock (or news/social) content embedding dimension.
- $E_t = 16$ — temporal (Time2Vec) embedding dimension.
- $E = E_s + E_t = 144$ — total token embedding dimension used by every transformer block.
- $H = 8$ — number of attention heads, $d_h = E/H = 18$ the per-head dimension.
- $L \in \{1, 5\}$ — number of transformer layers (1) for every transformer module, or the number of hidden layers (5) for MLP models.
- $K = 5$ — number of auxiliary items dynamically selected per (stock, timestep) pair.
- $M = 500$ — number of Monte Carlo samples used by the perturbed Top- K operator.
- $\sigma = 0.05$ — perturbation standard deviation of the perturbed Top- K operator. This value is annealed as training progresses.
- $\text{LN}(\cdot)$ — LayerNorm; $\text{GELU}(\cdot)$ — Gaussian Error Linear Unit; $\odot$ — Hadamard product (element-wise product for two same-sized tensors of any size).
- $\mathcal{H} \in \{10, 30\}$ — prediction horizon in minutes.
- N — number of sliding-window sequences in a split (training, validation, or testing split).
- Dropout — dropout operation that neutralizes a percentage of input activations in order to regularize training. A dropout value of 0.1 remains the same for all instances where it is applied and is constant all throughout training. During model validation and testing, dropout is turned off and activations remain unchanged.
- $c \in \{0, 1\}$ — the binary class label (up/down move over the next H minutes).
- $y \in \{0, 1\}^{N \times S}$ — ground-truth labels; $\hat{p} \in [0, 1]^{N \times S}$ — predicted probabilities of the positive class per stock.

Superscripts index layers, subscripts index tensor coordinates. Batch and stock indices

are suppressed when a formula applies identically to every (b, s) pair.

The Differentiable Top-K Operator

This module produces a *soft, differentiable* approximation of the hard operation “select the indices of the k largest coordinates of a vector”, following the perturbed-optimizer / stochastic smoothing framework of Berthet et al. It is used twice downstream: to softly select the K most relevant news items and the K most relevant social posts for each stock/timestep pair.

Forward Pass: Monte Carlo Smoothing

Let $x \in \mathbb{R}^d$ be one row of the (flattened) input score tensor $x \in \mathbb{R}^{N \times d}$, where N ranges over every (b, s, t) triple being scored jointly. Let $Z^{(1)}, \dots, Z^{(M)} \stackrel{iid}{\sim} \mathcal{N}(0, I_d)$. Define the perturbed vectors

$$\tilde{x}^{(m)} = x + \sigma Z^{(m)}, \quad m = 1, \dots, M. \quad (81)$$

For each sample, the hard operator $\pi^{(m)} = \arg \text{top}_k(\tilde{x}^{(m)}) \subset \{1, \dots, d\}$ returns (an implementation-defined ordering of) the k coordinates with largest value. Writing $\kappa^{(m)} : \{1, \dots, k\} \rightarrow \pi^{(m)}$ for the bijection induced by `torch.topk` that assigns each of the k “slots” to a coordinate, define the per-sample one-hot slot indicator

$$b_{\kappa,j}^{(m)} = \mathbb{1}[\kappa^{(m)}(\kappa) = j], \quad \kappa = 1, \dots, k, \quad j = 1, \dots, d. \quad (82)$$

The forward output is the empirical mean over samples,

$$\hat{I}_{\kappa,j}(x) = \frac{1}{M} \sum_{m=1}^M b_{\kappa,j}^{(m)} \approx \Pr_Z[\kappa^{(M)}(\kappa) = j \mid x] \in [0, 1], \quad (83)$$

i.e. $\hat{I} \in [0, 1]^{k \times d}$ is a doubly-substochastic matrix whose row κ is (approximately) the marginal distribution, under Gaussian perturbation of x , of which coordinate lands in slot κ of the top- k set. Summing $\hat{I}$ over κ gives the (approximate) marginal inclusion probability of coordinate j in the top- k set; summing over j gives 1 for every slot.

For memory efficiency the M samples are drawn in chunks of size c and the sums in (83)–(85) are accumulated chunk-by-chunk (an unbiased running sum), which is mathematically equivalent to drawing all M samples at once.

Backward Pass: Perturbed Optimizer Gradient Estimator

Because $\arg \text{topk}$ is piecewise constant, $\nabla_x \hat{I}$ does not exist classically; the module instead implements a Monte Carlo estimator of the Jacobian of the *smoothed* map $F_\sigma(x) = \mathbb{E}_Z[b(x + \sigma Z)]$ obtained from Stein’s lemma for Gaussian perturbations,

$$\frac{\partial}{\partial x_j} \mathbb{E}_Z[f(x + \sigma Z)] = \frac{1}{\sigma} \mathbb{E}_Z[Z_j f(x + \sigma Z)]. \quad (84)$$

Applying (84) with $f = b_{\kappa,\cdot}$ and estimating the expectation with the same M samples gives the *expected gradient* tensor

$$G_{\kappa,j}(x) = \frac{1}{M\sigma} \sum_{m=1}^M Z_j^{(m)} b_{\kappa,j}^{(m)} \approx \frac{1}{\sigma} \mathbb{E}_Z[Z_j b_{\kappa,j}(x + \sigma Z)] = \frac{\partial \hat{I}_{\kappa,j}}{\partial x_j}. \quad (85)$$

Only the *diagonal* term (index j of the noise matched to index j of the output coordinate) is retained — i.e. the full Jacobian $\partial \hat{I}_{\kappa,j} / \partial x_{j'}$ for $j' \neq j$ is approximated as zero, and G is cached as a constant tensor of shape (N, k, d) , independent of M once computed.

Given an upstream gradient $\partial \mathcal{L} / \partial \hat{I} \in \mathbb{R}^{N \times k \times d}$, the straight-through backward rule implemented is

$$\frac{\partial \mathcal{L}}{\partial x_{n,j}} = \sum_{\kappa=1}^k \frac{\partial \mathcal{L}}{\partial \hat{I}_{n,\kappa,j}} G_{n,\kappa,j}. \quad (86)$$

Time2Vec

For a scalar timestamp $t \in \mathbb{R}$, Time2Vec produces a vector embedding that mixes one linear (trend) component with $E_t - 1$ learned periodic (Fourier) components,

$$\tau_0(t) = w_0 t + b_0, \quad \tau_i(t) = \sin(w_i t + b_i), \quad i = 1, \dots, E_t - 1, \quad (87)$$

$$\Phi(t) = [\tau_0(t), \tau_1(t), \dots, \tau_{E_t-1}(t)] \in \mathbb{R}^{E_t}, \quad (88)$$

with learnable parameters $w_0, b_0 \in \mathbb{R}$ and $w, b \in \mathbb{R}^{E_t-1}$. This map is applied elementwise to any tensor of timestamps (broadcast over the last axis).

Algorithm 1 Perturbed Top-K forward pass (per chunk)

```

1: Input:  $x \in \mathbb{R}^{N \times d}$ ,  $k$ , total samples  $M$ ,  $\sigma$ , chunk size  $c$ 
2:  $\hat{I} \leftarrow 0 \in \mathbb{R}^{N \times k \times d}$ ,  $G \leftarrow 0 \in \mathbb{R}^{N \times k \times d}$ 
3: for each chunk of size  $s \leq c$  until  $M$  samples drawn do
4:    $Z \sim \mathcal{N}(0, I) \in \mathbb{R}^{N \times s \times d}$ 
5:    $\tilde{x} \leftarrow x[:, \text{None}, :] + \sigma Z$ 
6:    $\pi \leftarrow \text{TopK}_k(\tilde{x}, \text{dim} = -1)$   $\triangleright$  indices, shape  $(N, s, k)$ 
7:   accumulate  $\hat{I} +=$  one-hot scatter of  $\pi$ ;  $G +=$  scatter of  $Z$  gathered at  $\pi$ 
8: end for
9:  $\hat{I} \leftarrow \hat{I}/M$ ,  $G \leftarrow G/(M\sigma)$ 
10: return  $\hat{I}$  (cache  $G$  for backward)

```

Input Embeddings
FinEmbedding (Stock Token Embedding)

Given a raw feature vector $x \in \mathbb{R}^{d_{\text{in}}}$ and its timestamp $t \in \mathbb{R}$,

$$u = W_{\text{lin}}x + b_{\text{lin}} \in \mathbb{R}^{E_s}, \quad (89)$$

$$v = \Phi(t) \in \mathbb{R}^{E_t}, \quad (90)$$

$$e = \text{Dropout}_p(\text{Concat}(u, v)) \in \mathbb{R}^E, \quad E = E_s + E_t. \quad (91)$$

NewsEmbedding

Given a raw news feature vector $x \in \mathbb{R}^{d_{\text{tdim}}}$ (only the leading quarter $d_{\text{tdim}}/4$ coordinates are used — the remaining three quarters of the raw feature dimension are reserved for auxiliary fields consumed elsewhere in the pipeline) and timestamp $t \in \mathbb{R}$,

$$\bar{x} = \text{LN}(x_{1:d_{\text{tdim}}/4}), \quad (92)$$

$$u = W_{\text{news}}\bar{x} + b_{\text{news}} \in \mathbb{R}^{E_s}, \quad (93)$$

$$v = \Phi(t) \in \mathbb{R}^{E_t} \quad (\Phi \text{ shared with FinEmbedding}), \quad (94)$$

$$e_{\text{news}} = \text{Dropout}_p(\text{Concat}(u, v)) \in \mathbb{R}^E. \quad (95)$$

SocialEmbedding

Given a raw social-text feature vector $x \in \mathbb{R}^{d_{\text{tdim}}}$, a numeric social-signal vector $s \in \mathbb{R}^{d_{\text{tsoc}}}$ (e.g. likes/shares/impact counts), and timestamp t ,

$$\bar{x} = \text{LN}(x_{1:d_{\text{tdim}}/4}), \quad (96)$$

$$u_{\text{text}} = W_{\text{text}}\bar{x} + b_{\text{text}} \in \mathbb{R}^{E_s - E_{\text{soc}}}, \quad (97)$$

$$u_{\text{soc}} = W_{\text{soc}}s + b_{\text{soc}} \in \mathbb{R}^{E_{\text{soc}}}, \quad (98)$$

$$v = \Phi(t) \in \mathbb{R}^{E_t}, \quad (99)$$

$$e_{\text{social}} = \text{Dropout}_p(\text{Concat}(u_{\text{text}}, u_{\text{soc}}, v)) \in \mathbb{R}^E. \quad (100)$$

Here E_{soc} is the configured social-embedding width and $E_s - E_{\text{soc}}$ is the text width, chosen so the total again equals $E = E_s + E_t$.

Self-Attention Building Blocks

SelfAttentionBlock

Let $X \in \mathbb{R}^{n \times E}$ be a sequence of n tokens (LayerNorm is applied on the flattened batch $\times$ group axis, but is per-token so we write it for one row-batch). With $\hat{X} = \text{LN}(X)$, and per-head projections $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)} \in \mathbb{R}^{E \times d_h}$, $h = 1, \dots, H$,

$$Q^{(h)} = \hat{X}W_Q^{(h)}, \quad K^{(h)} = \hat{X}W_K^{(h)}, \quad V^{(h)} = \hat{X}W_V^{(h)}, \quad (101)$$

$$A^{(h)} = \text{softmax}\left(\frac{Q^{(h)}(K^{(h)})^\top}{\sqrt{d_h}} + M\right) \in \mathbb{R}^{n \times n}, \quad (102)$$

$$O^{(h)} = A^{(h)}V^{(h)}, \quad (103)$$

$$O = \text{Concat}(O^{(1)}, \dots, O^{(H)})W_O + b_O \in \mathbb{R}^{n \times E}, \quad (104)$$

$$\text{Out} = X + \text{Dropout}_p(O). \quad (105)$$

If the attention is causal, the additive mask $M \in \{0, -\infty\}^{n \times n}$ is strictly upper triangular, $M_{ij} = -\infty$ for $j > i$ and 0 otherwise, enforcing that token i only attends to tokens $j \leq i$ (used for the intra-stock time axis). Otherwise $M \equiv 0$ (used for the inter-stock axis, which has no natural ordering). $\bar{A} = \frac{1}{H} \sum_h A^{(h)}$.

This block is applied to a 4-D tensor $X \in \mathbb{R}^{B \times G \times n \times E}$ (where G is whichever axis is *not* being attended over — stocks S for the time-series transformer, or timesteps T_s for the inter-stock transformer). LayerNorm and attention are computed with the B and G axes flattened together, i.e. independently per (b, g) pair, exactly as above.

FeedForward

With $\hat{X} = \text{LN}(X)$, expansion ratio r , weights $W_1 \in \mathbb{R}^{E \times rE}$, $W_2 \in \mathbb{R}^{rE \times E}$, and an optional binary token mask $\mu \in \{0, 1\}^n$,

$$F = \text{Dropout}_p \left(\text{GELU}(\hat{X}W_1 + b_1)W_2 + b_2 \right), \quad (106)$$

$$F \leftarrow F \odot \mu \quad (\text{broadcast over the feature axis, if } \mu \text{ given}), \quad (107)$$

$$\text{Out} = X + F. \quad (108)$$

SelfAttnTransformerLayer and Stacking

One layer composes the two blocks,

$$X^{(\ell+1)} = \text{FeedForward}(\text{SelfAttentionBlock}(X^{(\ell)})), \quad (109)$$

and SelfAttnTransformerLayers stacks L such layers, $X^{(0)} = X$, returning $X^{(L)}$ together with the attention weights averaged across layers, $\bar{A} = \frac{1}{L} \sum_{\ell=1}^L \bar{A}^{(\ell)}$.

StockTransformer (base model)

Input: raw features $x \in \mathbb{R}^{B \times S \times T_s \times d_{\text{in}}}$ and timestamps $t \in \mathbb{R}^{B \times S \times T_s}$.

1. Token embedding. $e_{b,s,\tau} = \text{FinEmbedding}(x_{b,s,\tau}, t_{b,s,\tau}) \in \mathbb{R}^E$.

2. Time-series transformer (causal, per stock). For each fixed (b, s) , the sequence $\{e_{b,s,\tau}\}_{\tau=1}^{T_s}$ is passed through L causal self-attention layers ($n = T_s$, mask causal):

$$h_{b,s,1:T_s} = \text{SelfAttnTransformerLayers}_{\text{causal}}(e_{b,s,1:T_s}) \in \mathbb{R}^{T_s \times E}. \quad (110)$$

3. Inter-stock transformer (non-causal, per timestep). The tensor is transposed to put the stock axis last-but-one, $h \rightarrow h^\top \in \mathbb{R}^{B \times T_s \times S \times E}$, and for each fixed (b, τ) a (single-layer, by default) non-causal self-attention block attends over the S stocks:

$$g_{b,\tau,1:S} = \text{SelfAttnTransformerLayers}(h_{b,\tau,1:S}^\top) \in \mathbb{R}^{S \times E}. \quad (111)$$

4. Read-out. Only the last timestep is used, $z_{b,s} = g_{b,T_s,s} \in \mathbb{R}^E$, and a linear head produces the two logits ("up" / "down"):

$$\hat{y}_{b,s} = W_{\text{proj}} z_{b,s} + b_{\text{proj}} \in \mathbb{R}^2. \quad (112)$$

Optionally the model also returns the intra-stock attention maps $\bar{A}^{\text{ts}} \in \mathbb{R}^{B \times S \times T_s \times T_s}$ and inter-stock attention maps $\bar{A}^{\text{ist}} \in \mathbb{R}^{B \times T_s \times S \times S}$.

Dynamic Auxiliary-Item Selection (DynamicSelection)

This module scores every (stock, timestep, auxiliary-item) triple and produces a differentiable top- K soft selection of auxiliary items (news or social posts) per (stock, timestep) pair, subject to a temporal-causality mask.

Causality mask. Let $t \in \mathbb{R}^{B \times S \times T_s}$ be stock timestamps (identical across stocks, so only $t_{b,0,\cdot}$ is used) and $t^n \in \mathbb{R}^{B \times T_n}$ the auxiliary-item timestamps, with a validity mask $\mu \in \{0, 1\}^{B \times T_n}$. Define

$$\nu_{b,\tau,i} = \mathbb{1}[t_{b,i}^n \leq t_{b,0,\tau}] \cdot \mu_{b,i} \in \{0, 1\}, \quad \tau = 1, \dots, T_s, \quad i = 1, \dots, T_n, \quad (113)$$

i.e. item i is a valid candidate at time τ only if it is a real (non-padding) item that was published no later than τ . This mask is broadcast identically across all S stocks.

Relevance score. With $u = x \in \mathbb{R}^{B \times S \times T_s \times E}$ the current stock representation and $y = \text{news/social} \in \mathbb{R}^{B \times T_n \times E}$ the auxiliary embeddings,

$$p_{b,i} = W_n \text{LN}(y_{b,i}) \in \mathbb{R}^{E/2}, \quad (114)$$

$$q_{b,s,\tau} = W_s \text{LN}(u_{b,s,\tau}) \in \mathbb{R}^{E/2}, \quad (115)$$

$$c_{b,s,\tau,i} = \nu_{b,\tau,i} [q_{b,s,\tau} \parallel \nu_{b,\tau,i} p_{b,i}] \in \mathbb{R}^E, \quad (116)$$

$$\text{score}_{b,s,\tau,i} = \begin{cases} w_{\text{score}}^\top c_{b,s,\tau,i} + b_{\text{score}}, & \nu_{b,\tau,i} = 1, \\ -\infty, & \nu_{b,\tau,i} = 0. \end{cases} \quad (117)$$

(The masking is applied twice in the implementation — once multiplicatively before concatenation and once as an additive $-\infty$ fill after the score is computed — which is

redundant but harmless: any mathematically valid (b, s, τ, i) contributes a genuine score, and every invalid one is forced to $-\infty$.)

Differentiable selection. The score tensor $\text{score}_{b,s,\tau,\cdot} \in \mathbb{R}^{T_n}$ is fed, independently for every (b, s, τ) , into the perturbed Top- K operator.

$$\text{Ind}_{b,s,\tau} = \text{PerturbedTopK}_{K,M,\sigma}(\text{score}_{b,s,\tau,\cdot}) \in [0, 1]^{K \times T_n}. \quad (118)$$

Row κ of $\text{Ind}_{b,s,\tau}$ is (approximately) a probability distribution over which auxiliary item occupies selection slot κ ; because masked entries carry score $-\infty$, they receive probability 0 under any finite σ in the limit, and in practice negligible probability. The module returns (Ind, ν) .

Selective Cross-Attention (CrossAttentionBlock)

Given stock-side queries $X \in \mathbb{R}^{B \times S \times T_s \times E}$, auxiliary-side keys/values $Y \in \mathbb{R}^{B \times T_n \times E}$ (broadcast identically across stocks), the soft selection $\text{Ind} \in [0, 1]^{B \times S \times T_s \times K \times T_n}$, and the causal/validity mask $\nu \in \{0, 1\}^{B \times S \times T_s \times T_n}$:

Slot validity mask. A selection slot κ is considered active if it places non-zero mass on at least one valid item,

$$a_{b,s,\tau,\kappa} = \mathbb{1} \left[\sum_i \text{Ind}_{b,s,\tau,\kappa,i} \nu_{b,s,\tau,i} > 0 \right] \in \{0, 1\}. \quad (119)$$

Query/key/value projections. With $\hat{X} = \text{LN}(X)$, $\hat{Y} = \text{LN}(Y)$ and per-head projections,

$$Q_{b,s,\tau}^{(h)} = \hat{X}_{b,s,\tau} W_Q^{(h)}, \quad K_{b,i}^{(h)} = \hat{Y}_{b,i} W_K^{(h)}, \quad V_{b,i}^{(h)} = \hat{Y}_{b,i} W_V^{(h)}. \quad (120)$$

Soft gather of the top- K keys/values. Rather than indexing, the (differentiable) selection distribution is used as a linear gather operator,

$$\tilde{K}_{b,s,\tau,\kappa}^{(h)} = \sum_{i=1}^{T_n} \text{Ind}_{b,s,\tau,\kappa,i} K_{b,i}^{(h)} \in \mathbb{R}^{d_h}, \quad (121)$$

$$\tilde{V}_{b,s,\tau,\kappa}^{(h)} = \sum_{i=1}^{T_n} \text{Ind}_{b,s,\tau,\kappa,i} V_{b,i}^{(h)} \in \mathbb{R}^{d_h}, \quad (122)$$

i.e. each of the K slots receives a probability-weighted mixture of item keys/values, which reduces to an exact gather in the limit $\sigma \rightarrow 0$ (hard top- K).

Scaled dot-product attention over the K slots.

$$s_{b,s,\tau,\kappa}^{(h)} = \frac{Q_{b,s,\tau}^{(h)} \cdot \tilde{K}_{b,s,\tau,\kappa}^{(h)}}{\sqrt{d_h}}, \quad (123)$$

$$s_{b,s,\tau,\kappa}^{(h)} \leftarrow -\infty \text{ if } a_{b,s,\tau,\kappa} = 0, \quad (124)$$

$$\alpha_{b,s,\tau,\kappa}^{(h)} = \text{softmax}_{\kappa} \left(s_{b,s,\tau,\cdot}^{(h)} \right), \quad \alpha^{(h)} \leftarrow \text{Dropout}_p(\alpha^{(h)}) \rightarrow \text{NaNs} \rightarrow 0, \quad (125)$$

$$o_{b,s,\tau}^{(h)} = \sum_{\kappa=1}^K \alpha_{b,s,\tau,\kappa}^{(h)} \tilde{V}_{b,s,\tau,\kappa}^{(h)}. \quad (126)$$

(The NaN-to-zero step handles rows where *every* slot is invalid, i.e. softmax over an all $-\infty$ row.)

Output and residual. With $\bar{\mu}_{b,s,\tau} = \mathbb{1}[\sum_i \nu_{b,s,\tau,i} > 0]$ (whether *any* valid auxiliary item exists at all for this token),

$$o_{b,s,\tau} = \text{Concat}_h(o_{b,s,\tau}^{(h)})W_O + b_O, \quad (127)$$

$$o_{b,s,\tau} \leftarrow \bar{\mu}_{b,s,\tau} o_{b,s,\tau}, \quad (128)$$

$$\text{Out}_{b,s,\tau} = X_{b,s,\tau} + o_{b,s,\tau}. \quad (129)$$

Tokens with no valid auxiliary evidence thus fall back exactly to their input representation (pure residual, zero cross-attention contribution). The reported attention map is the head-averaged $\bar{\alpha}_{b,s,\tau,\kappa} = \frac{1}{H} \sum_h \alpha_{b,s,\tau,\kappa}^{(h)} \in \mathbb{R}^{B \times S \times T_s \times K}$.

Layer / stack. CrossAttnTransformerLayer composes this block with the masked FeedForward (mask $\bar{\mu}$), and CrossAttnTransformerLayers stacks L such layers with a fixed (Y, Ind, ν) context, averaging attention weights across layers as before.

StockNewsTransformer

Composes the base model with one news-fusion stage inserted between the time-series and inter-stock transformers:

$$h = \text{TimeSeriesTransform}(x, t) \quad (\text{causal}) \quad (130)$$

$$e_{b,i}^{\text{news}} = \text{NewsEmbedding}(\text{news}_{b,i}, t_{b,i}^n) \quad (131)$$

$$(\text{Ind}^n, \nu^n) = \text{DynamicSelection}(h, e^{\text{news}}, t, t^n, \mu^n) \quad (132)$$

$$h' = \text{CrossAttnTransformerLayers}(h, e^{\text{news}}, \text{Ind}^n, \nu^n) \quad (133)$$

$$\hat{y} = \text{InterStockTransform}(h') \quad (134)$$

StockSocialTransformer

Structurally identical to StockNewsTransformer but with the social-post embedding and its own selection/fusion parameters:

$$h = \text{TimeSeriesTransform}(x, t) \quad (135)$$

$$e_{b,i}^{\text{soc}} = \text{SocialEmbedding}(\text{text}_{b,i}, \text{social}_{b,i}, t_{b,i}^s) \quad (136)$$

$$(\text{Ind}^s, \nu^s) = \text{DynamicSelection}(h, e^{\text{soc}}, t, t^s, \mu^s) \quad (137)$$

$$h' = \text{CrossAttnTransformerLayers}(h, e^{\text{soc}}, \text{Ind}^s, \nu^s) \quad (138)$$

$$\hat{y} = \text{InterStockTransform}(h') \quad (139)$$

StockNewsSocialTransformer

Runs *both* fusion branches in parallel from the same time-series representation h and fuses them with a learned linear projection before the inter-stock transformer:

$$h = \text{TimeSeriesTransform}(x, t) \quad (140)$$

$$h^{\text{soc}}, \text{Ind}^s = \text{SocialFusion}(h, s, e_s, t, t^s, \mu^s) \quad (141)$$

$$h^{\text{news}}, \text{Ind}^n = \text{NewsFusion}(h, e_n, t, t^n, \mu^n) \quad (142)$$

$$h^{\text{fused}} = W_{\text{down}} \text{Concat}(h^{\text{soc}}, h^{\text{news}}) + b_{\text{down}} \in \mathbb{R}^{B \times S \times T_s \times E}, \quad W_{\text{down}} \in \mathbb{R}^{2E \times E} \quad (143)$$

$$\hat{y} = \text{InterStockTransform}(h^{\text{fused}}) \quad (144)$$

Both fusion branches consume the *same* pre-fusion stock representation h (not sequentially composed), so social and news relevance are scored independently against the same query representation, and their outputs are only combined via the final linear down-projection.

Baseline Non-Transformer Model

HiddenLayer

$$\text{HiddenLayer}(x) = \text{Dropout}_p(\text{GELU}(Wx + b)). \quad (145)$$

MultiLayerPerceptron

Given a single flattened feature vector $x \in \mathbb{R}^{d_{\text{feat}}}$ per stock (concatenating raw stock features, timestamps, and — if enabled — flattened news/social features into one vector, with no explicit temporal or attention structure),

$$h^{(0)} = \text{HiddenLayer}_0(x) \in \mathbb{R}^{d_m}, \quad (146)$$

$$h^{(\ell)} = \text{HiddenLayer}_\ell(h^{(\ell-1)}), \quad \ell = 1, \dots, L-1, \quad (147)$$

$$\hat{y} = (W_{\text{out}}h^{(L-1)} + b_{\text{out}}) \in \mathbb{R}^{1 \times 2} \quad (148)$$

Note that, in reference to the hyperparameters, $d_{\text{feat}} = d_i n + d_{\text{news}}$ with news enabled and social media unabled, $d_{\text{feat}} = d_i n + d_{\text{msoc}}$ with social media enabled and news unabled, and $d_{\text{feat}} = d_i n + d_{\text{msoc}} + d_{\text{news}}$ with all enabled.

Datasets and Temporal Windowing

Base Stock Windows (StockTransformerDataset)

Given pre-loaded tensors of per-stock features $X \in \mathbb{R}^{S \times T \times d_{\text{in}}}$, targets $Y \in \mathbb{R}^{S \times 2 \times T}$, and timestamps $\Theta \in \mathbb{R}^{S \times T}$ (all stocks share the same timestamp axis), the dataset exposes $N = T - L + 1$ overlapping windows. For window index i ,

$$x_i = X_{:,i:i+L,:} \in \mathbb{R}^{S \times L \times d_{\text{in}}}, \quad t_i = \Theta_{:,i:i+L} \in \mathbb{R}^{S \times L}, \quad y_i = Y_{:,i,i} \in \mathbb{R}^{S \times 2}, \quad (149)$$

i.e. a strided (stride 1) sliding window over the time axis, with the target read off at the window's final index $i + L - 1$ (equivalently i , since the target is stored pre-aligned to the

label at each window’s end).

Causal Auxiliary Evidence Windows (News / Social Media)

StockNewsTransformerDataset augments each stock window with a bounded, strictly-causal slice of news items. Let $t_i^{\text{last}} = t_{i,S,L}$ (the most recent stock timestamp in window i , identical across stocks). Two auxiliary quantities are computed from a reference time index:

$$\kappa(i) = \arg \min_u \left| \text{ref}[u] - t_i^{\text{last}} \right| \quad (\text{nearest reference timestamp}), \quad (150)$$

$$t_i^{\text{cut}} = \text{ElapsedTime}(\text{TextWindow}(\kappa(i), \mathcal{H})) \quad (\text{scaled lower cutoff}). \quad (151)$$

The retained news items are exactly those with timestamp in the half-open causal interval

$$\mathcal{W}_i = \left\{ j : t_i^{\text{cut}} < \theta_j^n \leq t_i^{\text{last}} \right\}, \quad (152)$$

and the dataset returns $(t_i, \theta^n[\mathcal{W}_i], x_i, e^n[\mathcal{W}_i], y_i)$, where e^n are the pre-computed news embeddings. This is precisely the causality window consumed by DynamicSelection: items are visible only if published no earlier than one prediction horizon before the window’s start and no later than the window’s last timestamp.

StockSocialTransformerDataset reuses the identical window $\mathcal{W}_i$ (constructed from the same timestamp reference and cutoff logic) but reads from a social-media tensor, adding the numeric impact vector $s[\mathcal{W}_i]$ alongside the text embeddings.

StockNewsSocialTransformerDataset computes *two independent* windows — one for news (inherited from the parent class) and a second, freshly computed window $\mathcal{W}_i^{\text{soc}}$ of the same form (152) applied to the social-media timestamp series — and returns both alongside the shared stock tensor:

$$(t_i, \theta^n[\mathcal{W}_i], \theta^s[\mathcal{W}_i^{\text{soc}}], x_i, s[\mathcal{W}_i^{\text{soc}}], e^n[\mathcal{W}_i], e^s[\mathcal{W}_i^{\text{soc}}], y_i). \quad (153)$$

Flat Baseline Dataset (StockMLPDataset)

Each training example is a single flattened feature vector $x \in \mathbb{R}^{d_{\text{feat}}}$ with scalar label $c \in \{0, 1\}$, and the label is one-hot expanded, $y = e_c \otimes \mathbb{1}_1 \in \{0, 1\}^{1 \times 2}$ (a length-1 leading

axis is kept so the tensor shape matches the transformer targets' trailing $(*, 2)$ convention).

Batch Collation

Because $|\mathcal{W}_i|$ varies across sequences within a batch, variable-length auxiliary tensors (news or social items, either 1-D timestamps or 2-D feature/embedding matrices whose last dimension matches the configured social or text embedding width) are right-padded with zeros to the batch-wise maximum length $T_n^{\max} = \max_i |\mathcal{W}_i|$, and further zero-padded up to K if $T_n^{\max} < K$ (so that the downstream perturbed top- K selector always has at least K candidate slots):

$$\tilde{e}_i^n = \text{Concat}\left(e^n[\mathcal{W}_i], 0_{(T_n^{\max}-|\mathcal{W}_i|)\times E}\right), \quad T_n = \max(T_n^{\max}, K). \quad (154)$$

For every text-embedding tensor a companion boolean validity mask is derived from the true (pre-padding) sequence lengths $\ell_i = |\mathcal{W}_i|$,

$$\mu_{i,j} = \mathbb{1}[j < \ell_i], \quad j = 0, \dots, T_n - 1, \quad (155)$$

which is exactly the mask μ consumed by DynamicSelection. Fixed-shape per-example tensors (stock features, timestamps, one-hot targets) are simply stacked.

Operationalization of Datasets and Models

Each of the dataset classes StockTransformerDataset, StockNewsTransformerDataset, StockSocialTransformerDataset, and StockNewsSocialTransformerDataset is instantiated twice—one for each prediction horizon setting (10-min or 30-min). Since the two settings have different targets, and with boundary conditions where prediction windows are not allowed to cross into the lunch break and the market close over to the next trading day, the 10-minute datasets do not contain the last 10 minutes before lunch and before market close, whereas the 30-minute datasets do not contain the last 30 minutes before lunch and market close, leading not only to different datasets, but also to slightly different dataset sizes. For the MLP models, StockMLPDataset is instantiated eight times, one for every combination of enabling news, enabling social media data, and switching between 30-minute and 10-minute prediction horizons, corresponding to an MLP-based experimental setting.

The model classes StockTransformer, StockNewsTransformer, StockSocialTransformer, StockNewsSocialTransformer, and MultiLayerPerceptron, then, simply ingest the tensor batch collations from every dataset, creating instances of the same model to match the corresponding dataset instantiations, in order to create the full suite of 16 experimental settings. Upon creation of each experimental setting, the following steps are undertaken to begin model training.

Class-Imbalance-Weighted Objective

Class frequencies are estimated once from (up to) the first 5000 training examples,

$$n_0 = \#\{(b, s) : c_{b,s} = 0\}, \quad n_1 = \#\{(b, s) : c_{b,s} = 1\}, \quad n = n_0 + n_1, \quad (156)$$

and inverse-frequency class weights are set as

$$w_0 = \frac{n}{2n_0}, \quad w_1 = \frac{n}{2n_1}, \quad (157)$$

(so that $w_0 n_0 = w_1 n_1 = n/2$, balancing each class's total contribution to the loss). The training criterion is then weighted cross-entropy over the two logits $\hat{y}_{b,s} \in \mathbb{R}^2$ produced by the model,

$$\mathcal{L}(\hat{y}, c) = -\frac{1}{|B|} \sum_{(b,s) \in B} w_{c_{b,s}} \log \frac{\exp(\hat{y}_{b,s, c_{b,s}})}{\sum_{k=0}^1 \exp(\hat{y}_{b,s,k})}. \quad (158)$$

Optimization Procedure

Optimizer and Gradient Accumulation

Parameters are updated with AdamW (decoupled weight decay λ), with gradients accumulated over A micro-batches before each step: for micro-batch $j = 1, \dots, A$ within an accumulation window,

$$\ell_j = \frac{1}{A} \mathcal{L}(f_\theta(x_j), c_j), \quad \theta \leftarrow \text{AdamW}\left(\theta, \sum_{j=1}^A \nabla_\theta \ell_j, \lambda\right), \quad (159)$$

i.e. an unbiased estimate of the full-batch gradient over an effective batch size $A \cdot |B|$.

Learning Rate Schedule

For the non-transformer (MLP) baseline only, a ReduceLROnPlateau scheduler halves the learning rate whenever the validation loss fails to improve by at least 10^{-4} for 4 consecutive validation rounds, down to a floor $\eta_{\min} = 10^{-6}$:

$$\eta \leftarrow \begin{cases} 0.5 \eta, & \text{plateau of length 4 detected,} \\ \eta, & \text{otherwise.} \end{cases} \quad (160)$$

Transformer variants use a fixed learning rate.

Perturbed Top-K Temperature Annealing (SigmaAnnealer)

For any model containing a PerturbedTopK module, the perturbation scale σ is annealed geometrically from σ_{start} to σ_{end} over the total planned number of optimizer steps $\mathcal{N}$:

$$\sigma(n) = \sigma_{\text{start}} \left(\frac{\sigma_{\text{end}}}{\sigma_{\text{start}}} \right)^{n/\mathcal{N}}, \quad n = 0, \dots, \mathcal{N}, \quad (161)$$

evaluated (and pushed into the shared `topk` module, floored at 10^{-6}) at every validation event. This is a log-linear decay: $\log \sigma(n)$ is linear in n , moving the selection operator from a soft, high-variance-but-well-conditioned regime early in training ($\sigma_{\text{start}} = 5 \times 10^{-2}$ by default) toward an almost-hard top- K regime by the end of training ($\sigma_{\text{end}} = 10^{-5}$ by default).

Early Stopping

Let $v_1, v_2, \dots$ be the sequence of validation losses observed at each validation event, and δ a minimum-improvement threshold. Maintaining a running best v^* and a stall counter κ ,

$$(v^*, \kappa) \leftarrow \begin{cases} (v_t, 0), & v_t < v^* - \delta \quad (\text{new best; checkpoint saved as "best"}) \\ (v^*, \kappa + 1), & \text{otherwise,} \end{cases} \quad (162)$$

and training halts once $\kappa \geq P$ (patience).

Checkpointing

A full training state — model/optimizer state dicts, global step, class weights, loss histories, the EarlyStopping object, and the sigma-annealer configuration — is persisted after every validation event (for resumability) and, separately, whenever a new best validation loss is found (for later evaluation), which fully decouples “latest” from “best” checkpoints.

Validation

At each validation event the model is switched to evaluation mode (dropout disabled) and the mean unweighted-average loss over the validation loader is computed under `torch.no_grad()`:

$$\bar{\mathcal{L}}_{\text{val}} = \frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{b \in \mathcal{D}_{\text{val}}} \mathcal{L}(f_{\theta}(x_b), c_b). \quad (163)$$

Model Training Operationalization

Training every deep learning model from each experimental setting is performed by executing the aforementioned steps with the following training hyperparameters:

- Maximum number of epochs ($\mathcal{E}$): 50
- Batch size (B): 2 for transformer models; 32 for MLP models
- Accumulation steps (A): 16 for transformer models; 1 for MLP models
- Validation checkpoints (by batch number): $(8k)^2$, $k = 1, 2, \dots$
- EarlyStopping patience: 10 for transformer models; 20 for MLP models
- ReduceLROnPlateau patience: 4 (for MLP models only)
- Minimum σ : 10^{-5}

The structure for the validation checkpoints is chosen since the models converge quickly, and granularizing the validation checks in the order of batches, instead of epochs, is necessary in order to distinguish training regimes for scheduling training and ascertaining convergence.

Chapter 7: Evaluation

The discussion in the following describes in full detail the processes used to evaluate every deep learning model constructed for the factorial experiment, with complete descriptions of how the dependent variables (MCC scores, drift scores, and trading simulation scores) were computed.

Notation

- $\tau \in \mathbb{R}^S$ — a per-stock decision threshold.
- TP, FP, TN, FN — true/false positive/negative counts (functions of τ).

The Matthews Correlation Coefficient Curve

Given per-stock probability scores $\hat{p} \in [0, 1]^{N \times S}$ and binary targets $y \in \{0, 1\}^{N \times S}$, and a grid of T candidate thresholds $\{\tau_1, \dots, \tau_T\} \subset [0, 1]$ (by default a uniform grid of 10^4 points on $[0, 1]$), the routine computes, *for every stock and every threshold simultaneously*, the confusion-matrix counts and the resulting Matthews correlation coefficient

$$\text{MCC}(\tau, s) = \frac{\text{TP} \cdot \text{TN} - \text{FP} \cdot \text{FN}}{\sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}}, \quad (164)$$

defined to be 0 whenever the denominator vanishes.

Vectorized computation via sorting and suffix sums. Rather than recomputing confusion matrices from scratch at each threshold ($O(NTS)$ naively), the implementation exploits monotonicity: for a fixed stock s , sort the N probabilities ascending, $\hat{p}_{(1)} \leq \dots \leq \hat{p}_{(N)}$, with targets $y_{(1)}, \dots, y_{(N)}$ permuted accordingly. Define the suffix (“at or above rank r ”) positive and negative counts,

$$P_{\geq r} = \sum_{k \geq r} y_{(k)}, \quad N_{\geq r} = \sum_{k \geq r} (1 - y_{(k)}), \quad r = 1, \dots, N, \quad P_{\geq N+1} = N_{\geq N+1} = 0, \quad (165)$$

computed once via a reversed cumulative sum. For threshold τ , let $r(\tau) = \min\{k : \hat{p}_{(k)} \geq \tau\}$ (found by vectorized binary search across all T thresholds and all S stocks at once). Then

$$\text{TP}(\tau) = P_{\geq r(\tau)}, \quad \text{FP}(\tau) = N_{\geq r(\tau)}, \quad \text{FN}(\tau) = P_{\geq 1} - \text{TP}(\tau), \quad \text{TN}(\tau) = N_{\geq 1} - \text{FP}(\tau), \quad (166)$$

reducing the whole computation to one sort, one cumulative sum, one batched binary search, and elementwise arithmetic over a dense (T, S) tensor — $O((N + T) \log N \cdot S)$ instead of $O(NTS)$.

Threshold Calibration

Static, Per-stock Calibration

Using $\text{MCC}(\tau, s)$ from (164) evaluated on the *validation* split, each stock’s decision threshold is chosen independently to maximize validation MCC:

$$\tau_s^* = \arg \max_{\tau \in \{\tau_1, \dots, \tau_T\}} \text{MCC}(\tau, s). \quad (167)$$

(For transformer models the label convention is flipped and logits reordered, $\text{softmax}(\hat{y})_{:,1} \leftrightarrow 1 - c$, to align the “positive” class consistently across model families before thresholds are fit.) $\{\tau_s^*\}_{s=1}^S$ and the calibration logits/targets are cached in the model checkpoint.

Expanding-window (Test-time-adaptive) Calibration

To allow thresholds to adapt to potential distribution shift over the test period without ever using future information, the test set is split into 10 contiguous chunks of size $\lceil N_{\text{test}}/10 \rceil$. Chunk 0 uses the purely validation-fit thresholds $\tau^{(0)} = \tau^*$. After chunk k is scored, the threshold for chunk $k + 1$ is refit by maximizing MCC over the *union* of the validation set and all test data seen so far:

$$\tau_s^{(k+1)} = \arg \max_{\tau} \text{MCC}_{\mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}[1:\text{end}_k]}(\tau, s), \quad (168)$$

where end_k is the last index of chunk k . This is a strictly causal, expanding-window (“anchored walk-forward”) recalibration scheme: predictions in chunk k are always scored using a threshold fit only on data available before that chunk began.

Testing and Metric Computation

For each split (train/test), the model is run in evaluation mode over the full loader; per-batch logits and targets are concatenated, class convention/orientation is normalized,

and probabilities are thresholded at the calibrated τ^* :

$$\hat{c}_{b,s} = \mathbb{1} \left[\text{softmax}(\hat{y}_{b,s})_1 \geq \tau_s^* \right]. \quad (169)$$

The reported metrics — computed on the fully flattened prediction/target vectors — are: accuracy, MCC (Eq. 164), precision/recall/F1 for the positive class, and precision/recall/F1 for the negative class (obtained by relabeling $c \rightarrow 1 - c$, $\hat{c} \rightarrow 1 - \hat{c}$), together with the mean weighted cross-entropy loss.

Model Drift Computation

Given the per-sample test loss $\ell \in \mathbb{R}^{N \times S}$ (cross-entropy, unreduced), an ADWIN (Adaptive Windowing) change detector is run independently per stock, streaming $\ell_{\cdot,s}$ sample-by-sample. ADWIN maintains an adaptive window whose width $w_s(n)$ shrinks whenever a statistically significant change in the mean of $\ell_{\cdot,s}$ is detected and grows otherwise, giving, for every step n , a windowed running mean

$$\bar{\ell}_s(n) = \frac{1}{w_s(n)} \sum_{k=n-w_s(n)+1}^n \ell_{k,s}. \quad (170)$$

Two normalized signals are derived and combined into a drift score:

$$\text{msd}(n, s) = |\ell_{n,s} - \bar{\ell}_s(n)|, \quad \widehat{\text{msd}}(n, s) = \frac{\text{msd}(n, s) - \min \text{msd}}{\max \text{msd} - \min \text{msd}}, \quad (171)$$

$$\widehat{w}(n, s) = \frac{w_s(n) - \min w}{\max w - \min w}, \quad \text{drift}_w(n, s) = 1 - \widehat{w}(n, s), \quad (172)$$

(min/max taken globally over the whole (n, s) grid). Per-stock summary scores average over time, $\text{msd}_s = \mathbb{E}_n[\widehat{\text{msd}}(n, s)]$, $\text{drift}_w_s = \mathbb{E}_n[\text{drift}_w(n, s)]$, and a per-stock, per-time combined drift score

$$\text{drift}(n, s) = \widehat{\text{msd}}(n, s) \cdot \text{drift}_w(n, s) \quad (173)$$

is reported alongside scalar summaries $\overline{\text{msd}}$, $\overline{\text{drift}_w}$, and their product $\overline{\text{msd}} \cdot \overline{\text{drift}_w}$.

Intuitively: large, persistent deviations of the instantaneous loss from its adaptively-windowed local mean, *co-occurring* with a detector-forced shrinkage of that adaptive window (i.e. ADWIN actively discarding old, now-stale data), jointly indicate non-stationary (drifting) predictive performance.

Statistical Analysis of Model Performance

Model configurations vary along four binary design factors, $\mathbf{f} = (\text{transformer}, \text{news}, \text{social}, \text{pred_30})$. To assess how each factor (and its interactions) affects an outcome $Y \in \{\text{mcc}, \text{drift}, \text{cum_profit}, \text{shap}\}$ recorded per (stock, model setting) pair, a linear model with the full two-way interaction expansion is fit,

$$Y = \beta_0 + \sum_{\text{stock}} \gamma_{\text{stock}} \mathbb{1}[\cdot] + \sum_k \beta_k f_k + \sum_{k < k'} \beta_{kk'} f_k f_{k'} + \varepsilon, \quad (174)$$

(stock fixed effects via $\mathbf{C}(\text{stock_id})$), fit by ordinary least squares with a *two-way cluster-robust* sandwich covariance estimator, clustering simultaneously on stock identity and on model setting (so inference is robust to arbitrary within-stock and within-setting correlation, including the fact that observations sharing a model are not independent replicates).

Marginal means. For every distinct configuration of the factors $\mathbf{f}$ present in the data, the fitted model’s prediction is averaged over the empirical distribution of any remaining (categorical) covariates — i.e. the marginal (population-averaged) predicted outcome for that configuration,

$$\hat{Y}(\mathbf{f}) = \frac{1}{|\mathcal{R}(\mathbf{f})|} \sum_{i \in \mathcal{R}(\mathbf{f})} \hat{Y}_i, \quad \mathcal{R}(\mathbf{f}) = \{i : \mathbf{f}_i = \mathbf{f}\}. \quad (175)$$

Classical Machine-Learning Baselines

Four non-deep baselines are fit on the flattened feature representation: logistic regression, linear SVM (via `LinearSVC`, whose decision scores are converted to pseudo-probabilities with the logistic sigmoid, $\hat{p} = \sigma(\text{decision_score})$, since it exposes no `predict_proba`), random forest, and gradient-boosted trees (`XGBoost`). Per-sample test cross-entropy is computed identically for all four,

$$\ell_i = -\log \hat{p}_i[c_i], \quad (176)$$

and used both for the standard classification metrics (accuracy, MCC, precision/recall/F1, both classes) under expanding-window-calibrated thresholds (reused verbatim for these

baselines) and for the same ADWIN-based drift analysis as the deep models, enabling apples-to-apples comparison across the full model suite in the downstream factorial analysis.

Trading Simulation

At each eligible timestamp t (restricted to timestamps landing exactly on multiples of the prediction horizon after a given intra-session, and excluding the session-boundary bins), and for a basket size parameter k , the model’s predicted up-probabilities $\hat{p}_t \in \mathbb{R}^S$ induce a long basket and a short basket of size $k + 1$ each:

$$\mathcal{L}_t = \arg \operatorname{topk}_{k+1}(\hat{p}_t), \quad \mathcal{S}_t = \arg \operatorname{topk}_{k+1}(-\hat{p}_t) \quad (\text{i.e. smallest } k+1). \quad (177)$$

With P_t, P_{t+1} the (correlation-aligned; see the following) close-price vectors before/after the horizon, the period return of an equal-weighted, dollar-neutral long/short portfolio is

$$r_t = \frac{1}{2(k+1)} \left(\sum_{i \in \mathcal{L}_t} \frac{P_{t+1,i}}{P_{t,i}} + \sum_{i \in \mathcal{S}_t} \frac{P_{t,i}}{P_{t+1,i}} \right), \quad (178)$$

(long legs earn the gross simple return, short legs earn its multiplicative inverse as an approximation of the short payoff), and the cumulative growth-of-\$1 curve is the running product

$$\Pi_t = \prod_{u \leq t} r_u. \quad (179)$$

This is evaluated for every $k \in \{0, \dots, 14\}$ and every valid intra-horizon, both for every trained deep-learning experiment and for the best-performing classical baseline (selected as the configuration maximizing marginal-mean cum_profit from a factorial fit, exactly analogous to how the best MCC/drift configuration is chosen elsewhere).

Chapter 8: Results and Discussion

Table 6 summarizes the result of the entire experiment, for all deep learning models in each experimental condition. The specification of an experimental setting is by inclusion of each independent variable’s keyword within the name, or by the value of such variable (“30” means that the model predicts 30-minute returns, whereas “10” corresponds to 10-minute return models).

Table 6

Model Performance and Drift Descriptives

Model	Test MCC	Train MCC	Drift Score
Stock News Social MLP 30	0.070	0.136	0.073
Stock News MLP 10	0.133	0.204	0.038
Stock News Social MLP 10	0.107	0.202	0.038
Stock News MLP 30	0.087	0.133	0.047
Stock MLP 30	0.096	0.139	0.064
Stock MLP 10	0.133	0.195	0.062
Stock News Social Transformer 30	0.059	0.141	0.045
Stock News Transformer 10	0.128	0.205	0.041
Stock News Social Transformer 10	0.118	0.197	0.049
Stock Social MLP 30	0.056	0.171	0.048
Stock News Transformer 30	0.036	0.153	0.063
Stock Social Transformer 30	0.062	0.143	0.036
Stock Social Transformer 10	0.121	0.214	0.041
Stock Social MLP 10	0.116	0.201	0.051
Stock Transformer 30	0.049	0.146	0.038
Stock Transformer 10	0.101	0.201	0.061

Across all sixteen model configurations, train MCC consistently exceeds test MCC (rolling), often by a factor of two or more. For example, Stock News MLP 10 achieves a train MCC of 0.204 but a rolling test MCC of only 0.133, and Stock Social Transformer 10 shows an even larger gap (0.214 versus 0.121). This pattern holds across every model in the table, indicating a systematic drop-off in predictive performance when moving from training to held-out rolling evaluation, consistent with some degree of overfitting or non-stationarity in the underlying data.

A clear horizon effect is also visible: models predicting 10 minutes ahead almost always outperform their 30-minute counterparts on rolling test MCC. Stock MLP 10 (0.133) and Stock News MLP 10 (0.133) post the highest rolling test MCC values in the entire table, while their 30-minute counterparts, Stock MLP 30 (0.096) and Stock News MLP 30 (0.087), lag well behind. The same pattern appears for transformer-based models: Stock News Transformer 10 (0.128) and Stock Social Transformer 10 (0.121) substantially outperform Stock News Transformer 30 (0.036) and Stock Social Transformer 30 (0.062), with Stock News Transformer 30 recording the single lowest test MCC of any model.

Comparing architectures, MLP models tend to edge out transformer models on rolling test MCC at the 10-minute prediction horizon (e.g., Stock MLP 10 at 0.133 versus Stock Transformer 10 at 0.101), though the gap narrows or reverses at the 30-minute horizon and once social or news features are added, where several transformer variants (e.g., Stock Social Transformer 10 at 0.121) perform competitively or better than their MLP counterparts. This suggests that architecture choice interacts with both prediction horizon and input features rather than uniformly favoring one architecture.

Drift scores are generally higher for the MLP-based and 30-minute models. Stock News Social MLP 30 has the highest combined drift score (0.073), followed by Stock MLP 30 (0.064) and Stock News Transformer 30 (0.063), while Stock Social Transformer 30 (0.036), Stock News MLP 10 (0.038), and Stock Transformer 30 (0.038) show the lowest drift. There is some indication that higher drift scores co-occur with weaker rolling test

performance among the 30-day models (e.g., Stock News Social MLP 30 combines the highest drift with a below-median test MCC).

MCC Model Performance

Table 7 showcases the coefficients of an OLS regression model fitted on MCC scores per stock for every experimental setting. The OLS regression model predicts MCC (Matthew’s Correlation Coefficient) per stock as a function of four categorical predictors—*transformer*, *social*, and *news* and *pred_30*—and all pairwise interactions among these variables.

Table 7
Regression Results (DV: MCC)

Variable	Coef.	Std. Err.	<i>t</i>	<i>p</i> -value	95% CI
Intercept	0.135	0.005	29.73	< 0.001***	[0.127, 0.144]
pred_30	−0.036	0.008	−4.43	< 0.001***	[−0.052, −0.020]
transformer × social	0.034	0.006	5.40	< 0.001***	[0.021, 0.046]
transformer × pred_30	−0.027	0.008	−3.42	0.001***	[−0.042, −0.011]
social	−0.020	0.006	−3.14	0.002**	[−0.033, −0.008]
transformer	−0.019	0.007	−2.66	0.008**	[−0.033, −0.005]
news × pred_30	−0.014	0.007	−2.14	0.033*	[−0.027, −0.001]
news	0.005	0.005	1.09	0.276	[−0.004, 0.015]
social × pred_30	−0.004	0.007	−0.56	0.577	[−0.018, 0.010]
news × social	−0.001	0.007	−0.12	0.902	[−0.014, 0.013]
transformer × news	0.000	0.007	0.06	0.950	[−0.013, 0.014]

Note: **p* < 0.05; ***p* < 0.01; ****p* < 0.001.

The intercept is 0.135 (SE = 0.005, *p* < .001), representing the baseline MCC—that is, the expected MCC when all predictors are at their reference level. This indicates a small positive baseline correlation, reflecting modest predictive performance in the reference

condition.

pred_30 (coef = -0.036 , $p < .001$): The largest main effect in the model. Holding other variables constant, **pred_30** is associated with a meaningful *decrease* in MCC of approximately 0.036, suggesting this condition tends to hurt model performance relative to baseline.

social (coef = -0.020 , $p = .002$): The social condition is associated with a statistically significant decrease in MCC of about 0.020 relative to baseline, holding other factors constant.

transformer (coef = -0.019 , $p = .008$): The transformer condition is associated with a small but significant decrease in MCC (≈ 0.019) relative to baseline, holding other variables fixed.

news (coef = 0.005 , $p = .276$): The news main effect is small, positive, and not statistically significant—its effect cannot be distinguished from zero.

Interaction terms in the following indicate whether the effect of one variable depends on the level of another—that is, whether combining two conditions produces something different from the sum of their individual effects.

transformer $\times$ **social** (coef = $+0.034$, $p < .001$): The largest and most significant interaction. Although both *transformer* and *social* have negative main effects individually, their combination yields an additional positive boost of 0.034, partially offsetting their negative main effects:

$$0.135 - 0.019 - 0.020 + 0.034 = 0.130 \quad (\text{near baseline})$$

transformer $\times$ **pred_30** (coef = -0.027 , $p = .001$): This negative interaction indicates that the already-negative effect of *pred_30* is compounded when combined with *transformer*, rather than offset:

$$0.135 - 0.019 - 0.036 - 0.027 = 0.053 \quad (\text{well below baseline})$$

news $\times$ **pred_30** (coef = -0.014 , $p = .033$): A smaller, marginally significant negative interaction—the negative effect of *pred_30* is slightly amplified in the presence of *news*.

social $\times$ **pred_30**, **news** $\times$ **social**, **transformer** $\times$ **news** (all $p > .5$): None of these interactions are statistically distinguishable from zero, indicating that these variable pairs do not meaningfully interact; their joint effects are well approximated by summing the corresponding main effects.

Overall, the following is a summary of the main and interaction effects:

1. *pred_30* is the most consistently harmful factor, both alone and combined with *transformer*—this pairing yields the lowest predicted MCC in the model (≈ 0.053).
2. *transformer* and *social* individually hurt performance but help in combination. This is a case where main effects cannot be interpreted in isolation: the interaction reverses the naive expectation that pairing two “negative” conditions would compound the harm.
3. News is essentially inert, with neither its main effect nor most of its interactions (excluding *pred_30*) reaching statistical significance.
4. Effect sizes are modest in absolute terms (all coefficients < 0.04 relative to a 0.135 baseline), though precisely estimated given the small standard errors. Statistical significance here reflects estimation *precision*, not necessarily practical importance; effect magnitudes should be contextualized against the overall variance of MCC in the data.

Table 8 reveals a comparison between the best deep learning model and the machine learning baselines according to MCC scores.

The best deep learning model (Stock News MLP 10) achieves the highest mean MCC (0.137) across the sample, though it is nearly indistinguishable from logistic regression, which posts an almost identical mean MCC of 0.137. This suggests that, on average, the

Table 8*Mean MCC of Best Deep Learning Model vs. Machine Learning Baselines*

Model	Mean MCC
Stock News MLP 10	0.137
Logistic Regression	0.137
Linear SVC	0.108
Random Forest	0.112
XGBoost	0.133

additional complexity of the deep learning architecture does not yield a meaningful performance advantage over a simple linear baseline in this setting. XGBoost follows closely behind with a mean MCC of 0.133, indicating that a well-tuned gradient-boosted tree ensemble remains competitive with both the deep learning model and logistic regression.

Random forest (0.112) and linear SVC (0.108) trail the other three models by a more noticeable margin, each falling roughly 0.02–0.03 below the top-performing models. This gap suggests that these two approaches are comparatively less well-suited to the structure of the underlying prediction task, though they still achieve positive mean MCC values, indicating some degree of predictive signal above chance. Table 9 supports these results with an OLS regression fitted on MCC scores per ML baseline and the best deep learning architecture.

The intercept of 0.161 represents the marginal mean MCC for the best deep learning model, the omitted reference category, and is highly significant ($p < .001$). Each of the four baseline coefficients therefore reflects the difference in mean MCC between that baseline and the deep learning model, holding all else constant.

Linear SVC and random forest both show statistically significant negative coefficients, indicating that these two baselines perform reliably worse than the deep learning model. Linear SVC trails by an average of 0.029 MCC points ($t = -5.44$, $p < .001$), and random

Table 9*Regression Results: MCC by Model Type (Reference: Stock MLP News 10)*

Model	Coef.	Std. Err.	t	p -value	95% CI
Intercept	0.161	0.003	49.98	$< 0.001^{***}$	[0.155, 0.168]
Linear SVC	-0.029	0.005	-5.44	$< 0.001^{***}$	[-0.039, -0.018]
Random Forest	-0.025	0.006	-4.54	$< 0.001^{***}$	[-0.036, -0.014]
XGBoost	-0.004	0.006	-0.75	0.455	[-0.016, 0.007]
Logistic Regression	-0.000	0.005	-0.02	0.986	[-0.010, 0.009]

Note: $*p < 0.05$; $**p < 0.01$; $***p < 0.001$. Deep learning is the omitted reference category.

forest trails by 0.025 points ($t = -4.54$, $p < .001$), with confidence intervals for both comfortably excluding zero. This confirms the pattern suggested by the earlier descriptive comparison: these two approaches underperform the deep learning model by a small but consistent and statistically reliable margin.

In contrast, XGBoost and logistic regression show coefficients that are not statistically distinguishable from zero. XGBoost's mean MCC is only 0.004 points lower than the deep learning model's, and this difference is far from significant ($t = -0.75$, $p = .455$), with a confidence interval that spans zero. Logistic regression's coefficient is essentially null (-0.00008 , $t = -0.02$, $p = .986$), confirming that its performance is statistically indistinguishable from the deep learning model.

Taken together, these results formalize the pattern seen in the descriptive statistics: the deep learning model does not significantly outperform logistic regression or XGBoost, but it does significantly outperform linear SVC and random forest. This indicates that the practical benefit of using a deep learning approach over simpler baselines depends heavily on which baseline is being considered, with tree-ensemble and linear-classifier baselines like XGBoost and logistic regression offering statistically comparable performance, while

margin-based and bagged-tree methods like linear SVC and random forest fall meaningfully short.

Model Drift

Table 10 shows a summary of an OLS regression model fitted on the drift scores of every stock for each experimental setting.

Table 10

Regression Results (DV: Drift Score)

Variable	Coef.	Std. Err.	<i>t</i>	<i>p</i> -value	95% CI
Intercept	0.061	0.002	30.16	< 0.001***	[0.057, 0.065]
news	−0.027	0.005	−5.66	< 0.001***	[−0.036, −0.018]
news × pred_30	0.022	0.006	3.69	< 0.001***	[0.011, 0.034]
news × social	0.016	0.006	2.71	0.007**	[0.005, 0.028]
transformer × pred_30	−0.013	0.006	−2.17	0.030*	[−0.025, −0.001]
transformer × news	0.013	0.006	2.14	0.032*	[0.001, 0.025]
social	−0.010	0.004	−2.42	0.016*	[−0.019, −0.002]
transformer × social	−0.008	0.006	−1.28	0.199	[−0.020, 0.004]
social × pred_30	0.004	0.006	0.57	0.567	[−0.009, 0.016]
pred_30	−0.002	0.005	−0.47	0.640	[−0.012, 0.007]
transformer	−0.002	0.004	−0.46	0.648	[−0.010, 0.006]

Note: * $p < 0.05$; ** $p < 0.01$; *** $p < 0.001$.

The intercept of 0.061 represents the baseline combined drift score when all predictors are at their reference level, and is highly significant ($p < .001$). Unlike the MCC model, where *transformer* and *social* were the strongest standalone drivers, the drift score model is dominated by the *news* predictor, both as a main effect and through its interactions.

News shows the largest main effect in the model, with a significant negative coefficient

of -0.027 ($t = -5.66$, $p < .001$), indicating that the presence of news data is associated with a substantial reduction in drift relative to baseline, holding other variables constant. However, this negative main effect is qualified by two significant positive interactions: $news \times pred_30$ (0.022 , $p < .001$) and $news \times social$ (0.016 , $p = .007$). This pattern indicates that news reduces drift substantially on its own, but this drift-reducing benefit is partially eroded when news is combined with either the $pred_30$ condition or the social condition—the combined effect in either case is only mildly negative rather than strongly negative (e.g., news alone: -0.027 ; news with $pred_30$: $-0.027 + 0.022 = -0.005$).

Social also shows a significant negative main effect (-0.010 , $p = .016$), suggesting a modest independent drift-reducing effect, though considerably smaller than that of news. Its interaction with news is positive and significant, as noted above, while its interaction with transformer is negative but not significant (-0.008 , $p = .199$), and its interaction with $pred_30$ is negligible and non-significant (0.004 , $p = .567$).

The *transformer* predictor shows no significant main effect on drift (-0.002 , $p = .648$), in contrast to its significant negative effect on MCC in the earlier model. However, transformer does participate in two significant interactions: with $pred_30$ (-0.013 , $p = .030$), where the combination modestly increases drift reduction, and with *news* (0.013 , $p = .032$), where the combination works in the opposite direction, partially counteracting news’s drift-reducing effect. This mirrors the offsetting-interaction pattern seen in the MCC model, where combined conditions did not simply sum their individual effects.

Finally, $pred_30$ alone shows no significant main effect on drift (-0.002 , $p = .640$), unlike in the MCC model where it was the strongest predictor. This suggests that $pred_30$ ’s impact on drift is conditional—it only becomes relevant in combination with news or transformer—whereas its impact on MCC was direct and substantial regardless of other conditions.

Overall, these results suggest that *news* is the primary lever for reducing combined drift, but its benefit is context-dependent: it is strongest in isolation and weakens when

paired with pred_30 or social. *Social* contributes a smaller independent drift-reducing effect, while *transformer* and *pred_30* influence drift only through their interactions rather than as standalone predictors, a notably different pattern from what was observed in the MCC regression, where transformer and pred_30 were among the strongest main effects.

Table 11 reveals a summary of the drift scores for each machine learning model trained with the same data modalities as the deep learning model with the lowest drift.

Table 11
Mean Combined Drift Score by Model Type: Best Deep Learning Model vs. Machine Learning Baselines

Model	Mean Drift Score
Stock Social MLP 30	0.036
Logistic Regression	0.070
Linear SVC	0.033
Random Forest	0.024
XGBoost	0.096

The best deep learning model (model with the lowest drift—Stock Social MLP 30) shows a mean drift score of 0.036, placing it toward the lower end of the five models but not the lowest. Random forest achieves the lowest mean drift score overall (0.024), followed closely by linear SVC (0.033) and then the deep learning model (0.036). This ordering suggests that, contrary to the framing of Stock Social MLP 30 as the “least drift” deep learning variant, both random forest and linear SVC actually exhibit less drift on average within this comparison set.

Logistic regression (0.070) and, in particular, XGBoost (0.096) show substantially higher mean drift scores than the other three models. XGBoost’s mean drift score is roughly 2.7 times that of Stock Social MLP 30 and nearly 4 times that of random forest,

making it the clear outlier in terms of drift instability. This pattern is notable given that XGBoost and logistic regression were among the strongest performers in the earlier MCC comparison, suggesting a potential trade-off between predictive performance and drift stability for these two baselines. Table 12 shows these results further with OLS regression coefficients for each machine learning model with the reference best deep learning model.

Table 12

Regression Results: Combined Drift Score by Model Type (Reference: Stock Social MLP 30)

Model	Coef.	Std. Err.	<i>t</i>	<i>p</i> -value	95% CI
Intercept	0.036	0.001	67.42	< 0.001***	[0.035, 0.037]
XGBoost	0.060	0.002	24.29	< 0.001***	[0.055, 0.065]
Logistic Regression	0.033	0.001	43.66	< 0.001***	[0.032, 0.035]
Random Forest	−0.012	0.001	−9.42	< 0.001***	[−0.014, −0.009]
Linear SVC	−0.003	0.001	−5.90	< 0.001***	[−0.005, −0.002]

Note: * $p < 0.05$; ** $p < 0.01$; *** $p < 0.001$. Deep learning is the omitted reference category.

The intercept of 0.036 represents the mean drift score for Stock Social MLP 30, the omitted reference category, and is highly significant ($p < .001$), consistent with the descriptive mean reported above. Unlike the MCC baseline regression, where two of the four baseline coefficients were non-significant, every baseline model here shows a statistically significant difference from Stock Social MLP 30, indicating that drift behavior differs reliably across all five model types.

XGBoost and logistic regression both show large, significant, positive coefficients, confirming that these two baselines exhibit substantially more drift than Stock Social MLP 30. XGBoost’s coefficient of 0.060 ($t = 24.29$, $p < .001$) is the largest in the model by a wide margin, indicating that XGBoost’s mean drift score exceeds that of Stock Social MLP 30 by an average of 0.060 points, consistent with the descriptive comparison showing

XGBoost as the clear outlier. Logistic regression’s coefficient of 0.033 ($t = 43.66$, $p < .001$) is smaller in magnitude but still substantial and highly precisely estimated, given its very small standard error.

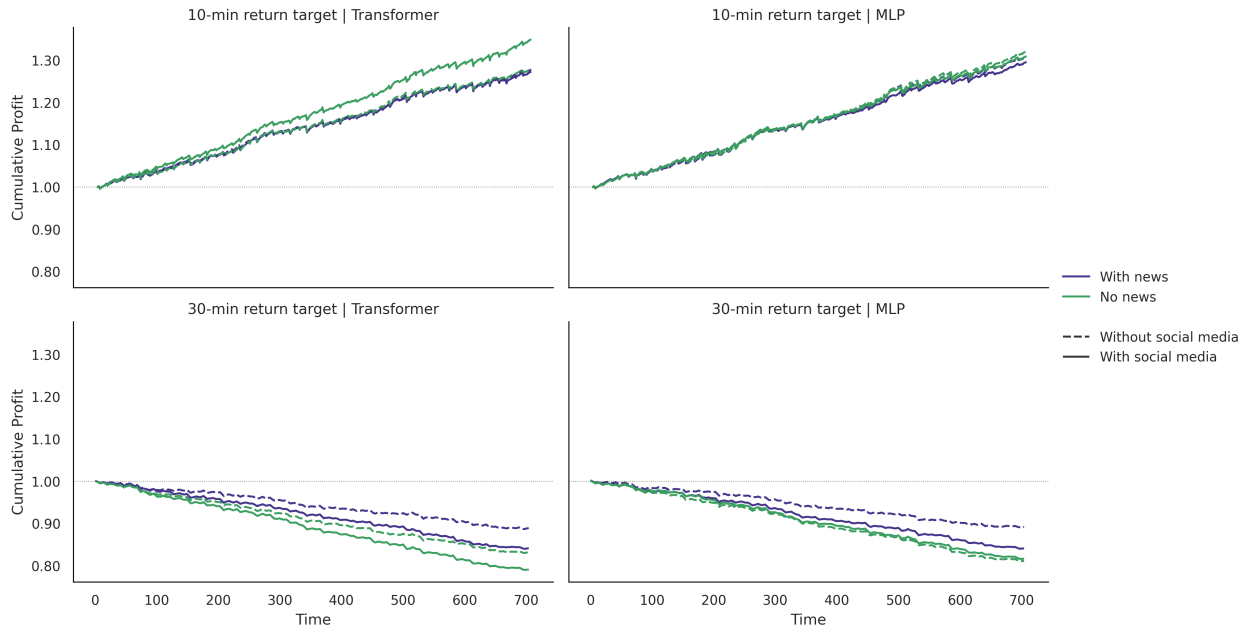
In contrast, random forest and linear SVC both show significant negative coefficients, indicating that these two baselines exhibit reliably less drift than Stock Social MLP 30. Random forest’s coefficient of -0.012 ($t = -9.42$, $p < .001$) is the larger of the two negative effects, while linear SVC’s coefficient of -0.003 ($t = -5.90$, $p < .001$), though smaller in magnitude, is still highly significant given the precision of the estimate.

Taken together, these results indicate a clear ordering of drift behavior across model types: XGBoost and logistic regression drift substantially more than the best deep learning model, while random forest and linear SVC drift somewhat less. This is the inverse of the pattern seen in the MCC baseline regression, where XGBoost and logistic regression were statistically indistinguishable from the best deep learning model in performance, while linear SVC and random forest performed significantly worse. Considered jointly, the MCC and drift results suggest that XGBoost and logistic regression achieve competitive predictive performance at the cost of higher drift, whereas random forest and linear SVC sacrifice some predictive performance in exchange for greater stability, and the deep learning model occupies a middle ground on both dimensions.

Trading Simulation Model Performance

Figure 18 presents cumulative trading profit over time across four panels, cross-cutting model architecture (Transformer vs. MLP) and prediction horizon (10-minute vs. 30-minute return target), with each panel further disaggregating results by the presence of news (line color) and social media (line style) features.

The most striking pattern is the divergence between the two horizons: at the 10-minute horizon (top row), all model and feature configurations produce steadily increasing cumulative profit, ending between roughly 1.27 and 1.35 times the initial value, whereas at the 30-minute horizon (bottom row), all configurations produce steadily

Figure 18*Cumulative Trading Profit over Time per Experimental Setting*

declining cumulative profit, ending between roughly 0.80 and 0.89. This indicates that the 30-minute-ahead prediction target is fundamentally unprofitable as a trading signal in this simulation, regardless of architecture or auxiliary features, consistent with the earlier regression finding that `pred_30` was associated with significantly lower MCC.

Within the 10-minute horizon, the Transformer model (top left) shows a clearer separation between feature configurations than the MLP model (top right): the “no news” condition (green) outperforms the “with news” condition (purple) by a small but visible margin, while social media inclusion (solid vs. dashed lines) makes little difference within each news condition. The MLP model at the 10-minute horizon, by contrast, shows all four line configurations nearly overlapping, suggesting that MLP trading performance at this horizon is largely insensitive to both news and social media inputs.

Within the 30-minute horizon, the pattern reverses in terms of which configuration performs best: the “with news” condition (purple) now clearly outperforms the “no news”

condition (green) in both the Transformer and MLP panels, with the gap widening over the course of the simulation. The “without social media” dashed lines also tend to sit above the “with social media” solid lines within each news condition at this horizon, particularly toward the end of the simulation window, suggesting that excluding social media data is mildly beneficial for the 30-minute target. Overall, the figure suggests that the value of news and social media inputs is horizon-dependent: they appear mildly detrimental to profit at the 10-minute horizon but mildly protective (i.e., reducing losses) at the 30-minute horizon.

To support these findings further, Table 13 showcases the OLS regression coefficients of every experimental setting’s cumulative profit scores.

Table 13

Regression Results: Trading Simulation Cumulative Profit

Variable	Coef.	Std. Err.	<i>t</i>	<i>p</i> -value	95% CI
Intercept	1.407	0.021	67.66	< 0.001***	[1.366, 1.447]
pred_30	−0.487	0.019	−25.98	< 0.001***	[−0.524, −0.450]
news × pred_30	0.080	0.010	7.75	< 0.001***	[0.060, 0.100]
social × pred_30	−0.045	0.010	−4.50	< 0.001***	[−0.064, −0.025]
news × social	−0.034	0.010	−3.37	0.001***	[−0.054, −0.014]
social	0.021	0.013	1.61	0.107	[−0.004, 0.046]
transformer	−0.013	0.011	−1.18	0.238	[−0.036, 0.009]
transformer × pred_30	0.013	0.010	1.31	0.189	[−0.006, 0.033]
transformer × news	−0.013	0.010	−1.32	0.188	[−0.033, 0.006]
transformer × social	0.013	0.010	1.29	0.198	[−0.007, 0.032]
news	−0.003	0.008	−0.32	0.751	[−0.018, 0.013]

Note: * $p < 0.05$; ** $p < 0.01$; *** $p < 0.001$.

The intercept of 1.407 represents the baseline cumulative profit when all predictors are at their reference level, and is highly significant ($p < .001$). By far the largest and most significant effect in the model is *pred_30*, with a coefficient of -0.487 ($t = -25.98$, $p < .001$), confirming what is visually apparent in Figure 18: the 30-minute prediction horizon is associated with a dramatic reduction in cumulative trading profit relative to the 10-minute horizon, holding other factors constant. This is by far the dominant driver of cumulative profit in the simulation, dwarfing every other coefficient in the model by an order of magnitude.

Three interaction terms involving *pred_30* are also significant and provide insight into the horizon-dependent patterns observed in the figure. The *news* $\times$ *pred_30* interaction is positive and highly significant (0.080 , $p < .001$), indicating that news partially offsets the severe profit penalty associated with the 30-minute horizon—consistent with the figure showing the “with news” lines sitting above the “no news” lines in the bottom row. Conversely, the *social* $\times$ *pred_30* interaction is negative and significant (-0.045 , $p < .001$), indicating that social media inclusion *worsens* the 30-minute horizon penalty rather than mitigating it, consistent with the “without social media” dashed lines outperforming the “with social media” solid lines in the bottom-row panels.

The *news* $\times$ *social* interaction is also significant and negative (-0.034 , $p = .001$), suggesting that combining both news and social media inputs together yields lower profit than would be expected from summing their individual effects—these two feature sources appear to work against each other rather than compounding positively when used jointly.

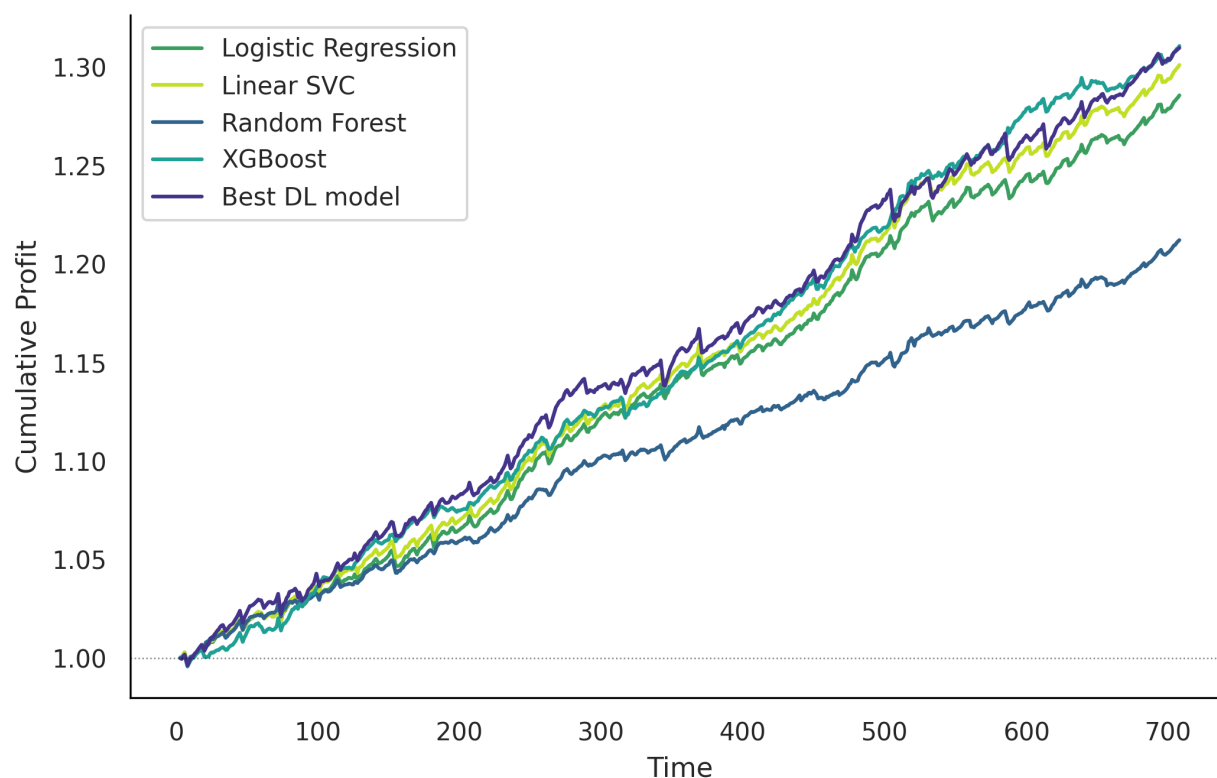
None of the remaining terms reach statistical significance at conventional thresholds, including the main effects of *social* (0.021 , $p = .107$), *transformer* (-0.013 , $p = .238$), and *news* (-0.003 , $p = .751$), nor the *transformer* interactions with *pred_30*, *news*, or *social* (all $p > .18$). This indicates that, unlike prediction horizon and the horizon-dependent feature interactions, model architecture (Transformer vs. MLP) does not have a statistically detectable independent effect on cumulative trading profit in this simulation, nor does it

significantly interact with the other predictors—consistent with the relatively similar overall shapes of the Transformer and MLP panels within each horizon in Figure 18.

On one hand, Figure 19 compares cumulative trading profit over time for the best deep learning model (Stock Social MLP 10) against four machine learning baselines.

Figure 19

Cumulative Trading Profit over Time for The Best DL Model Compared against ML Baselines



All five models produce positive cumulative profit over the simulation window, but they separate into two distinct clusters. The best deep learning model, XGBoost, linear SVC, and logistic regression track closely together throughout the simulation, ending in the range of roughly 1.28 to 1.31, with the deep learning model holding a slight and fairly consistent lead over this cluster from around time 100 onward. Random forest, by contrast, diverges sharply from the other four models beginning around time 100 and remains the

clear laggard for the remainder of the simulation, ending at approximately 1.21—noticeably below the rest of the cluster.

Within the top cluster, the ordering among the deep learning model, XGBoost, linear SVC, and logistic regression is relatively stable but close, with XGBoost briefly tracking near or even above the deep learning model between roughly time 550 and 650 before the deep learning model regains the lead toward the end of the series. Logistic regression consistently sits at or near the bottom of this top cluster, though it remains far closer to the other three models than to random forest. This visual pattern suggests that random forest is the clear outlier among the baselines in terms of trading profitability, while the deep learning model’s advantage over the remaining baselines, though present, is comparatively modest. These results are supported further by Table 14, which shows that the machine learning models exhibit lower scores than the best deep learning model, only some of which although are statistically significant.

Table 14

Regression Results: Trading Simulation Cumulative Profit, Best DL Model vs. ML Baselines

Model	Coef.	Std. Err.	<i>t</i>	<i>p</i> -value	95% CI
Intercept	1.405	0.005	299.20	< 0.001***	[1.396, 1.415]
Random Forest	−0.098	0.008	−11.62	< 0.001***	[−0.114, −0.081]
Logistic Regression	−0.024	0.006	−4.26	< 0.001***	[−0.035, −0.013]
Linear SVC	−0.009	0.006	−1.47	0.140	[−0.020, 0.003]
XGBoost	0.001	0.007	0.14	0.885	[−0.013, 0.015]

Note: * $p < 0.05$; ** $p < 0.01$; *** $p < 0.001$. Best DL model is the omitted reference category.

The intercept of 1.405 represents the mean cumulative profit for the best deep learning model, the omitted reference category, and is highly significant ($p < .001$), consistent with

its position at or near the top of the cluster in Figure 19. Each baseline coefficient reflects the average difference in cumulative profit between that baseline and the deep learning model.

Random forest shows by far the largest and most significant negative coefficient (-0.098 , $t = -11.62$, $p < .001$), confirming that it underperforms the deep learning model substantially and consistently, matching its clear visual separation from the other four series in the figure. Logistic regression also shows a significant, though considerably smaller, negative coefficient (-0.024 , $t = -4.26$, $p < .001$), indicating that it too underperforms the deep learning model on average, consistent with its position at the lower edge of the top cluster in the figure.

In contrast, linear SVC and XGBoost are not statistically distinguishable from the deep learning model. Linear SVC's coefficient of -0.009 ($t = -1.47$, $p = .140$) is negative but does not reach significance, and its confidence interval spans zero. XGBoost's coefficient is essentially null (0.001 , $t = 0.14$, $p = .885$), indicating that its performance is statistically indistinguishable from the deep learning model, consistent with the two series tracking each other closely and even briefly crossing in the figure.

Taken together, these results indicate a two-tier structure among the baselines: random forest and, to a lesser extent, logistic regression underperform the deep learning model by a statistically reliable margin, while linear SVC and XGBoost achieve trading performance that is statistically comparable to the deep learning model. This mirrors the earlier MCC baseline regression, where XGBoost and logistic regression were statistically indistinguishable from the deep learning model in classification performance; here, however, it is linear SVC and XGBoost that emerge as the closest competitors in terms of realized trading profit, suggesting that MCC and trading-simulation profit do not rank the baselines in exactly the same order.

Chapter 9: Conclusions

This study examined the performance, robustness, and economic value of deep learning models for short-horizon stock return prediction, benchmarking them against standard machine learning baselines across three complementary analyses: predictive performance (MCC), model drift, and trading simulation profitability, both across deep learning configurations and against baseline models.

Several consistent themes emerge from these analyses. First, prediction horizon is the single most important determinant of both predictive and economic performance. Across every analysis in this study, the 30-minute return target underperformed the 10-minute target by a wide margin: in the MCC regression, `pred_30` was the strongest main effect and, in combination with the transformer architecture, produced the lowest predicted MCC of any configuration; in the trading simulation, `pred_30` was by far the dominant predictor of cumulative profit, with 30-minute models losing profits consistently while 10-minute models generated steady gains regardless of architecture or auxiliary features. This convergence across two different outcome measures—a statistical classification metric and a realized trading outcome—provides strong evidence that the 30-minute horizon is fundamentally harder to predict with the available features, rather than an artifact of any single modeling choice.

Second, the value of auxiliary data sources (news and social media) is highly context-dependent rather than uniformly beneficial or harmful. News showed no significant standalone effect on MCC but was the dominant driver of reduced drift, while its interaction with `pred_30` significantly improved both MCC and trading profit at the 30-minute horizon—suggesting that news data becomes more valuable precisely in the harder, longer-horizon setting where models otherwise struggle. Social media told a more mixed story: it had a significant negative main effect on MCC and drift individually, but combined positively with the transformer architecture to substantially offset both of their negative main effects on MCC, while simultaneously worsening the `pred_30` profit penalty

in the trading simulation. Taken together, these results caution against treating any single feature source as categorically “helpful” or “harmful”; its effect depends on the architecture, horizon, and outcome measure under consideration.

Third, architecture choice (transformer vs. MLP) mattered less as a standalone factor than through its interactions. Transformer had a significant negative main effect on MCC but no significant effect on drift or trading profit, and its interactions with other features were more informative than its main effect alone—most notably the strong positive $\text{transformer} \times \text{social interaction}$ on MCC, which reversed the expectation that combining two individually negative conditions would compound the harm. This underscores that model architecture should not be evaluated in isolation from the feature configuration it is paired with.

Fourth, comparisons against machine learning baselines revealed that the deep learning models’ advantage over simpler methods is real but narrower and less consistent than might be expected. On MCC, the best deep learning model was statistically indistinguishable from logistic regression and XGBoost, and significantly outperformed only linear SVC and random forest. In the trading simulation, this pattern shifted: linear SVC and XGBoost were the baselines statistically indistinguishable from the deep learning model, while logistic regression joined random forest as a significant underperformer. This inconsistency in which baselines are “competitive” across metrics suggests that MCC and realized trading profit, while related, are not interchangeable measures of model quality, and that baseline comparisons should be interpreted relative to the specific outcome of interest rather than assumed to generalize across metrics.

Finally, the drift analysis revealed an important performance-stability trade-off among baselines: XGBoost and logistic regression, which achieved MCC statistically comparable to the deep learning model, also exhibited substantially higher drift, while random forest and linear SVC, which underperformed on MCC, were more stable. The best deep learning model occupied a middle position on both dimensions. This suggests that model selection

in this domain should not rest on predictive accuracy alone; stability under distributional shift is a distinct and practically important consideration, particularly for models intended for extended deployment in non-stationary financial markets.

Taken as a whole, these findings suggest that no single model, architecture, or feature configuration dominates uniformly across performance, drift, and profitability. The clearest and most robust conclusion is that prediction horizon fundamentally constrains what is achievable, and that the value of additional architectural complexity or auxiliary data sources is realized unevenly and often only in interaction with other design choices. Future work could usefully extend this analysis by examining a wider range of horizons to determine whether the sharp 10-minute/30-minute divide observed here reflects a genuine discontinuity in predictability or a smoother degradation that these two horizons happen to bracket, and by incorporating transaction costs and slippage into the trading simulation to assess whether the profit differences observed here would survive more realistic execution assumptions.

Chapter 10: Recommendations

Recommendations for Future Research

In terms of the overall planning and implementation, several areas of the study are afflicted by certain conceptual limitations, data availability limitations, architectural caveats, as well as budgetary limitations on time and compute. Describing these limitations in the order of background concepts, data collection, preprocessing, modeling, and evaluation steps, a comprehensive picture of the current study's limitations are summarized in the following.

Despite the objectives of this study to capture general insights regarding design principles on Philippine stocks, it is limited in breadth; a number of many other potentially interesting effects exists not only outside the scope of this study, but within the scope as well. Conceptually speaking, for example, stocks have been seen to react to earnings calls and company announcements, which can be a form of *news data*. This is the entire basis of the field of study known as *fundamental analysis*, of which this study barely scratches the surface through the news and social media. While news and social media may be forms of fundamental data to a limited extent, the field of fundamental analysis deals mostly with these other well-established sources of fundamental information, and the current study has made no inclusion of such sources due to data availability and feasibility. Conceptually speaking, the study also foregoes many other interesting international financial markets, such as commodity markets for silver, coffee, other kinds of oil, and other exchange rates. This makes the study potentially prone to bias towards the chosen avenues for oil, copper, foreign exchange rates, and other specific chosen representative market of a commodity, bond, etc.; however, these selections are carefully chosen such that they are *representative standards* of their modality relative to the Philippine context—i.e. COMEX for copper, ICE Brent crude, and PHP- relative XAU and USD prices. All in all, these limitations constrain the claims made in this study to be under the assumption of *journalistic news data sources* as the news modality, and to some extent, *standard commodity and exchange*

financial markets as the exogenous commodity/exchange markets within the technical data. In response to this, future studies are encouraged to study fundamental data as a separate modality, and create reasonable architectural designs to process such information.

The data range for training and testing is also one of the most significant limitations to the study, where only one year’s worth of data (from March 2025 to April 2026) encompass all of the training, validation, and testing periods. This limited data may constrain deep learning models to small sample sizes required for effective learning, albeit the current dataset sizes may already offer sufficient training samples given the model parameter sizes. The dataset size for news data is afflicted the most, given the discussion on recovering news timestamps due to inaccuracies from the LSEG News Monitor acting as a filter due to timestamp unavailability from source news article pages, as well as a potential generator of inaccurate timestamps, although such cases may be few. While the dataset size for social media data is large, it may be prone to types of content that are not well-defined—the social media dataset is a mixture of investor sentiment, customer sentiment, news outlets posting on X, and the like, which makes asserting claims about investor sentiment using the results on social media a questionable leap. Given these constraints, the results are to be interpreted with news data as a less statistically stable modality than social media, whereas social media as a modality is to be interpreted as general X posts appearing in Philippine financial domain search queries, as this is how the social media data was extracted from the Apify scraper. Future studies are encouraged to look at data from a broader temporal scope, potentially crossing multiple years in order to capture a greater number of market regimes that can offer additional model learning benefits.

Certain architectural limitations also constrain some of the assertions. While the discussion often uses the terms “transformer” and “MLP” to describe the two modes of the independent variable *overall pipeline complexity*, this variable encompasses more modifications than just a simple change between a transformer architecture and an MLP. Precisely, it also indicates a change between, corresponding to settings with a transformer

model and settings with an MLP model respectively, whether the text datasets enter the model as full semantic embeddings or only derived sentiment indicators, whether stock inter-correlations are accounted for in the model architecture, whether window sequences of time-series feature vectors for the technical data modality or only a single vector of the current timestamp’s features enter the model, and whether dynamic filtering is used for textual datasets, or not. These may be seen as confounding factors, but the current study argues that there is a qualitative reframing of this independent variable that is reasonable, where confounding disappears. The qualitative factor that best describes this independent variable is reasonable pipeline design corresponding to a chosen deep learning architecture (transformer or MLP). Namely, if a researcher were to design a deep learning model given the data available in this study, certainly, the preprocessing steps and model design must conform with the input shapes required by the chosen deep learning architecture; for instance, since transformers can process sequences, input sequence vectors are used, and inter-stock correlation is almost a natural result of the availability of the self-attention mechanism, whereas since the MLP supports none of these naturally, only single feature vectors enter the model for every forward pass. Essentially, differences in preprocessing steps between the transformer and the MLP model, although seemingly confounding, are in fact necessary/reasonable steps to make in order to support the use of such architectures. In this case, the *validity* of the independent variable *overall pipeline complexity* arguably does not fail from the confounding-related objections. However, it *is* limited in generalizability due to the unavoidable specificity in model architectures given that the variable is only binary. In response to this, future studies can experiment with perhaps “more ordinal levels” of overall pipeline complexity as opposed to only binary levels, and try replicating the results with deep learning models exhibiting other common transformer design principles found in the literature, to see if the results robustly generalize to models of “the same class”, per se, which is a much stronger setup compared to a limited MLP-or-transformer binary setup.

A certain architectural flaw in preprocessing steps, however, was discovered post-hoc. Namely, the social media impact features (author followers, post likes, etc.) were standardized for the MLP models, but were left unstandardized for the transformer models. This is unideal and may introduce a confounding factor between the MLPs and transformers, with no reasonably counteracting way to reframe the situation. Due to such oversight, the effect sizes of *overall pipeline complexity*, *inclusion of social media*, and the combined two-way interaction are affected, and results that reference these effect sizes may be taken with more skepticism. In response to this, future studies that aim to replicate this study are recommended to replicate such preprocessing structure with the corrected standardizing step installed for the transformer models, in order to fairly weight all experimental settings as intended by the qualitative interpretations of the independent variables.

A subtle limitation also mildly affects the validity of the OLS regression model for the baseline model comparisons. Namely, OLS regression normally assumes that samples are drawn independently; however, since the performance metrics per stock for a single model were taken from that single model, there is an element of sample dependence. Taking the model variable as a clustering variable under a cluster-robust framework would resolve this by recomputing standard errors that account for this dependence; however, the baseline comparison OLS regression models can only support one clustering variable, which a certain other variable (stocks) has already occupied, since observations from the same stocks across different models also count as a dependency structure. With this in mind, the effect sizes for the baseline comparison models may be more optimistic. Further research that encounters these situations may instead opt for other kinds of statistical testing that naturally sidestep input data dependency structures.

Recommendations for Practitioners

Given the results, it is the case that designing models with shorter prediction horizons rather than longer horizons is the most beneficial change among the possible changes to

other independent variables for optimizing raw model performance and stock ranking capacity. In that case, retail investors and algorithmic traders should carefully consider this variable and test different short prediction horizons (instead of potentially wasting resources on attempting to tune long prediction horizons).

Given the fact that news and social media do not necessarily offer overall advantages consistent in all experimental settings, their application must be guided by the design goals. If the goal of the model designer is to prioritize temporal stability of model confidence, especially across longer prediction horizons, the recommendation would be to incorporate news data into the study. For the addition of social media data, which shines more in moderative effects due to its context-dependent property, the model designer is recommended to evaluate through ablation the effect of adding social media instead of taking the main and interaction effects values presented at face value—which is recommended all the more given the limitation mentioned about social media data effects.

Deployment objectives must be taken into account in order to choose among the models, given that the results show no evidence of a single clear winning model, even when comparing against the machine learning baselines. Based on the conclusions above, practitioners would be recommended to prioritize deep learning models when robustness to drift is valuable—namely in order to minimize model retraining. That said, predictive performance should not be the only factor for choosing models, as model drift can provide practical insight through assessing retraining frequency needs.

Finally, application-oriented assessments are crucial for selecting models; consistent with other studies, this paper presents a different ranking of “best models” on the trading simulations as compared to the MCC scores. Given that the process for trading simulations takes into account a more granular approach (ranking models per forward pass in terms of prediction probability), which may be more informative than collapsing scores into hard binary labels for MCC computations, the trading simulations may provide a reliable assessment of core model behaviors while being related directly to realistic market

conditions given the derivation of its process from backtesting strategies.

REFERENCES

- Abolmakarem, S., Abdi, F., Khalili-Damghani, K., & Didehkhani, H. (2024). A multi-stage machine learning approach for stock price prediction: Engineered and derivative indices. *Intelligent Systems With Applications*, 24, 200449.
<https://doi.org/10.1016/j.iswa.2024.200449>
- Ahn, D. P. (2019). *Principles of commodity economics and finance*. The MIT Press.
- AI at Meta. (2022). NLLB-200 Distilled 600M [Model card. Accessed: 2026-07-31].
- Aït-Sahalia, Y., Fan, J., Xue, L., & Zhou, Y. (2022). *How and when are high-frequency stock returns predictable?* (Tech. rep.). National Bureau of Economic Research.
<https://doi.org/10.3386/w30366>
- Akita, R., Yoshihara, A., Matsubara, T., & Uehara, K. (2016). Deep learning for stock prediction using numerical and textual information. *2016 IEEE/ACIS 15th International Conference on Computer and Information Science (ICIS)*, 1–6.
<https://doi.org/10.1109/ICIS.2016.7550882>
- Albada, M. C., & Sonola, M. J. (2025). Predicting stock movement with BERTweet and transformers [ArXiv.org]. <https://doi.org/10.48550/arxiv.2503.10957>
- Aldasoro, I., Hördahl, P., Schrimpf, A., & Zhu, S. (2025, March). Predicting financial market stress with machine learning. <https://www.bis.org/publ/work1250.htm>
- Apergis, N., & Miller, S. M. (2009). Do structural oil-market shocks affect stock prices? *Energy Economics*, 31(4), 569–575. <https://doi.org/10.1016/j.eneco.2009.03.001>
- Araci, D. (2019). FinBERT: Financial sentiment analysis with pre-trained language models [arXiv (Cornell University)]. <https://doi.org/10.48550/arxiv.1908.10063>
- Arora, A., Datta, A., & Ding, V. (2019). *Using news titles and financial features to predict intraday movements of the DJIA* (tech. rep.) (Stanford CS224N Project Report). Stanford University. <https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1194/reports/custom/15744462.pdf>

- Aydin, R., Lögün, A., & Aydin, B. (2023). The relationship between exchange rates and stock prices: Comparative example of ASEAN and BRICS countries. *Romanian Journal of Economic Forecasting*, 26(4), 128–142.
- Baele, L., Bekaert, G., & Inghelbrecht, K. (2010). The determinants of stock and bond return comovements. *The Review of Financial Studies*, 23(6), 2374–2428. Retrieved July 10, 2026, from <http://www.jstor.org/stable/40604737>
- Baker, S. R., Bloom, N., Davis, S. J., & Sammon, M. (2025). *What triggers stock market jumps?* (Tech. rep.). National Bureau of Economic Research. <https://doi.org/10.3386/w28687>
- Barrios, I., D. M., & Gerardo, B. D. (2024). Forecasting of Philippine Stock Exchange index using optimized artificial neural networks with enhanced PSO algorithm. *International Journal of Engineering Trends and Technology*, 72(6), 128–135. <https://doi.org/10.14445/22315381/ijett-v72i6p113>
- Basak, S., Kar, S., Saha, S., Khaidem, L., & Dey, S. R. (2018). Predicting the direction of stock market prices using tree-based classifiers. *The North American Journal of Economics and Finance*, 47, 552–567. <https://doi.org/10.1016/j.najef.2018.06.013>
- Baur, D. G., & Lucey, B. M. (2010). Is gold a hedge or a safe haven? An analysis of stocks, bonds and gold. *Financial Review*, 45(2), 217–229. <https://doi.org/10.1111/j.1540-6288.2010.00244.x>
- Beniwal, M. (2025). Adaptive weighted genetic algorithm-optimized SVR for robust long-term forecasting of global stock indices for investment decisions [arXiv (Cornell University)]. <https://doi.org/10.48550/arxiv.2512.15113>
- Bernard, V. L., & Thomas, J. K. (1989). Post-earnings-announcement drift: Delayed price response or risk premium? *Journal of Accounting Research*, 27, 1. <https://doi.org/10.2307/2491062>

- Berthet, Q., Blondel, M., Teboul, O., Cuturi, M., Vert, J.-P., & Bach, F. (2020). Learning with differentiable perturbed optimizers [arXiv (Cornell University)].
<https://doi.org/10.48550/arxiv.2002.08676>
- Bhattacharya, D., Li, W.-H., & Sonaer, G. (2015). Has momentum lost its momentum? *Review of Quantitative Finance and Accounting*, 48(1), 191–218.
<https://doi.org/10.1007/s11156-015-0547-8>
- Bifet, A., & Gavalà, R. (2007). Learning from time-changing data with Adaptive Windowing. *Proceedings of the 2007 SIAM International Conference on Data Mining*, 443–448. <https://doi.org/10.1137/1.9781611972771.42>
- Bodie, Z., Kane, A., & Marcus, A. J. (2023). *Investments (thirteenth edition, international student edition.)* McGraw Hill LLC.
- Bollinger, J. (2001). *Bollinger on bollinger bands*. McGraw-Hill.
- Box, G. E. P., & Jenkins, G. M. (1970). *Time series analysis: Forecasting and control*. Holden-Day.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/a:1010933404324>
- Bustarviejo, J., & Bousoño-Calzón, C. (2026). Multimodal information fusion for financial forecasting via cross-attention and calibrated uncertainty. *Machine Learning With Applications*, 23, 100840. <https://doi.org/10.1016/j.mlwa.2026.100840>
- Bustos, O., Pomares-Quimbaya, A., & Stellan, R. (2025). Machine learning, stock market forecasting, and market efficiency: A comparative study. *International Journal of Data Science and Analytics*, 20(7), 6815–6839.
<https://doi.org/10.1007/s41060-025-00854-4>
- Casao, A. C., Perez, C. A., & Bragas, C. M. (2025). Social media platforms: Its digital influence on shaping investment decisions among Filipino working millennials. *International Journal of Research Publication and Reviews*, 6(4), 2774–2787.
<https://www.researchgate.net/publication/390794287>

- Catão, L., & Timmermann, A. (2010). Volatility regimes and global equity returns. In *Oxford university press ebooks* (pp. 257–295).
<https://doi.org/10.1093/acprof:oso/9780199549498.003.0013>
- Chen, J., Liu, Y., Chen, Y., & Zhang, B. (2026). News-driven stock forecasting with enhanced attention mechanisms. *Service Science. ICSS 2025. Communications in Computer and Information Science*, 2626, 275–287.
https://doi.org/10.1007/978-981-95-1581-3_17
- Chen, P., Boukouvalas, Z., & Corizzo, R. (2024). A deep fusion model for stock market prediction with news headlines and time series data. *Neural Computing and Applications*, 36(34), 21229–21271. <https://doi.org/10.1007/s00521-024-10303-1>
- Chen, Q. (2021). Stock movement prediction with financial news using contextualized embedding from BERT [arXiv (Cornell University)].
<https://doi.org/10.48550/arxiv.2107.08721>
- Chen, T., & Guestrin, C. (2016). XGBoost [arXiv (Cornell University)].
<https://doi.org/10.1145/2939672.2939785>
- Ciner, C., Gurdgiev, C., & Lucey, B. M. (2013). Hedges and safe havens: An examination of stocks, bonds, gold, oil and exchange rates. *International Review of Financial Analysis*, 29, 202–211. <https://doi.org/10.1016/j.irfa.2012.12.001>
- Cordonnier, J.-B., Mahendran, A., Dosovitskiy, A., Weissenborn, D., Uszkoreit, J., & Unterthiner, T. (2021). Differentiable patch selection for image recognition. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) 2021*. <https://doi.org/10.48550/arXiv.2104.03059>
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297. <https://doi.org/10.1007/bf00994018>
- Daniluk, M., Dabrowski, J., Rychalska, B., & Goluchowski, K. (2021). Synerise at RecSys 2021: Twitter user engagement prediction with a fast neural model. *Proceedings of*

the Recommender Systems Challenge 2021, 15–21.

<https://doi.org/10.1145/3487572.3487599>

Darwish, M., Hassanien, E. E., & Eissa, A. H. B. (2025). Stock market forecasting: From traditional predictive models to large language models. *Computational Economics*.

<https://doi.org/10.1007/s10614-025-11024-w>

Dashtaki, S. M., Chagahi, M. H., Bahadori, A., Moshiri, B., Piran, M. J., & Montazeri, A. (2025). HSIF: A transformer-based cross-attention framework for cryptocurrency trend forecasting via multimodal sentiment—market fusion. *IEEE Access*, 13,

156600–156612. <https://doi.org/10.1109/access.2025.3605522>

Deep, A., Shirvani, A., Monico, C., Rachev, S., & Fabozzi, F. J. (2024). Risk-adjusted performance of random forest models in high-frequency trading. *arXiv (Cornell University)*. <https://doi.org/10.48550/arxiv.2412.15448>

de Oliveira, F. A., Nobre, C. N., & Zárata, L. E. (2013). Applying artificial neural networks to prediction of stock price and improvement of the directional prediction index — case study of PETR4, Petrobras, Brazil. *Expert Systems with Applications*, 40(18), 7596–7606. <https://doi.org/10.1016/j.eswa.2013.06.071>

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding [ArXiv.org].

<https://doi.org/10.48550/arXiv.1810.04805>

Du, S., & Zhu, H. (2017). What is the optimal trading frequency in financial markets? *The Review of Economic Studies*. <https://doi.org/10.1093/restud/rdx006>

Enilov, M., Fazio, G., & Ghoshray, A. (2021). Global connectivity between commodity prices and national stock markets: A time-varying MIDAS analysis. *International Journal of Finance & Economics*, 28(3), 2607–2619.

<https://doi.org/10.1002/ijfe.2552>

Fabozzi, F. J., & Fabozzi, F. A. (2026). *Bond markets, analysis, and strategies*. The MIT Press.

- Fama, E. F. (1970). Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2), 383. <https://doi.org/10.2307/2325486>
- Fataliyev, K., & Liu, W. (2023). MCASP: Multi-modal cross attention network for stock market prediction. *Proceedings of the 21st Annual Workshop of the Australasian Language Technology Association*. <https://alta2023.alta.asn.au/files/29.pdf>
- Feng, C., Li, Y., Chen, Z., & Guo, L. (2025). The evolution and breakthrough of natural language processing: The revolution from rules to deep learning. *Proceedings of the 2024 5th International Conference on Computer Science and Management Technology (ICCSMT '24)*, 307–311. <https://doi.org/10.1145/3708036.3708089>
- Funde, Y., & Damani, A. (2023). Comparison of ARIMA and exponential smoothing models in prediction of stock prices. *The Journal of Prediction Markets*, 17(1), 21–38. <https://doi.org/10.5750/jpm.v17i1.2017>
- Gallardo, M., & Viray, E., Jr. (2025). Philippine Stock Exchange index (PSEi) and its selected macroeconomic indicators: Catalyst for sustainable economic growth. *Bedan Research Journal*, 10(1), 82–116. <https://doi.org/10.58870/berj.v10i1.81>
- Garcia-Rubio, A. M., Heck, L., Schweitzer, K. M., Bateman, R. M., Alaeddini, A., & Castillo-Villar, K. K. (2026). Detecting concept drift in object detection models: A collaborative AI-human approach for defense applications. *IEEE Access*, 14, 7412–7429. <https://doi.org/10.1109/access.2026.3652106>
- Gerety, M. S., & Mulherin, J. H. (1992). Trading halts and market activity: An analysis of volume at the open and the close. *The Journal of Finance*, 47(5), 1765. <https://doi.org/10.2307/2328995>
- Ghosh, P., Neufeld, A., & Sahoo, J. K. (2022). Forecasting directional movements of stock prices for intraday trading using LSTM and random forests. *Finance Research Letters*, 46, 102280. <https://doi.org/10.1016/j.frl.2021.102280>

- Goluza, S., Kovačević, T., Bauman, T., & Kostanjčar, Z. (2024). Deep reinforcement learning with positional context for intraday trading. *Evolving Systems*, 15(5), 1865–1880. <https://doi.org/10.1007/s12530-024-09593-6>
- Gu, S., Kelly, B., & Xiu, D. (2020). Empirical asset pricing via machine learning. *Review of Financial Studies*, 33(5), 2223–2273. <https://doi.org/10.1093/rfs/hhaa009>
- Gu, W., Zhong, Y., Li, S., Wei, C., Dong, L., Wang, Z., & Yan, C. (2024). Predicting stock prices with FinBERT-LSTM: Integrating news sentiment analysis [arXiv (Cornell University)]. <https://doi.org/10.48550/arxiv.2407.16150>
- Guennioui, O., Chiadmi, D., & Amghar, M. (2024). Global stock price forecasting during a period of market stress using LightGBM. *International Journal of Computing and Digital Systems*, 15(1), 19–27. <https://doi.org/10.12785/ijcds/150103>
- Gunduz, H., Yaslan, Y., & Cataltepe, Z. (2017). Intraday prediction of Borsa Istanbul using convolutional neural networks and feature correlations. *Knowledge-Based Systems*, 137, 138–148. <https://doi.org/10.1016/j.knosys.2017.09.023>
- Gurrib, I. (2018). Performance of the average directional index as a market timing tool for the most actively traded USD based currency pairs. *Banks and Bank Systems*, 13(3), 58–70. [https://doi.org/10.21511/bbs.13\(3\).2018.06](https://doi.org/10.21511/bbs.13(3).2018.06)
- Halder, S. (2022). FinBERT-LSTM: Deep learning based stock price prediction using news sentiment analysis [arXiv (Cornell University)]. <https://doi.org/10.48550/arxiv.2211.07392>
- He, S., & Gu, S. (2021). Multi-modal attention network for stock movements prediction [arXiv (Cornell University)]. <https://doi.org/10.48550/arxiv.2112.13593>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Holt, C. C. (1957). *Forecasting seasonals and trends by exponentially weighted moving averages* (tech. rep. No. ONR Memorandum No. 52). Carnegie Institute of Technology. <https://doi.org/10.1016/j.ijforecast.2003.09.015>

- Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366.
[https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8)
- Huddleston, D., Liu, F., & Stentoft, L. (2021). Intraday market predictability: A machine learning approach. *Journal of Financial Econometrics*, 21(2), 485–527.
<https://doi.org/10.1093/jjfinec/nbab007>
- Ito, T., & Lin, W.-L. (1992). Lunch break and intraday volatility of stock returns. *Economics Letters*, 39(1), 85–90. [https://doi.org/10.1016/0165-1765\(92\)90106-9](https://doi.org/10.1016/0165-1765(92)90106-9)
- Jang, J. (2025). Selective news selection model for explainable stock prediction via cross-attention integration. *Finance Research Letters*, 85, 108146.
<https://doi.org/10.1016/j.frl.2025.108146>
- Jarmulska, B. (2021). Random forest versus logit models: Which offers better early warning of fiscal stress? *Journal of Forecasting*, 41(3), 455–490.
<https://doi.org/10.1002/for.2806>
- Jegadeesh, N., & Titman, S. (1993). Returns to buying winners and selling losers: Implications for stock market efficiency. *The Journal of Finance*, 48(1), 65–91.
<https://doi.org/10.1111/j.1540-6261.1993.tb04702.x>
- Ji, L., Hou, W., Zhou, H., Xiong, L., Liu, C., Yuan, Z., & Li, L. (2025). EBMGP: A deep learning model for genomic prediction based on elastic net feature selection and bidirectional encoder representations from transformer's embedding and multi-head attention pooling. *Theoretical and Applied Genetics*, 138(5), 103.
<https://doi.org/10.1007/s00122-025-04894-z>
- Jiang, X. (2025). A multi-task learning model for stock market forecasting. *Applied and Computational Engineering*, 146(1), 1–8.
<https://doi.org/10.54254/2755-2721/2025.tj21873>
- Kang, B.-K. (2021). Improving MACD technical analysis by optimizing parameters and modifying trading rules: Evidence from the Japanese Nikkei 225 futures market.

Journal of Risk and Financial Management, 14(1).

<https://doi.org/10.3390/jrfm14010037>

Karadaş, F., Eravcı, B., & Özbayoğlu, A. M. (2025). Multimodal stock price prediction [ArXiv.org]. <https://doi.org/10.48550/arxiv.2502.05186>

Karpoff, J. M. (1987). The relation between price changes and trading volume: A survey. *The Journal of Financial and Quantitative Analysis*, 22(1), 109–126. Retrieved July 11, 2026, from <http://www.jstor.org/stable/2330874>

Kazemi, S. M., Goel, R., Eghbali, S., Ramanan, J., Sahota, J., Thakur, S., Wu, S., Smyth, C., Poupart, P., & Brubaker, M. (2019). Time2Vec: Learning a vector representation of time [arXiv (Cornell University)].

<https://doi.org/10.48550/arxiv.1907.05321>

Ke, B., & Ramalingegowda, S. (2004). Do institutional investors exploit the post-earnings announcement drift? *Journal of Accounting and Economics*, 39(1), 25–53.

<https://doi.org/10.1016/j.jacceco.2004.02.002>

Ke, Z. T., Kelly, B., & Xiu, D. (2019). *Predicting returns with text data* (tech. rep.).

National Bureau of Economic Research. <https://doi.org/10.3386/w26186>

Kerdprasop, N., Kerdprasop, K., Chansilp, K., Kaoungku, N., Sornlertlamvanish, P., Srivisut, K., Kanjanawattana, S., Bunrit, S., & Chuaybamroong, P. (2025). Multi-output classification for predicting stock market trend in the electronic sector of Thailand. *Advances in Transdisciplinary Engineering*.

<https://doi.org/10.3233/atde250583>

Kerssenfischer, M., & Schmeling, M. (2024). What moves markets? *Journal of Monetary Economics*, 145, 103560. <https://doi.org/10.1016/j.jmoneco.2024.103560>

Khan, A. H., Shah, A., Ali, A., Shahid, R., Zahid, Z. U., Sharif, M. U., Jan, T., & Zafar, M. H. (2023). A performance comparison of machine learning models for stock market prediction with novel investment strategy. *PLoS ONE*, 18(9), e0286362. <https://doi.org/10.1371/journal.pone.0286362>

- Kim, R., So, C. H., Jeong, M., Lee, S., Kim, J., & Kang, J. (2019). HATS: A hierarchical graph attention network for stock movement prediction [arXiv (Cornell University)]. <https://doi.org/10.48550/arxiv.1908.07999>
- Kim, T., & Kim, H. Y. (2019). Forecasting stock prices with a feature fusion LSTM-CNN model using different representations of the same data. *PLoS ONE*, *14*(2), e0212320. <https://doi.org/10.1371/journal.pone.0212320>
- Kong, A., Zhu, H., & Azencott, R. (2020). Predicting intraday jumps in stock prices using liquidity measures and technical indicators. *Journal of Forecasting*, *40*(3), 416–438. <https://doi.org/10.1002/for.2721>
- Krauss, C., Anh, X., Do, & Huck, N. (2016). Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the S&P 500. *European Journal of Operational Research*, *259*(2), 689–702. <https://doi.org/10.1016/j.ejor.2016.10.031>
- Kyriakou, I., Mousavi, P., Nielsen, J. P., & Scholz, M. (2019). Forecasting benchmarks of long-term stock returns via machine learning. *Annals of Operations Research*, *297*(1-2), 221–240. <https://doi.org/10.1007/s10479-019-03338-4>
- Lacanilao, J., G., & Signo, J. (2021). *Application and implementation of regression and time-series forecasting machine learning algorithm for Philippine stock market* [Doctoral dissertation, Jose Rizal University]. <https://doi.org/10.13140/RG.2.2.19476.01927>
- Lane, G. C. (1984). Lane's stochastics. *Technical Analysis of Stocks & Commodities*, *2*(3), 87–90.
- Lee, H., Surdeanu, M., MacCartney, B., & Jurafsky, D. (2014). On the importance of text analysis for stock price prediction. In N. Calzolari, K. Choukri, T. Declerck, H. Loftsson, B. Maegaard, J. Mariani, A. Moreno, J. Odijk, & S. Piperidis (Eds.), *Proceedings of the ninth international conference on language resources and evaluation (lrec 2014)* (pp. 1170–1175). European Language Resources Association. <https://aclanthology.org/L14-1048/>

- Lee, J. Y., & Yoo, S. J. (2025). Stock price prediction using transformer and Time2Vec. *2025 International Conference on Artificial Intelligence in Information and Communication (ICAIIIC)*, 687–689.
<https://doi.org/10.1109/icaaiic64266.2025.10920805>
- Lee, S., Min, K., Son, Y., & Do, H. (2026). Adaptive time encoding for irregular multivariate time-series classification. *The Thirty-Ninth Annual Conference on Neural Information Processing Systems*.
<https://openreview.net/forum?id=l0kR6m9NDT>
- Li, Q., Tan, J., Wang, J., & Chen, H. (2020). A multimodal event-driven LSTM model for stock prediction using online news. *IEEE Transactions on Knowledge and Data Engineering*, 33(10), 3323–3337. <https://doi.org/10.1109/tkde.2020.2968894>
- Li, S., Schulwol, Z. B., & Miikkulainen, R. (2025). Transformer based time-series forecasting for stock [ArXiv.org]. <https://doi.org/10.48550/arxiv.2502.09625>
- Liao, P., Lee, H., Cheng, Y., Chen, L., Lee, H., & Wang, H. (2026). Generalized stock price prediction for multiple stocks combined with news fusion [arXiv (Cornell University)]. <https://doi.org/10.48550/arxiv.2603.19286>
- Liu, H. (2025). A hybrid CNN-LSTM approach for effective stock price prediction in optimizing investment strategies. In *Advances in economics, business and management research* (pp. 628–640). https://doi.org/10.2991/978-94-6463-652-9_65
- Lo, A. W. (2004). The adaptive markets hypothesis. *The Journal of Portfolio Management*, 30(5), 15–29. <https://doi.org/10.3905/jpm.2004.442611>
- Lohrmann, C., & Luukka, P. (2018). Classification of intraday S&P500 returns with a random forest. *International Journal of Forecasting*, 35(1), 390–407.
<https://doi.org/10.1016/j.ijforecast.2018.08.004>
- Lu, J., Henchion, M., Bacher, I., & Mac Namee, B. (2021). A sentence-level hierarchical BERT model for document classification with limited labelled data [arXiv (Cornell University)]. <https://doi.org/10.48550/arxiv.2106.06738>

- Mandal, R. C., Kler, R., Tiwari, A., Keshta, I., Abonazel, M. R., Tageldin, E. M., & Umaralievich, M. S. (2023). Enhancing stock price prediction with deep cross-modal information fusion network. *Fluctuation and Noise Letters*, 23(02).
<https://doi.org/10.1142/s0219477524400170>
- Mehtab, S., & Sen, J. (2019). A robust predictive model for stock price prediction using deep learning and natural language processing. *SSRN Electronic Journal*.
<https://doi.org/10.48550/arXiv.1912.07700>
- Mishkin, F. S. (2018). *Economics of money, banking and financial markets (12th ed.)*. Pearson.
- Moberg, J.-M., & Sucarrat, G. (2007). *Stock market return, order flow and financial market linkages* (tech. rep.). OlsenData. <https://www.sucarrat.net/research/stocknflows.pdf>
- Mohan, S., Mullapudi, S., Sammeta, S., Vijayvergia, P., & Anastasiu, D. C. (2019). Stock price prediction using news sentiment analysis. *2019 IEEE Fifth International Conference on Big Data Computing Service and Applications (BigDataService)*.
<https://doi.org/10.1109/bigdataservice.2019.00035>
- Muruganandam, N. S., & Arumugam, U. (2022). Seminal stacked long short-term memory (SS-LSTM) model for forecasting particulate matter (PM_{2.5} and PM₁₀). *Atmosphere*, 13(10), 1726. <https://doi.org/10.3390/atmos13101726>
- Olorunnimbe, K., & Viktor, H. (2022). Deep learning in the stock market — a systematic survey of practice, backtesting, and applications. *Artificial Intelligence Review*, 56(3), 2057–2109. <https://doi.org/10.1007/s10462-022-10226-0>
- Omranpour, S., Rabusseau, G., & Rabbany, R. (2024). Higher order transformers: Enhancing stock movement prediction on multimodal time-series data.
<https://arxiv.org/abs/2412.10540>
- Ou, J. A., & Penman, S. H. (1989). Financial statement analysis and the prediction of stock returns. *Journal of Accounting and Economics*, 11(4), 295–329.
[https://doi.org/10.1016/0165-4101\(89\)90017-7](https://doi.org/10.1016/0165-4101(89)90017-7)

- Park, J., & Ratti, R. A. (2008). Oil price shocks and stock markets in the U.S. and 13 European countries. *Energy Economics*, 30(5), 2587–2608.
<https://doi.org/10.1016/j.eneco.2008.04.003>
- Patel, J., Shah, S., Thakkar, P., & Kotecha, K. (2014). Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques. *Expert Systems With Applications*, 42(1), 259–268.
<https://doi.org/10.1016/j.eswa.2014.07.040>
- Phylaktis, K., & Ravazzolo, F. (2005). Stock prices and exchange rate dynamics. *Journal of International Money and Finance*, 24(7), 1031–1053.
<https://doi.org/10.1016/j.jimonfin.2005.08.001>
- Pires, T., Schlinger, E., & Garrette, D. (2019). How multilingual is Multilingual BERT? [arXiv (Cornell University)]. <https://doi.org/10.48550/arxiv.1906.01502>
- Puh, K., & Babac, M. B. (2023). Predicting stock market using natural language processing. *American Journal of Business*, 38(2), 41–61.
<https://doi.org/10.1108/ajb-08-2022-0124>
- Pulicharla, M. R. (2019). Detecting and addressing model drift: Automated monitoring and real-time retraining in ML pipelines. *World Journal of Advanced Research and Reviews*, 3(2), 147–152. <https://doi.org/10.30574/wjarr.2019.3.2.0189>
- Rahman, M. M., Watanobe, Y., & Nakamura, K. (2021). A bidirectional LSTM language model for code evaluation and repair. *Symmetry*, 13(2), 247.
<https://doi.org/10.3390/sym13020247>
- Ravat, A., Khilji, R., Suryavanshi, D., & Panigrahi, P. (2025). Data science approach to intraday stock prediction using LSTM networks and social media sentiment. *Journal of Advance and Future Research*, 3(11), 194–199.
<https://rjwave.org/jaafr/papers/JAAFR2511249.pdf>
- Reed, A. V. (2013). Short selling. *Annual Review of Financial Economics*, 5(Volume 5, 2013), 245–258. <https://doi.org/10.1146/annurev-financial-110311-101813>

- Ridhawi, M. A., Ali, M. H., & Osman, H. A. (2026). Stock market prediction using node transformer architecture integrated with BERT sentiment analysis [Open MIND. (2026b)]. <https://doi.org/10.48550/arxiv.2603.05917>
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *65*, (6), 386–408. <https://doi.org/10.1037/h0042519>
- Rudhistiar, D., Wahyani, W., Yusri, T. A. S., & Faisol, A. (2025). Hybrid CNN-BiLSTM model with attention mechanism for stock price prediction: A multivariate approach. *IAENG International Journal of Applied Mathematics*, *55*(12), 4468–4475. https://www.iaeng.org/IJAM/issues_v55/issue_12/IJAM_55_12_48.pdf
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, *323*(6088), 533–536. <https://doi.org/10.1038/323533a0>
- Sable, R., Goel, S., & Chatterjee, P. (2019). Empirical study on stock market prediction using machine learning. *2019 International Conference on Advances in Computing, Communication and Control (ICAC3)*, 1–5. <https://doi.org/10.1109/icac347590.2019.9036786>
- Sable, R., Goel, S., & Chatterjee, P. (2023). Techniques for stock market prediction: A review. *International Journal on Recent and Innovation Trends in Computing and Communication*, *11*(5s), 381–402. <https://doi.org/10.17762/ijritcc.v11i5s.7056>
- Safari, S. A., & Schmidhuber, C. (2025). Trends and reversion in financial markets on time scales from minutes to decades [arXiv (Cornell University)]. <https://doi.org/10.48550/arxiv.2501.16772>
- San Miguel, E. B. (2019). Interrelationship between Philippine Stock Exchange Services sector index and selected macroeconomic indicators. *Indian Journal of Science and Technology*, *12*(1), 1–10. <https://doi.org/10.17485/ijst/2019/v12i30/145444>

- Saravanos, C., & Kanavos, A. (2024). Forecasting stock market volatility using social media sentiment analysis. *Neural Computing and Applications*, 37(17), 10771–10794. <https://doi.org/10.1007/s00521-024-10807-w>
- Sarkar, B., & Shahid, A. (2025). Hybrid FinBERT-LSTM deep learning framework for stock price prediction: A sentiment analysis approach using earnings call transcripts. In *Lecture notes in networks and systems* (pp. 355–367). https://doi.org/10.1007/978-981-96-6438-2_24
- Saud, A. S., & Shakya, S. (2022). Directional movement index based machine learning strategy for predicting stock trading signals. *International Journal of Power Electronics and Drive Systems/International Journal of Electrical and Computer Engineering*, 12(4), 4185. <https://doi.org/10.11591/ijece.v12i4.pp4185-4194>
- Schabacker, R. (2021). *Technical analysis and stock market profits*. Harriman House Limited.
- Shen, F., Chao, J., & Zhao, J. (2015). Forecasting exchange rate using deep belief networks and conjugate gradient method. *Neurocomputing*, 167, 243–253. <https://doi.org/10.1016/j.neucom.2015.04.071>
- Shi, L., Teng, Z., Wang, L., Zhang, Y., & Binder, A. (2018). DeepClue: Visual interpretation of text-based deep stock prediction. *IEEE Transactions on Knowledge and Data Engineering*, 31(6), 1094–1108. <https://doi.org/10.1109/tkde.2018.2854193>
- Singh, T., Kalra, R., Mishra, S., Satakshi, & Kumar, M. (2022). An efficient real-time stock prediction exploiting incremental learning and deep learning. *Evolving Systems*, 14(6), 919–937. <https://doi.org/10.1007/s12530-022-09481-x>
- Sirnes, E., & Dinh, M. T. H. (2021). Tick size and price reversal after order imbalance. *International Journal of Financial Studies*, 9(2), 19. <https://doi.org/10.3390/ijfs9020019>

- Slutzky, E. (1937). The summation of random causes as the source of cyclic processes. *Econometrica*, 5(2), 105. <https://doi.org/10.2307/1907241>
- Soebhag, A., Baltussen, G., & Da, Z. (2024). End-of-day reversal. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.5039009>
- Sombrito, S. A. E., Gonzales, K. J. G., Nolasco, J. K. L. T., & Tiglao, N. M. C. (2023). Philippine stock direction forecasting utilizing technical, fundamental, and news sentiment data. In S. I. Lopes, P. Fraga-Lamas, T. M. Fernández-Camáres, B. R. Dawadi, D. B. Rawat, & S. Shakya (Eds.), *Smart technologies for sustainable and resilient ecosystems: Edge-iot smartgov 2022* (Vol. 510). Springer. https://doi.org/10.1007/978-3-031-35982-8_4
- Somkunwar, R. K., & Srivastava, V. (2024). ARIMA and XGBoost stock market forecasting: A review. *International Journal of Scientific Research & Engineering Trends*, 10(2), 427–434. https://ijsret.com/wp-content/uploads/2024/03/IJSRET_V10_issue2_200.pdf
- Son, Y., Noh, D., & Lee, J. (2012). Forecasting trends of high-frequency KOSPI200 index data using learning classifiers. *Expert Systems With Applications*, 39(14), 11607–11615. <https://doi.org/10.1016/j.eswa.2012.04.015>
- Sun, G., & Li, Y. (2025). Intraday and post-market investor sentiment for stock price prediction: A deep learning framework with explainability and quantitative trading strategy. *Systems*, 13(5), 390. <https://doi.org/10.3390/systems13050390>
- Sun, L., Xu, W., & Liu, J. (2021). Two-channel attention mechanism fusion model of stock price prediction based on CNN-LSTM. *ACM Transactions on Asian and Low-Resource Language Information Processing*, 20(5), 1–12. <https://doi.org/10.1145/3453693>
- Sun, W., Yuan, C., You, S., Liu, Z., Chen, Y., & Cui, W. (2026). Intelligent stock price prediction model research integrating multimodal information and KAN networks. *Computational Economics*. <https://doi.org/10.1007/s10614-026-11314-x>

TabularisAI. (2025). ModernFinBERT [Model card. Accessed: 2026-07-31].

<https://huggingface.co/tabularisai/ModernFinBERT>

Tan, L., Chen, J., Zheng, B., & Ouyang, F. (2016). Exploring market state and stock interactions on the minute timescale. *PLoS ONE*, 11(2), e0149648.

<https://doi.org/10.1371/journal.pone.0149648>

Tanaka-Yamawaki, M., & Tokuoka, S. (2007). Adaptive use of technical indicators for the prediction of intra-day stock prices. *Physica A: Statistical Mechanics and Its Applications*, 383(1), 125–133. <https://doi.org/10.1016/j.physa.2007.04.126>

Thompson, O. (2024). Factors influencing stock market prices: A comprehensive analysis. *Academy of Accounting and Financial Studies Journal*, 28(4), 1–2.

Thorp, W. A. (2000). ID'ing when to buy and sell using the stochastic oscillator. *AII Journal*, 25–28.

[https://cdn.preterhuman.net/texts/unsorted2/Stock%5C%20books%5C%20058/](https://cdn.preterhuman.net/texts/unsorted2/Stock%5C%20books%5C%20058/Wayne%5C%20A.%5C%20Thorp/Wayne%5C%20A.%5C%20Thorp%5C%20-%5C%20Technical%5C%20Analysis.pdf)

[Wayne%5C%20A.%5C%20Thorp/Wayne%5C%20A.%5C%20Thorp%5C%20-%5C%20Technical%5C%20Analysis.pdf](https://cdn.preterhuman.net/texts/unsorted2/Stock%5C%20books%5C%20058/Wayne%5C%20A.%5C%20Thorp/Wayne%5C%20A.%5C%20Thorp%5C%20-%5C%20Technical%5C%20Analysis.pdf)

Țițan, A. G. (2015). The efficient market hypothesis: Review of specialized literature and empirical research. *Procedia Economics and Finance*, 32, 442–449.

[https://doi.org/10.1016/s2212-5671\(15\)01416-1](https://doi.org/10.1016/s2212-5671(15)01416-1)

Tsang, W. W. H., & Chong, T. T. L. (2009). Profitability of the on-balance volume indicator. *Economics Bulletin*, 29(3), 2424–2431.

Ughi, R., Lomurno, E., & Matteucci, M. (2023). Two steps forward and one behind: Rethinking time series forecasting with deep learning [arXiv (Cornell University)].

<https://doi.org/10.48550/arxiv.2304.04553>

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need [arXiv (Cornell University)].

<https://doi.org/10.65215/2q58a426>

- Vogel, A., & Kraus, M. (2025). Transformer-based discovery of dynamic relationships in high-frequency financial time series. *Journal of Computing and Electronic Information Management*, 18(2), 47–52. <https://doi.org/10.54097/dg5nnh24>
- Wang, D., Cheng, M., Liu, Z., & Liu, Q. (2025). TimeDART: A diffusion autoregressive transformer for self-supervised time series representation. <https://arxiv.org/abs/2410.05711>
- Wang, L., Zhang, Y., Yuan, J., Hu, K., & Cao, S. (2022). FEBDNN: Fusion embedding-based deep neural network for user retweeting behavior prediction on social networks. *Neural Computing and Applications*, 34(16), 13219–13235. <https://doi.org/10.1007/s00521-022-07174-9>
- Wang, T. W., Lu, Y. S., Ku, C. S., Mohammed, O., Raj, B., & Por, L. Y. (2026). A multimodal deep fusion method for stock movement prediction using heterogeneous data source. *IEEE Access*, 14, 9882–9896. <https://doi.org/10.1109/access.2026.3654950>
- Wang, X., Wu, C., & Xu, W. (2015). Volatility forecasting: The role of lunch-break returns, overnight returns, trading volume and leverage effects. *International Journal of Forecasting*, 31(3), 609–619. <https://doi.org/10.1016/j.ijforecast.2014.10.007>
- Wang, Y., Wang, Q., & Zhou, W. (2026). High-frequency trading in the Chinese stock market: Predictability, profitability, and regulation. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.6156666>
- Wang, Y. (2014). Stock price direction prediction by directly using prices data: An empirical study on the KOSPI and HSI. *International Journal of Business Intelligence and Data Mining*, 9(2), 145–160. <https://doi.org/10.1504/IJBIDM.2014.065091>
- Wilder, J. W. (1978). *New concepts in technical trading systems*.
- Winters, P. R. (1960). Forecasting sales by exponentially weighted moving averages. *Management Science*, 6(3), 324–342. <https://doi.org/10.1287/mnsc.6.3.324>

- Xu, H., Wan, H., Wu, Y., Zheng, J., & Xie, L. (2025). HRFormer: A hybrid relational transformer for stock time series forecasting. *Electronics*, 14(22), 4459.
<https://doi.org/10.3390/electronics14224459>
- Yadav, N. (2024). Enhancing stock trend prediction using BERT-based sentiment analysis and machine learning techniques. *International Journal of Quantitative Research and Modeling*, 5(1), 1–11. <https://doi.org/10.46336/ijqrm.v5i1.567>
- Yang, B., Liang, T., Xiong, J., & Zhong, C. (2022). Deep reinforcement learning based on transformer and U-Net framework for stock trading. *Knowledge-Based Systems*, 262, 110211. <https://doi.org/10.1016/j.knosys.2022.110211>
- Yao, S., Luo, L., & Peng, H. (2018). High-frequency stock trend forecast using LSTM model. *2018 13th International Conference on Computer Science & Education (ICCSE)*, 1–4. <https://doi.org/10.1109/iccse.2018.8468703>
- Yule, G. U. (1927). On a method of investigating periodicities in disturbed series with special reference to Wolfer's sunspot numbers. *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character*, 226, 267–298. <https://www.jstor.org/stable/91170>
- Yurastika, F., & Wibowo, B. (2021). Volatility spillover between stock and bond returns: Evidence from ASEAN-5 countries. *Proceedings of the 1st International Conference on Sustainable Management and Innovation, ICoSMI 2020*.
<https://doi.org/10.4108/eai.14-9-2020.2304427>
- Zanotti, M. (2025). Do global forecasting models require frequent retraining? [arXiv (Cornell University)]. <https://doi.org/10.48550/arxiv.2505.00356>
- Zaznov, I., Kunkel, J. M., Badii, A., & Dufour, A. (2024). The intraday dynamics predictor: A trioflow fusion of convolutional layers and gated recurrent units for high-frequency price movement forecasting. *Applied Sciences*, 14(7), 2984.
<https://doi.org/10.3390/app14072984>

- Zhang, C. Y., & Jacobsen, B. (2020). The Halloween indicator, “sell in May and go away”: Everywhere and all the time. *Journal of International Money and Finance*, 110, 102268. <https://doi.org/10.1016/j.jimonfin.2020.102268>
- Zhang, P., Harris, R. D., & Zheng, J. (2025). GNN-based social media sentiment analysis for stock market forecasting and trading. *Expert Systems With Applications*, 291, 128425. <https://doi.org/10.1016/j.eswa.2025.128425>
- Zhang, X., & Gao, W. (2024). Predicting viral rumors and vulnerable users with graph-based neural multi-task learning for infodemic surveillance. *Information Processing & Management*, 61(1), 103520. <https://doi.org/10.1016/j.ipm.2023.103520>
- Zhang, Y., & Chen, L. (2021). A study on forecasting the default risk of bond based on XGBoost algorithm and over-sampling method. *Theoretical Economics Letters*, 11(02), 258–267. <https://doi.org/10.4236/tel.2021.112019>
- Zhang, Y., & Yang, Q. (2017). An overview of multi-task learning. *National Science Review*, 5(1), 30–43. <https://doi.org/10.1093/nsr/nwx105>